

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
“GIOVANNI DEGLI ANTONI”



LAUREA MAGISTRALE IN
SICUREZZA INFORMATICA

TODO: TITOLO DELLA TESI

Autore: Enea Manzi
Matricola: 65935A

Relatore: Prof. Marco Anisetti
Correlatore: Prof. Claudio Agostino Ardagna

A.A. 2025-2026

--

--

--

Abstract

Ringraziamenti

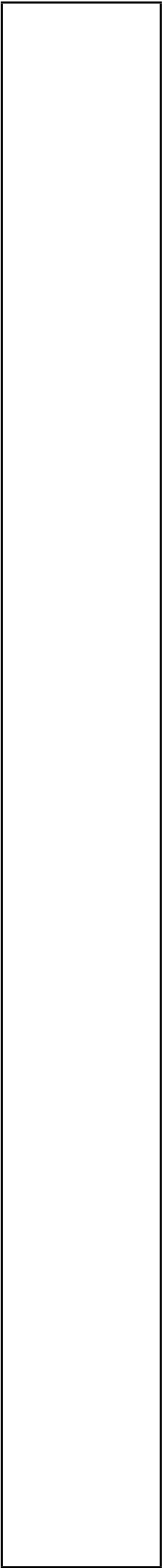
Indice

Abstract	ii
Ringraziamenti	iii
Elenco delle figure	vii
Elenco delle tabelle	viii
Elenco dei codici	ix
1 Introduzione	1
1.1 Motivazione e Contesto	1
1.2 Definizione del Problema e Gap nella Ricerca	2
1.3 Obiettivi della Ricerca	3
1.4 Organizzazione della Tesi	4
2 Background e Stato dell'Arte	6
2.1 Paradigmi delle Architetture API Moderne	6
2.1.1 Evoluzione dei Sistemi Distribuiti	6
2.1.2 Panoramica degli Stili Architettureali di Comunicazione	7
2.1.3 Il Fenomeno dell'API Sprawl	9
2.2 Il Pattern dell'API Gateway in Ottica Zero-Trust	10
2.2.1 Analisi Comparativa dei Modelli di Esposizione in Rete	10
2.2.2 Criteri di Selezione e Giustificazione Architettureale	12
2.2.3 Enforcing del Paradigma Zero-Trust	13
2.3 Tassonomia delle Minacce e Vulnerabilità Logiche	14
2.3.1 Evoluzione del Panorama delle Minacce e Semantic Gap	14
2.3.2 Manifestazioni Concrete del Semantic Gap	15
2.4 L'Evoluzione del Testing di Sicurezza: dall'Analisi Cieca ai Contratti . . .	17
2.4.1 Paradigmi Tradizionali e i Limiti del DAST/SAST	17
2.4.2 Asimmetria Informativa e il Box Gradient	18
2.4.3 Contract-Driven Security e Approcci Ibridi	19

3	Metodologia, Architettura e Principi di Design	23
3.1	Principi Fondamentali: il Perché delle Scelte Architettureali	23
3.1.1	Agnosticismo Applicativo (D1.P1)	24
3.1.2	La Specifica OpenAPI come Fondamento Contrattuale (D3.P2) . .	25
3.1.3	Il Problema dell'Oracolo nel Contract-Driven Security Testing . .	26
3.1.4	Config-Driven Development (D3.P1)	27
3.1.5	Riproducibilità Deterministica (D3.P4)	28
3.2	Architettura dell'Assessment Engine	29
3.2.1	Flusso di Dipendenze Monodirezionale (D1.P2)	29
3.2.2	Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)	30
3.2.3	Split State e Immutabilità (D1.P3)	31
3.2.4	Gateway-Agnostic Adapter (D1.P6)	33
3.2.5	Streaming Evidence Store (D4.P1)	33
3.2.6	Assenza di Dipendenze di Stato Esterno (D3.P5)	34
3.3	Domini di Sicurezza e Box Gradient Applicato	34
3.3.1	Tassonomia degli Otto Domini di Assurance	35
3.3.2	Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)	36
4	Implementazione	38
4.1	La Pipeline di Esecuzione a Sette Fasi e il Core Engine	38
4.1.1	Fail-Safe Error Isolation (D4.P3)	43
4.1.2	Gerarchia delle Eccezioni con Phase Mapping (D4.P4)	43
4.1.3	Costruzione dei Contesti di Esecuzione (D1.P3)	44
4.2	Discovery Dinamico e Risoluzione del Grafo	46
4.2.1	Discovery Dinamico via <code>pkgutil</code> (D2.P1)	46
4.2.2	Scheduling Topologico e <code>DAGScheduler</code> (D2.P2)	48
4.3	Architettura dei Test Nativi e Contratto <code>BaseTest</code>	49
4.3.1	Gerarchia Duale <code>BaseTest</code> / <code>ExternalToolTest</code> (D1.P4)	50
4.3.2	Invarianti di <code>TestResult</code> a Livello di Tipo (D4.P9)	51
4.3.3	Auth Abstraction Layer e Idempotenza dei Token (D2.P5)	52
4.3.4	Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)	52
4.3.5	Oracle a Tre Esiti e Safe HTTP Probing Policy (D4.P7)	53
4.3.6	Path Seed System (D2.P7)	55
4.3.7	Catalogo dei Payload di Attacco (D2.P8)	55
4.3.8	Teardown Best-Effort in Ordine LIFO (D4.P6)	55
4.4	Gerarchia dei Connettori e Integrazione Tool Esterni	56
4.4.1	Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)	56
4.4.2	Lifecycle dei Connector e Dependency Injection (D2.P4)	58
4.4.3	Classificazione A/B dei Connector (D2.P6)	58
4.4.4	Isolamento della Dipendenza AGPL (D7.P2)	59

4.5	Implementazione del Gateway Adapter	59
4.5.1	KongGatewayAdapter: Accesso Read-Only all'Admin API	59
4.5.2	Degradazione Controllata a Tre Livelli (D4.P5)	60
4.6	Evidence Store e Sanitizzazione delle Credenziali	61
4.6.1	Ciclo di Vita dello Streaming Evidence Store (D4.P1)	61
4.6.2	Sanitizzazione Centralizzata delle Credenziali (D4.P2)	62
4.6.3	Dual Audit Trail e Report Domain-Centric (D5.P5)	64
5	Validazione sperimentale e risultati	66
5.1	Topologia del Testbed Sperimentale	66
5.1.1	Composizione e Connettività dell'Ambiente di Test	67
5.1.2	Configurazione del Testbed per la Validazione Funzionale	68
5.1.3	Struttura RBAC e Provisioning degli Utenti	70
5.2	Qualità della Codebase come Prerequisito di Credibilità	70
5.2.1	Analisi Statica della Codebase	71
5.2.2	Dual-Layer Type Safety (D6.P3)	71
5.2.3	Indicatori di Qualità dell'Ingegneria di Produzione	73
5.3	Copertura Funzionale e Idempotenza dell'Assessment	74
5.3.1	Analisi dei Risultati per Dominio	74
5.3.2	Idempotenza e Validazione del Teardown	77
5.4	Footprint Computazionale e Benchmarking del DAG	77
5.4.1	Regime di Esecuzione: Dominato dalla Latenza di Rete	78
5.4.2	Topologia del DAG e Implicazioni sull'Ordinamento dell'Esecuzione	79
	Sintesi della Validazione	80
6	Discussione e Sviluppi Futuri	81
6.1	Analisi Critica e Limiti del Paradigma Contract-Driven	81
6.1.1	Il Paradosso del Ground Truth (D3.P2)	81
6.1.2	La Tensione dell'Astrazione (D1.P6)	82
6.1.3	La Dipendenza dall'Admin API (D4.P5)	82
6.2	Applicabilità Industriale e Integrazione DevSecOps	83
6.2.1	Semantic Exit Codes come Canale di Comunicazione con la Pipeline (D6.P1)	84
6.2.2	Quality Gate a Due Velocità: Fail-Fast e Assessment Completo (D4.P8)	85
6.2.3	Variabilità di Privilegio nei Deployment Industriali	85
6.3	Sviluppi Futuri	86
6.3.1	Estensioni Operative	86
6.3.2	Traiettorie di Ricerca Aperta	88
7	Conclusioni	91
7.1	Sintesi del Lavoro	91
7.2	Risoluzione dei Gap e Verifica degli Obiettivi	92
7.3	Contributi Metodologici e Ingegneristici	93

7.4	Considerazioni Finali sull'Assurance Continuo	93
	Bibliografia	96



Elenco delle figure

Figura 2.1 Il <i>combinatorial wall</i> : fuzzer sintattico (Via A) vs. approccio contract-driven OpenAPI Specification (OAS) (Via B).	18
Figura 3.1 Architettura di APIGuard Assurance: Engine, DAG Scheduler ed EvidenceStore.	29
Figura 4.1 Pipeline a sette fasi: startup bloccante (Fasi 1–4) ed esecuzione non bloccante (Fasi 5–7).	39
Figura 4.2 Layout di <code>src/</code> con mapping alle fasi della pipeline.	40
Figura 4.3 Ciclo di vita di un test in Fase 5: scenario normale e scenario di eccezione con error isolation (D4.P3).	44
Figura 4.4 Gerarchia delle 11 eccezioni custom (D4.P4): bloccanti (Fasi 1–4), recuperate (Fasi 5–6) e Command-Line Interface (CLI)	45
Figura 4.5 Struttura interna del <code>TestContext</code> (D1.P3): canali Token, Teardown Last In, First Out (LIFO) e Shared Data.	46
Figura 4.6 Topologia del Directed Acyclic Graph (DAG): Phase A (16 test), Phase B (<code>depends_on=["1.1"]</code>) e Phase C (pianificata).	49
Figura 4.7 Gerarchia duale <code>BaseTest/ExternalToolTest</code> (D1.P4) e gerarchia dei connettori <code>Library/Subprocess</code> (D1.P5).	50
Figura 4.8 Sanitizzazione a imbuto dell’evidenza (D4.P2): key-pattern, JSON Web Token (JWT) heuristic e header prefix in sequenza.	63
Figura 5.1 Topologia dell’ambiente di validazione sperimentale (Forgejo 14.0.3 + Kong 3.9 DB-less, Docker Compose).	67
Figura 5.2 Mappa di copertura (v0.1.0): test attivi per dominio e relativi esiti. . .	75
Figura 6.1 Routing degli exit code semantici (D6.P1) nella pipeline Continuous Integration / Continuous Deployment (CI/CD).	84
Figura 6.2 Evoluzione di APIGuard Assurance verso una piattaforma modulare: separazione netta tra il core invariante e i plugin di protocollo.	89

Elenco delle tabelle

Tabella 2.1	Confronto dei quattro modelli di esposizione gateway.	11
Tabella 2.2	Uso della specifica OpenAPI e dell'oracolo nei principali approcci contract-driven al testing delle Application Programming Interface (API) REST.	22
Tabella 3.1	Proprietà APIGuard Assurance per dominio.	24
Tabella 3.2	Domini di Assurance di APIGuard Assurance.	35
Tabella 3.3	Box Gradient applicato in APIGuard Assurance.	36
Tabella 5.1	Componenti e versioni dell'ambiente di test.	68
Tabella 5.2	Risultati dell'analisi statica su APIGuard Assurance v0.1.0.	72
Tabella 5.3	Sintesi delle verifiche di qualità sulla codebase.	74
Tabella 5.4	Status e finding count per test.	76
Tabella 5.5	KPI di idempotenza su due run indipendenti.	77
Tabella 5.6	Metriche di sistema sui due run.	78
Tabella 5.7	Topologia del DAG nell'assessment di validazione.	79

Elenco dei codici

Capitolo 1

Introduzione

Le architetture a microservizi cloud-native hanno reso le API REST il principale confine operativo tra sistemi: ogni servizio espone un insieme di endpoint con le proprie regole di autenticazione, autorizzazione e validazione dei dati, e la superficie di attacco complessiva cresce proporzionalmente alla proliferazione di questi endpoint. Valutarne la sicurezza richiede strumenti che conoscano il contratto che ogni richiesta dovrebbe rispettare; gli scanner tradizionali, privi di quella conoscenza, operano sulla sintassi e rimangono ciechi alla classe di vulnerabilità semantica che caratterizza le API REST. Questo lavoro progetta, implementa e valida empiricamente un framework di security assessment che usa la specifica OpenAPI come oracolo formale per costruire verifiche riproducibili e prioritarie su qualsiasi sistema REST documentato, senza dipendenze da un target specifico.

1.1 Motivazione e Contesto

L'adozione delle architetture a microservizi cloud-native ha redistribuito la complessità applicativa su un numero crescente di servizi autonomi, ciascuno responsabile di un sottodominio del sistema e comunicante con gli altri tramite API REST. La conseguenza diretta è la moltiplicazione della superficie esposta: ogni servizio aggiunge endpoint al perimetro, spesso in assenza di un inventario centralizzato e di una governance sistematica del loro ciclo di vita. In questo contesto il gateway API è emerso come punto di enforcement centralizzato: posto tra i client e i servizi interni, concentra in un unico piano di configurazione ispezionabile politiche di autenticazione, autorizzazione, rate limiting e altre funzioni di controllo, tra le molte che un deployment reale può richiedere. Verificare se il gateway applica correttamente le politiche dichiarate e se gli endpoint esposti corrispondono a quelli documentati è parte dell'obiettivo di un sistema di security assessment; ma le vulnerabilità rilevanti non si limitano alla configurazione del piano

di controllo. Molte emergono interrogando la logica semantica degli endpoint stessi, in modo indipendente dal gateway che li espone.

L'Open Web Application Security Project (OWASP) API Security Top 10 [1] formalizza le dieci categorie di rischio più frequenti nelle API REST, con le prime posizioni occupate da vulnerabilità semantiche, tra cui violazioni di autorizzazione a livello di oggetto (Broken Object Level Authorization (BOLA)), difetti di autenticazione ed esposizione non intenzionale di proprietà. Queste vulnerabilità si manifestano come richieste formalmente corrette che violano le regole che il sistema impone su chi può accedere a cosa: quali risorse un client autenticato può leggere o modificare, con quali credenziali, e in quale sequenza di operazioni; per questa ragione non producono anomalie sintattiche né nella forma della richiesta né nella struttura della risposta, e sfuggono agli strumenti che cercano esclusivamente questi segnali. La frequenza di violazioni correlata a configurazioni errate dei gateway e a controlli di autorizzazione insufficienti, già documentata nelle prime posizioni del framework OWASP API Security Top 10 citato sopra, segnala un disallineamento strutturale tra la velocità con cui le API vengono esposte e la maturità degli strumenti disponibili per valutarne la sicurezza.

Questa complessità semantica rende difficile affiancare al ciclo di sviluppo continuativo una valutazione sistematica della sicurezza. Un penetration test manuale produce un'osservazione approfondita ma circoscritta al momento e al contesto in cui viene condotto; non si presta a essere ripetuto automaticamente a ogni variazione del target, né a produrre risultati confrontabili tra esecuzioni successive senza intervento umano. Dotare il processo di assessment di riproducibilità deterministica, tracciabilità dei verdetti agli standard normativi e integrabilità nelle pipeline di sviluppo continuativo richiede che le verifiche siano parametrizzate sul contratto del target e indipendenti dall'operatore che le esegue; solo a quella condizione possono essere eseguite in forma autonoma su qualsiasi sistema documentato.

1.2 Definizione del Problema e Gap nella Ricerca

Tra le limitazioni degli approcci esistenti al security assessment delle API REST, tre si sono rivelate centrali per questo lavoro: ciascuna è radicata in una scelta di design diversa e porta implicazioni architetturali distinte.

G₁ Cecità Semantica Gli scanner Dynamic Application Security Testing (DAST) e i fuzzer generici operano sulla sintassi delle richieste HyperText Transfer Protocol (HTTP): generano payload, osservano i codici di risposta e cercano anomalie nella forma della request (come le injection sui parametri) o nella struttura della response (come codici di errore inattesi). Le vulnerabilità semantiche delle API REST si manifestano invece

come richieste sintatticamente corrette che violano le regole di accesso del sistema: quali risorse un client può richiedere, con quali credenziali, rispetto a quali altri soggetti. Per rilevarle è necessario conoscere il contratto che regola ogni interazione, ad esempio quali ruoli possono accedere a quale risorsa, quale sia il ciclo di vita atteso di un token, e quali sequenze di operazioni abbiano senso semantico. Uno scanner privo di quella conoscenza non può costruire sistematicamente le probe che raggiungono questa logica. Una revisione sistematica della letteratura sul testing delle API REST documenta che questa classe di verifiche rimane tra le sfide aperte più frequentemente citate, affrontata esplicitamente da meno del 10% dei contributi analizzati [2].

G₂ Portabilità e Profondità Semantica Gli strumenti specializzati di security assessment per API presentano un compromesso ricorrente: quelli progettati per un gateway o una piattaforma specifica offrono verifiche approfondite ma sono applicabili solo a quel contesto; quelli sufficientemente generici da operare su qualsiasi target, usano la OAS principalmente per generare richieste valide dal punto di vista sintattico, senza impiegarla come oracolo per valutare se le risposte rispettino il contratto di sicurezza dichiarato. Il risultato è che profondità e portabilità vengono ottimizzate separatamente, e la specifica del target costituisce raramente l'oracolo condiviso tra le due esigenze. Questo riflette anche un limite dell'ecosistema: l'adozione di specifiche formali è ancora disomogenea nel panorama reale, e quando esistono non sempre riflettono fedelmente l'implementazione effettiva, rendendo difficile affidarsi a esse come fonte autoritativa per la valutazione della sicurezza.

G₃ Riproducibilità Deterministica Un assessment condotto da un operatore in un momento specifico produce un'osservazione il cui esito dipende tanto dalle caratteristiche del target quanto dalle scelte metodologiche di chi lo ha condotto. In un processo di sviluppo continuativo, dove il sistema evolve costantemente e ogni modifica può introdurre regressioni di sicurezza, questa dipendenza dall'operatore impedisce di affiancare al ciclo di rilascio una verifica automatica e confrontabile nel tempo: un operatore umano non può seguire la frequenza dei cambiamenti, e la mancanza di determinismo nell'output rende impossibile distinguere una regressione reale da una variazione metodologica. Senza un meccanismo che renda il comportamento del tool indipendente dal contesto di esecuzione e verificabile su run successive, la riproducibilità rimane un'assunzione implicita piuttosto che una garanzia dimostrabile.

1.3 Obiettivi della Ricerca

Ai tre gap identificati corrispondono tre obiettivi di ricerca. Ciascuno è formulato attorno a una proprietà concreta che le scelte architetturali realizzano e che la validazione empirica può valutare.

O₁ Tool Contract-Driven Agnostico Il primo obiettivo, in risposta al G₁ Cecità Semantica, è progettare e implementare un tool che adotti il paradigma contract-driven: la OAS del target funge da oracolo formale per costruire verifiche semanticamente significative, mentre il file di configurazione fornisce la conoscenza concreta del deployment specifico, come i valori degli endpoint parametrici e le credenziali dei ruoli. La combinazione dei due mira a eliminare la necessità di riferimenti hardcoded a qualsiasi sistema specifico e a organizzare le verifiche in una tassonomia strutturata di domini di sicurezza con priorità esplicite; ogni test sarà corredato dai relativi riferimenti normativi, producendo evidenza tracciabile verso categorie OWASP, codici Common Weakness Enumeration (CWE) e standard applicabili.

O₂ Superamento del Compromesso Portabilità-Profondità Il secondo obiettivo, in risposta al G₂ Portabilità e Profondità Semantica, è una conseguenza diretta dell'approccio adottato in O₁ Tool Contract-Driven Agnostico: un tool che deriva tutta la conoscenza del target dalla specifica e dalla configurazione, senza dipendenze hardcoded, è per costruzione applicabile a qualsiasi sistema REST documentato senza modifiche al codice. La profondità semantica delle verifiche non dipende quindi dalla piattaforma su cui il target è deployato, ma dalla qualità della specifica che lo descrive.

O₃ Determinismo e Integrazione nel Ciclo Continuativo Il terzo obiettivo, in risposta al G₃ Riproducibilità Deterministica, è progettare il tool in modo che esecuzioni successive con la stessa configurazione sullo stesso target producano output identici, indipendentemente dall'operatore, e verificare questa proprietà empiricamente. Il determinismo dell'output è la condizione che rende la verifica affiancabile al ciclo di sviluppo continuativo tramite exit code semantici, senza richiedere interpretazione manuale dei risultati tra un'esecuzione e la successiva.

1.4 Organizzazione della Tesi

La tesi è organizzata nei seguenti capitoli.

Nel **Capitolo 2** viene costruito il contesto tecnico che i capitoli successivi presuppongono. Si analizzano l'evoluzione verso le architetture a microservizi e i principali stili di comunicazione con le loro implicazioni di sicurezza; il ruolo del gateway API come punto di enforcement centralizzato e i criteri che ne motivano la scelta; il divario strutturale tra gli strumenti di testing tradizionali e le vulnerabilità semantiche delle API REST; e l'evoluzione degli approcci di testing dal fuzzing cieco agli strumenti contract-driven.

Nel **Capitolo 3** vengono presentati la metodologia e l'architettura del tool, con un approccio top-down che parte dal livello di sistema per scendere ai componenti. Si

introducono agnosticismo applicativo, paradigma contract-driven e riproducibilità deterministica come fondamenta del design; si descrive l'architettura dell'assessment engine con il suo scheduling basato su DAG, e si definisce la tassonomia degli otto domini di sicurezza con il box gradient come mapping tra precondizioni di accesso e profondità dell'assessment.

Nel **Capitolo 4** le proprietà architettureali del capitolo precedente vengono portate al livello implementativo. Si descrive la pipeline di esecuzione a sette fasi, il meccanismo di discovery dinamico dei test, il contratto delle classi base `BaseTest` ed `ExternalToolTest`, la gerarchia dei connector per l'integrazione di tool esterni, e i meccanismi di gestione sicura delle credenziali e di streaming dell'evidence store.

Nel **Capitolo 5** viene condotta la validazione sperimentale del sistema su un target reale: Forgejo 14.0.3 esposto tramite Kong 3.9 in modalità DB-less. La validazione si articola su quattro livelli: la composizione del testbed, la qualità ingegneristica del codebase come prerequisito di affidabilità dello strumento di misura, la copertura funzionale e l'idempotenza su due run indipendenti, e infine il profilo computazionale dell'assessment.

Nel **Capitolo 6** i limiti strutturali del paradigma contract-driven vengono ricondotti alle scelte architettureali che li originano; si analizzano le condizioni concrete di integrabilità nelle pipeline CI/CD industriali e la variabilità di copertura nei diversi contesti di deployment; e infine si tracciano le traiettorie di sviluppo futuro distinte in estensioni operative, che seguono direttamente dall'architettura corrente, e in direzioni di ricerca, che richiedono validazione sperimentale indipendente.

Nel **Capitolo 7** vengono tratte le conclusioni della tesi. Per ciascuno dei tre gap identificati nell'introduzione si verifica come l'obiettivo corrispondente sia stato raggiunto, valutandone la portata effettiva e collocando il contributo nel contesto più ampio della sicurezza come processo continuo e verificabile.

Capitolo 2

Background e Stato dell'Arte

Le scelte architetturali e metodologiche dei capitoli successivi presuppongono un contesto tecnico preciso: un ecosistema in cui la logica applicativa è distribuita su reti di servizi autonomi che comunicano tramite API, esposti attraverso gateway che concentrano l'enforcement delle policy di sicurezza. Questo capitolo costruisce quel contesto in quattro passi. La Sezione 2.1 analizza l'evoluzione verso le architetture a microservizi, i protocolli di comunicazione dominanti e il fenomeno della proliferazione incontrollata degli endpoint. La Sezione 2.2 esamina i modelli di esposizione in rete disponibili, argomenta la scelta del gateway enterprise come punto di enforcement e analizza le implicazioni del paradigma Zero-Trust. La Sezione 2.3 classifica le categorie di vulnerabilità semantica rilevanti per le API REST e mostra perché la loro eterogeneità richiede approcci di valutazione distinti. La Sezione 2.4 ripercorre l'evoluzione degli strumenti di testing, dal fuzzing cieco agli approcci contract-driven, identificando i gap strutturali che questa tesi affronta.

2.1 Paradigmi delle Architetture API Moderne

Questa sezione ripercorre l'evoluzione architetturale che ha portato alla proliferazione delle API REST come interfaccia primaria tra sistemi (Sezione 2.1.1), analizza i principali stili di comunicazione e le loro implicazioni di sicurezza (Sezione 2.1.2), e descrive il fenomeno dell'API sprawl come conseguenza diretta della proliferazione (Sezione 2.1.3).

2.1.1 Evoluzione dei Sistemi Distribuiti

Per decenni, l'architettura dominante nello sviluppo enterprise ha concentrato tutta la logica applicativa in un singolo processo deployabile: il monolite. In questo modello, i moduli del sistema condividono lo stesso spazio di memoria e comunicano attraverso chiamate di funzione locali, il che semplifica le transazioni interne, rende il debugging

lineare e consente un toolchain di sviluppo consolidato. La semplicità operativa ha un costo che diventa evidente con la crescita: scalare il sistema significa replicare l'intero processo, indipendentemente da quale sottodominio funzionale sia effettivamente sotto carico, e ogni modifica richiede il redeployment dell'intera applicazione, con un rischio di regressione che cresce proporzionalmente alla dimensione della codebase. Questi limiti hanno spinto l'industria verso un modello alternativo che decompone il sistema lungo le linee del dominio di business.

L'architettura a microservizi risponde a questi limiti decomponendo il sistema in servizi autonomi, ciascuno responsabile di un sottodominio delimitato del business e deployabile o scalabile indipendentemente dagli altri [3]. Ogni servizio può essere sviluppato, rilasciato e scalato secondo le proprie esigenze, con team indipendenti che possono scegliere il linguaggio e le tecnologie più adatte al proprio sottodominio. La granularità del deployment consente di allocare risorse in modo proporzionale al carico reale di ciascuna funzione, eliminando la necessità di replicare l'intera applicazione per scalare un singolo componente.

La decomposizione ridistribuisce la complessità senza eliminarla: ogni servizio è più semplice del monolite che sostituisce, ma il sistema nel suo complesso introduce una nuova categoria di problemi assente nel modello monolitico. La comunicazione tra servizi, che nel monolite avveniva attraverso chiamate locali in memoria, avviene ora attraverso interfacce programmabili esposte in rete, ciascuna con il proprio contratto, il proprio ciclo di vita e la propria superficie di attacco. Il numero di queste interfacce cresce più che proporzionalmente rispetto alla dimensione del sistema, perché ogni componente aggiuntivo introduce non solo la propria interfaccia esterna ma anche i canali di comunicazione verso le dipendenze che richiede; ciascuno di questi canali è potenzialmente raggiungibile, interrogabile e abusabile in modo indipendente dal contesto applicativo in cui opera.

2.1.2 Panoramica degli Stili Architetture di Comunicazione

Le interfacce tra servizi non seguono tutte lo stesso modello: stili architetture diversi espongono superfici di attacco strutturalmente diverse, e comprendere queste differenze è il presupposto per valutare quali strumenti e approcci siano applicabili in ciascun contesto. La rassegna che segue presenta i principali stili in uso nelle architetture cloud-native, con attenzione sia al loro funzionamento che alle implicazioni di sicurezza che ne derivano.

REST Representational State Transfer (REST), formalizzato da Fielding nella sua dissertazione del 2000 [4], è lo stile architetture dominante nelle architetture cloud-native. È basato su risorse identificate da Uniform Resource Identifier (URI) e manipolate tramite i verbi HTTP standard (GET, POST, PUT, DELETE, PATCH); ogni richiesta è

stateless e autocontenuta. Una proprietà fondamentale del modello REST è la separazione tra l'identità di una risorsa, espressa dal suo URI, e la sua rappresentazione: lo stesso documento può essere restituito in formato JSON, XML o HTML a seconda di come il client negozia il tipo di contenuto, senza che l'identità della risorsa cambi. Questa semplicità strutturale, combinata con l'adozione pervasiva di HTTP come protocollo di trasporto, ne ha fatto il modello predefinito per le API pubbliche e per la comunicazione inter-servizio nelle architetture moderne. Sul piano della sicurezza, la struttura resource-oriented introduce una responsabilità che il protocollo non gestisce: l'identificatore della risorsa, specificato dal client, è tipicamente incluso nel path della richiesta, e la verifica che il chiamante abbia il diritto di accedere a quella specifica risorsa è interamente demandata alla logica applicativa. Questa delega è la radice strutturale delle vulnerabilità di autorizzazione a livello di oggetto descritte nella Sezione 2.3.2.

gRPC Google Remote Procedure Calls (gRPC) è un framework per chiamate di procedura remota sviluppato da Google, progettato per la comunicazione ad alta efficienza tra servizi. Le operazioni sono metodi nominati e tipizzati, descritti in file `.proto`; i messaggi sono serializzati in formato binario tramite Protocol Buffers e trasportati su HTTP/2, che supporta multiplexing di stream e comunicazione bidirezionale. Queste caratteristiche lo rendono particolarmente adatto alla comunicazione inter-servizio dove latenza e throughput sono critici. La serializzazione binaria, che è anche la fonte delle sue prestazioni, rende il traffico opaco agli strumenti di analisi che operano su payload testuali: senza un decoder Protocol Buffers, il contenuto dei messaggi non è leggibile, il che limita l'applicabilità delle tecniche di inspection tradizionali.

GraphQL Graph Query Language (GraphQL) è un linguaggio di query per API sviluppato da Facebook e rilasciato come open source nel 2015. A differenza di REST, espone un singolo endpoint che accetta query arbitrarie su un grafo di dati tipizzato: il client dichiara esattamente i campi che vuole ottenere, compresi i percorsi di attraversamento del grafo, e il server restituisce solo quelli. A differenza di REST, dove ogni endpoint restituisce una struttura fissa indipendentemente da ciò che il client richiede, GraphQL consente al client di dichiarare esattamente i campi necessari, eliminando il trasferimento di dati superflui. Sul piano dell'evoluzione del contratto, REST gestisce i cambiamenti attraverso versioni esplicite degli endpoint; in GraphQL è invece possibile aggiungere nuovi campi allo schema senza modificare le query esistenti, permettendo un'evoluzione graduale che non richiede aggiornamenti sincronizzati dei client. Le implicazioni di sicurezza sono però strutturalmente diverse da REST: il meccanismo di introspezione, se non disabilitato, espone l'intera struttura dello schema a chiunque possa inviare una query; la flessibilità delle query consente il batching di centinaia di operazioni in una singola richiesta HTTP, aggirando rate limiting configurato per conteggio di richieste; la profondità di attraversamento del grafo può essere sfruttata per query ricorsive che saturano le risorse del server.

SOAP Simple Object Access Protocol (SOAP) è un protocollo per lo scambio di messaggi strutturati tra sistemi, basato su Extensible Markup Language (XML) e su contratti formali descritti tramite Web Services Description Language (WSDL). Rimane diffuso in contesti enterprise legacy, in particolare nei settori finanziario e della pubblica amministrazione, dove è stato adottato prima della diffusione di REST e dove la migrazione verso stili più moderni è limitata dai costi di integrazione con sistemi esistenti. Il modello di elaborazione basato su XML introduce vulnerabilità strutturali indipendenti dall'applicazione: l'XML External Entity (XXE) injection sfrutta la capacità del parser di risolvere entità esterne per esfiltrare file di sistema o effettuare richieste verso reti interne, mentre le XML Bomb sfruttano l'espansione ricorsiva di entità per saturare la memoria del parser. Queste vulnerabilità non derivano dall'implementazione specifica ma dalla specifica XML stessa.

WebSocket e Server-Sent Events WebSocket e Server-Sent Events (SSE) sono protocolli progettati per la comunicazione in tempo reale. WebSocket stabilisce un canale bidirezionale persistente tra client e server, avviato tramite un handshake HTTP e poi indipendente dal ciclo request-response; è ampiamente usato per applicazioni collaborative, notifiche push e streaming di dati. SSE offre invece uno streaming unidirezionale dal server al client su una connessione HTTP mantenuta aperta, più semplice da implementare per casi d'uso di sola ricezione. Entrambi i modelli pongono problemi specifici per l'enforcement della sicurezza: i controlli applicati sull'handshake iniziale, come autenticazione e rate limiting, non si estendono automaticamente ai frame successivi della connessione persistente, lasciando il canale post-handshake fuori dal perimetro di ispezione del gateway. Nel caso di WebSocket, la mancata verifica dell'`Origin` header espone il servizio ad attacchi di hijacking della connessione; nel caso di SSE, le connessioni di lunga durata con token a scadenza possono esaurire le risorse disponibili se la scadenza non viene gestita correttamente lato server.

Questa tesi si focalizza su API REST documentate tramite OAS: è lo stile architetturale più adottato nelle architetture a microservizi cloud-native e quello per cui esistono specifiche formali sufficientemente strutturate da essere usate come oracolo di assessment.

2.1.3 Il Fenomeno dell'API Sprawl

La decomposizione in microservizi, combinata con cicli di sviluppo rapidi e team decentralizzati, produce un effetto collaterale documentato nel panorama della sicurezza: la proliferazione incontrollata di endpoint esposti che sfugge a qualsiasi inventario centralizzato, un fenomeno noto come *API sprawl*. Il framework OWASP API Security Top 10 [1] lo formalizza come categoria di rischio autonoma (API9:2023, Improper Inventory Management), riconoscendone la rilevanza sistemica indipendentemente dalle vulnerabilità

specifiche dei singoli endpoint.

Il fenomeno si articola in tre categorie distinte per origine e natura del problema. Le *Shadow API* sono endpoint attivi ma assenti da qualsiasi specifica pubblicata o inventario ufficiale: emergono da processi di sviluppo che bypassano la governance documentale, da integrazioni non autorizzate tra team, o da endpoint mai formalmente registrati nel catalogo aziendale. Le *Zombie API* sono endpoint logicamente dismessi, rimossi dalla specifica corrente o ufficialmente deprecati, che continuano a rispondere in produzione perché la dismissione tecnica non ha seguito quella logica; spesso mantengono configurazioni di sicurezza obsolete e librerie non aggiornate. Le *Deprecated API* sono endpoint ancora presenti nella specifica con marcatore di obsolescenza, formalmente visibili ma privi di una data di dismissione programmata, che continuano a essere raggiunti da client non aggiornati. La medesima tassonomia è riconosciuta a livello normativo dal NIST SP 800-228 [5], nella Sezione 2.1, che identifica shadow API, zombie API e API orfane come fattori di rischio primari per la protezione delle API cloud-native, riconducendone l'origine alla frammentazione organizzativa dei processi di sviluppo.

Tutte e tre le categorie condividono una proprietà rilevante per il security assessment: non sono raggiungibili dagli strumenti che costruiscono la propria conoscenza del target esclusivamente dalla documentazione disponibile. Uno strumento che enumera la superficie di attacco dalla specifica pubblicata non può osservare endpoint che quella specifica non dichiara; per rilevare questa classe di esposizione occorre confrontare la superficie effettivamente raggiungibile con quella dichiarata, identificando la discrepanza tra le due.

2.2 Il Pattern dell'API Gateway in Ottica Zero-Trust

La moltiplicazione delle interfacce tra servizi pone il problema di come esporle in modo governabile e sicuro. Questa sezione analizza i modelli di esposizione disponibili (Sezione 2.2.1) e argomenta la scelta del gateway API dedicato come punto di enforcement centralizzato (Sezione 2.2.2), infine ne inquadra il ruolo nel contesto del paradigma Zero-Trust (Sezione 2.2.3).

2.2.1 Analisi Comparativa dei Modelli di Esposizione in Rete

Le architetture a microservizi adottano modelli diversi per esporre i propri servizi al traffico esterno e per governare il traffico interno. Quattro di questi, per diffusione e rilevanza nel contesto del security assessment, meritano un'analisi distinta: ciascuno posiziona il perimetro di sicurezza in modo diverso e impone vincoli diversi sulla verificabilità della configurazione, come sintetizzato nella Tabella 2.1. Il NIST SP 800-228 [5], nella Sezione 4.

formalizza una tassonomia analoga che distingue tre pattern: gateway centralizzato, ibrido e distribuito. I modelli analizzati in questa sezione vi si collocano in modo diretto: il gateway API dedicato e il gateway managed cloud corrispondono al pattern centralizzato; il service mesh corrisponde al pattern distribuito; le soluzioni Kubernetes Ingress e Gateway API si collocano in una posizione intermedia, con un punto di ingresso centralizzato ma politiche applicative distribuite ai servizi, avvicinandosi al pattern ibrido.

Modello	Traffico presidiato	Enforcement policy	Accesso al piano di con- figurazione	Verifica WHITE_BOX
Gateway API Dedicato (es. Kong, Apigee)	Nord-sud	Centralizzato (singolo strato)	Admin API diretta	Nativa
Kubernetes Ingress / Gateway API	Nord-sud	Centralizzato (via K8s API)	kubect1 / Kubernetes API	Controller- dipendente
Service Mesh (es. Istio)	Est-ovest	Distribuito per sidecar proxy	Control plane (istiod)	N/A (est-ovest)
Gateway Managed Cloud (AWS, Azure, GCP)	Nord-sud	Centralizzato dal provider	SDK / API del cloud provider	Adapter dedicato

Tabella 2.1: Confronto dei quattro modelli di esposizione gateway.

Gateway API Dedicato Il gateway API dedicato è un componente software il cui scopo specifico è gestire il traffico nord-sud¹ che proviene da client esterni verso i servizi interni. Prodotti come Kong, Apigee o AWS API Gateway appartengono a questa categoria: si frappongono tra il client e il backend e concentrano in un unico strato le funzioni trasversali che altrimenti andrebbero replicate in ogni servizio, tra cui autenticazione e autorizzazione, rate limiting, trasformazione dei payload e terminazione Transport Layer Security (TLS). La centralizzazione rende il gateway il punto naturale in cui applicare le policy di sicurezza prima che le richieste raggiungano la logica applicativa; è anche il punto in cui un errore di configurazione produce conseguenze sistemiche, non localizzate a un singolo servizio.

Kubernetes Ingress e Gateway API In ambienti containerizzati orchestrati con Kubernetes, il traffico nord-sud è gestito tramite risorse dichiarative native alla piattaforma. Il modello Ingress, disponibile fin dalle prime versioni di Kubernetes, consente di definire regole di routing basate su hostname e path che un Ingress Controller, tipicamente un reverse proxy come Nginx o Traefik, traduce in configurazione operativa. Il limite di que-

¹Con *traffico nord-sud* si intende il traffico tra client esterni e i servizi interni del sistema, contrapposto al traffico est-ovest che designa le comunicazioni tra servizi all'interno del perimetro.

sto modello è la dipendenza da annotation controller-specifiche per funzionalità avanzate, che producono configurazioni strettamente legate al controller scelto. La Kubernetes Gateway API, diventata standard stabile nel 2023, affronta questo problema con risorse tipizzate e interoperabili tra controller diversi, separando esplicitamente i ruoli di chi gestisce l'infrastruttura da chi configura il routing applicativo. Entrambe le soluzioni operano al livello del traffico HTTP in ingresso al cluster; i protocolli che stabiliscono canali persistenti dopo l'handshake iniziale, come WebSocket, richiedono configurazioni specifiche del controller e possono essere gestiti in modo disomogeneo tra le diverse implementazioni di Ingress.

Service Mesh Il service mesh è un'infrastruttura dedicata alla gestione del traffico est-ovest, ovvero le comunicazioni tra i servizi interni. Il modello prevalente, esemplificato da Istio², affianca a ogni istanza di servizio un proxy che intercetta tutto il traffico in modo trasparente, senza richiedere modifiche al codice applicativo. Questo meccanismo abilita funzionalità trasversali come la mutua autenticazione TLS tra servizi, il circuit breaking e la raccolta di metriche di rete. Il service mesh presidia le comunicazioni interne al perimetro; il traffico proveniente dall'esterno rimane responsabilità di un gateway o di una soluzione di ingress separata.

Gateway Managed in Ambienti Cloud Nei deployment serverless e nelle piattaforme cloud gestite, il traffico verso le funzioni o i servizi è gestito da un gateway che la piattaforma stessa fornisce e opera: AWS API Gateway, Azure API Management e Google Cloud Endpoints sono esempi di questa categoria. Il principale vantaggio è l'eliminazione della gestione infrastrutturale: scaling automatico, alta disponibilità e aggiornamenti di sicurezza sono responsabilità del provider. Sul piano dell'accesso alla configurazione, però, questo modello si differenzia significativamente dai gateway dedicati: il piano di controllo è accessibile solo attraverso le API del cloud provider, con meccanismi di autenticazione e autorizzazione specifici della piattaforma, e non tramite un'interfaccia amministrativa diretta e ispezionabile. Questa caratteristica impone vincoli specifici sulle tipologie di verifica applicabili in sede di assessment.

2.2.2 Criteri di Selezione e Giustificazione Architetturale

Il gateway API dedicato è il modello più adottato nelle architetture che richiedono governance centralizzata del traffico esterno. La sua diffusione è motivata da tre proprietà tecniche che gli altri modelli offrono solo parzialmente.

²Istio è il service mesh open source più diffuso nelle architetture Kubernetes. Utilizza Envoy come proxy sidecar: un proxy ad alte prestazioni che opera come intermediario trasparente per il traffico di rete, senza richiedere modifiche al codice applicativo.

La prima è la concentrazione dell'enforcement in un unico punto ispezionabile. Un gateway dedicato applica le policy di autenticazione, autorizzazione e rate limiting prima che le richieste raggiungano i servizi backend, e rende questa configurazione accessibile tramite un'interfaccia amministrativa diretta. Nei modelli serverless l'accesso alla configurazione è mediato dalle API del provider cloud; nei service mesh è distribuita tra sidecar e control plane. In entrambi i casi accedere alla configurazione effettiva richiede strumenti e accessi specifici alla piattaforma, mentre l'interfaccia amministrativa di un gateway dedicato è indipendente dal contesto di deployment.

La seconda è la separazione tra l'applicazione delle policy e la logica dei singoli servizi. Il gateway può essere configurato e verificato indipendentemente dalle applicazioni che ospita: aggiungere un vincolo di autenticazione su un endpoint non richiede modifiche all'applicazione, e verificare che quel vincolo sia effettivamente attivo non richiede conoscenza della logica interna del servizio. Questa separazione consente di valutare i meccanismi di sicurezza del gateway come strato autonomo, indipendentemente dal comportamento specifico di ciascun backend.

La terza è il ruolo del gateway come confine documentato dell'interfaccia esposta. Il gateway dedicato è il punto in cui le applicazioni rendono accessibile la propria OAS: la specifica descrive quali endpoint esistono, quali parametri accettano e quali risposte producono, e il gateway la pubblica o la riferisce come parte della propria configurazione. Questa disponibilità è la condizione che rende possibile costruire verifiche sistematiche dell'interfaccia esposta, derivando la conoscenza del target dalla specifica anziché dall'esplorazione empirica.

2.2.3 Enforcing del Paradigma Zero-Trust

Il paradigma Zero-Trust, formalizzato nel NIST SP 800-207 [6], ridefinisce il perimetro di sicurezza spostando il punto di controllo dalla rete all'identità: nessuna richiesta è considerata attendibile per via della sua provenienza da una rete ritenuta fidata, e ogni accesso è valutato in modo indipendente rispetto all'identità del chiamante, al contesto della richiesta e ai permessi sulla risorsa specifica. Applicato alle API REST, il principio si traduce in tre requisiti operativi: autenticazione obbligatoria su ogni endpoint, autorizzazione valutata per ogni coppia chiamante-risorsa, e assenza di fiducia implicita derivante dalla posizione di rete del client. Il NIST SP 800-228 [5], nella Sezione 1, declina questi principi nel contesto specifico delle API cloud-native.

L'API gateway è il punto architetturale in cui questi requisiti trovano enforcement naturale: si colloca tra il client e il backend, ha visibilità su ogni richiesta in transito, e può rifiutare le richieste che non soddisfano i controlli configurati prima che raggiungano la logica applicativa. La verifica che questo enforcement sia effettivamente attivo e correttamente configurato è esattamente il perimetro del framework proposto: le garanzie

che il gateway dichiara nella propria configurazione (autenticazione richiesta su tutti gli endpoint, rate limiting attivo, header di sicurezza iniettati) devono corrispondere al comportamento osservabile a runtime, e questa corrispondenza non è verificabile senza un assessment sistematico. La distanza tra configurazione dichiarata e comportamento effettivo è il gap che un sistema di assessment sistematico deve colmare: verificare che le garanzie dichiarate nella configurazione del gateway corrispondano al comportamento effettivamente osservabile a runtime è il perimetro che motiva l'approccio adottato nei capitoli successivi.

2.3 Tassonomia delle Minacce e Vulnerabilità Logiche

La transizione verso le architetture distribuite ha modificato non solo la superficie di attacco ma anche la natura delle vulnerabilità più rilevanti. Questa sezione analizza il cambiamento partendo dall'evoluzione del panorama delle minacce e dal divario strutturale tra strumenti tradizionali e vulnerabilità semantiche (Sezione 2.3.1); ne illustra successivamente alcune manifestazioni concrete per mostrare perché la loro eterogeneità richiede approcci di valutazione differenziati (Sezione 2.3.2).

2.3.1 Evoluzione del Panorama delle Minacce e Semantic Gap

I sistemi monolitici tradizionali concentravano la logica applicativa in un processo unico, con una superficie di attacco relativamente compatta e strumenti di testing consolidati. La decomposizione in microservizi ha modificato questa equazione: moltiplicando i confini tra componenti, ha spostato parte della responsabilità di sicurezza dagli strati infrastrutturali alla logica applicativa di ogni singolo servizio. Le vulnerabilità più rilevanti nel panorama attuale non si manifestano nella forma delle richieste, ma nelle relazioni tra identità, risorse e operazioni che il sistema presuppone.

Questa evoluzione ha messo in evidenza un divario strutturale tra gli strumenti di testing tradizionali e la classe di vulnerabilità più diffusa nelle API REST moderne. Un Web Application Firewall (WAF) intercetta una SQL injection perché riconosce nel payload un pattern sintattico noto; la stessa infrastruttura non rileva una richiesta `GET /users/456` inviata da un utente autorizzato ad accedere solo a `/users/123`, perché quella violazione non si esprime nella forma della richiesta ma nella relazione tra l'identità del chiamante e la risorsa richiesta. Il NIST SP 800-228 [5], nella Sezione 4.4.1, formalizza questo limite nel contesto delle API cloud-native: un WAF può scansionare i payload alla ricerca di pattern di attacco noti, ma non può operare al livello semantico dell'API, verificando ad esempio che un campo rispetti i vincoli dichiarati dal contratto o che l'identità del chiamante abbia i diritti sulla risorsa specifica richiesta. La distanza tra ciò che uno strumento può osservare e ciò che una vulnerabilità semantica richiede di comprendere

costituisce il *semantic gap*: il divario tra la forma della transazione HTTP e la semantica del contratto che quella transazione dovrebbe rispettare. Questa classe di vulnerabilità non è raggiungibile da strumenti il cui oracolo si limita ai codici di stato HTTP: un oracolo di questo tipo non può rilevare vulnerabilità che non si manifestino in quei codici, tra cui l'esposizione non autorizzata di informazioni [7]. Il riconoscimento di questo limite ha motivato l'introduzione di oracoli formali per classi specifiche di violazioni semantiche, verificabili automaticamente su sequenze di richieste [8].

Il framework OWASP API Security Top 10 nella sua edizione 2023 [1] documenta questa transizione: le prime categorie di rischio, tra cui Broken Object Level Authorization (API1), Broken Authentication (API2), Broken Object Property Level Authorization (API3) e Broken Function Level Authorization (API5), sono tutte vulnerabilità semantiche. La loro manifestazione comune è una richiesta sintatticamente corretta che viola le assunzioni del sistema sull'identità, i privilegi o il comportamento atteso del chiamante.

Colmare il semantic gap richiede che lo strumento disponga di una rappresentazione formale di ciò che il sistema dichiara di fare, da usare come criterio per valutare se le risposte osservate siano conformi o meno. La OAS è una di queste rappresentazioni: descrive la struttura degli endpoint, i tipi e i vincoli dei parametri, e gli schemi di risposta attesi. Va però sottolineato che la specifica descrive il contratto sintattico e strutturale dell'interfaccia, non la semantica applicativa completa: stabilisce cosa un endpoint dovrebbe accettare e restituire, ma non chi ha il diritto di invocarlo e su quale risorsa. Questa conoscenza supplementare, che comprende ruoli, identità e relazioni tra risorse, deve essere fornita separatamente, tipicamente attraverso la configurazione del sistema di assessment. Esistono approcci alternativi per affrontare il semantic gap, come l'inferenza della specifica dall'analisi del traffico reale o il testing differenziale; l'approccio basato su specifica formale, quando la specifica è disponibile e accurata, offre la maggiore sistematicità nella costruzione delle verifiche.

2.3.2 Manifestazioni Concrete del Semantic Gap

Le vulnerabilità semantiche descritte nella sezione precedente si manifestano in forme concrete e distinte, ciascuna con precondizioni di rilevazione diverse. Esaminarne alcune permette di rendere tangibile perché il testing sintattico non raggiunga questa classe di problemi.

Il Broken Object Level Authorization (BOLA, API1:2023) si manifesta quando un utente autenticato accede a risorse di altri utenti sostituendo l'identificatore nel path della richiesta. La richiesta è sintatticamente corretta; il fallimento risiede nell'assenza del controllo sulla coppia chiamante-oggetto. Rilevarlo richiede credenziali per almeno due account distinti e la capacità di confrontare le risposte, perché il problema emerge solo dall'osservazione di ciò che due identità diverse ottengono sulla stessa risorsa. La proprietà

violata, formalizzata come *user-namespace rule* in [8] e oggetto di oracoli automatizzati in letteratura [9], stabilisce che una risorsa creata nel namespace di un utente non deve essere accessibile da un namespace distinto. Il Broken Function Level Authorization (Broken Function Level Authorization (BFLA), API5:2023) segue la stessa logica a livello di operazione: un utente con ruolo ristretto invoca endpoint riservati a ruoli privilegiati perché i controlli di autorizzazione sono applicati in modo non uniforme tra endpoint dello stesso servizio.

Il mass assignment e l'esposizione eccessiva di dati rientrano entrambi sotto API3:2023 (Broken Object Property Level Authorization), mentre nella versione 2019 dell'OWASP API Security Top 10 erano due categorie distinte, unificate nella versione successiva per radice comune. Il mass assignment si verifica quando i framework web applicano il binding automatico tra i campi JSON in ingresso e il modello di dominio, senza restringerlo ai soli campi previsti dallo schema di input: un campo presente nello schema di risposta di un'operazione di lettura ma omissso dallo schema di input delle operazioni di scrittura sullo stesso tipo di risorsa è implicitamente read-only per contratto, ma il backend lo accetta e lo persiste se incluso in una richiesta [10]. La controparte nella risposta è l'esposizione eccessiva di dati: il servizio restituisce campi sensibili perché la logica applicativa serializza l'intero oggetto di dominio senza filtrare i campi per ruolo del chiamante. Entrambi questi fallimenti sono rilevabili solo da uno strumento che conosce lo schema contrattuale atteso e può confrontare ciò che transita con ciò che il contratto dichiara.

Il Server-Side Request Forgery (SSRF) (API7:2023) introduce un meccanismo di attacco indiretto: alcuni endpoint accettano Uniform Resource Locator (URL) forniti dal client per funzionalità come callback verso sistemi esterni o recupero di risorse remote. Un attaccante può fornire URL che puntano a infrastrutture interne o ai metadata endpoint dei cloud provider, inducendo il server a effettuare richieste verso reti altrimenti inaccessibili dall'esterno. La richiesta che il client invia al gateway è formalmente valida; il comportamento anomalo avviene nelle richieste che il server genera internamente verso destinazioni non raggiungibili dall'esterno, senza che alcun segnale di quelle richieste compaia nella risposta che il server restituisce al client.

Questi esempi mostrano che le vulnerabilità semantiche non condividono né le precondizioni di accesso necessarie alla rilevazione né il tipo di evidenza che producono: alcune emergono dal confronto tra risposte con identità diverse, altre dalla conoscenza dello schema contrattuale, altre ancora richiedono di osservare richieste che il server genera verso l'esterno, non visibili nel canale HTTP normale tra client e gateway. Questa eterogeneità mostra perché uno strumento che adatta la propria strategia di verifica al tipo di garanzia e alle precondizioni disponibili raggiunge una copertura che uno scanner uniforme, privo di quella distinzione, non può garantire. La ricerca sul testing delle API ha sviluppato approcci che vanno in questa direzione, introducendo le specifiche formali come base per costruire verifiche mirate, come discusso nella Sezione 2.4.3.

2.4 L'Evoluzione del Testing di Sicurezza: dall'Analisi Cieca ai Contratti

Descrivere il panorama delle minacce è il primo passo; il secondo è comprendere perché gli strumenti esistenti non riescano a coprire sistematicamente quella superficie. Questa sezione ripercorre l'evoluzione degli approcci di security testing, dai paradigmi tradizionali e i loro limiti strutturali sulle API REST (Sezione 2.4.1), all'asimmetria informativa come fattore che determina la portata di un assessment (Sezione 2.4.2), fino alla linea di ricerca che ha introdotto le specifiche formali come base per la generazione dei test (Sezione 2.4.3). Una revisione sistematica di 92 contributi sul testing automatizzato delle API REST pubblicati tra il 2009 e il 2022 documenta la crescita esponenziale di questo campo di ricerca a partire dal 2017 e ne cataloga le sfide aperte; tra queste si nota la scarsità di approcci orientati esplicitamente alla sicurezza, intesa come valutazione della superficie di attacco e identificazione delle vulnerabilità sfruttabili, distinta dalla correttezza funzionale delle risposte [2].

2.4.1 Paradigmi Tradizionali e i Limiti del DAST/SAST

Gli approcci classici al security testing delle applicazioni web si dividono in due famiglie distinte per punto di osservazione. Il Static Application Security Testing (SAST) analizza il codice sorgente o i binari compilati senza eseguire l'applicazione, identificando pattern noti di vulnerabilità a livello sintattico: buffer overflow, uso di funzioni non sicure, propagazione di dati non validati verso punti critici del codice (query al database, chiamate di sistema, e così via.). Il DAST interagisce con l'applicazione in esecuzione attraverso la sua interfaccia esterna, inviando richieste e osservando le risposte alla ricerca di comportamenti anomali: codici di errore rivelatori, stack trace nelle risposte, variazioni nei tempi di risposta correlate a payload specifici. Applicati alle API REST, entrambi i paradigmi incontrano un limite strutturale, per ragioni distinte.

Il DAST genera variazioni di parametri, sequenze di valori anomali e input strutturalmente inusuali per osservare reazioni impreviste. Per raggiungere la logica applicativa di un'API REST, le richieste devono essere strutturalmente valide rispetto al contratto dell'endpoint: tipi corretti, formati rispettati, vincoli soddisfatti. Uno strumento che genera input senza conoscere quel contratto si trova di fronte a un'esplosione combinatoria in cui la maggior parte dei payload viene respinta dalla validazione dell'input prima di raggiungere qualsiasi logica di business; questo è il limite strutturale del fuzzing cieco applicato alle API REST [2, 7], illustrato schematicamente nella Figura 2.1.

Il SAST affronta un limite strutturalmente diverso: analizza il codice sorgente e identifica pattern vulnerabili nella logica implementata, ma il suo punto di osservazione è il testo del programma, non il sistema in esecuzione. Le vulnerabilità semantiche delle API REST

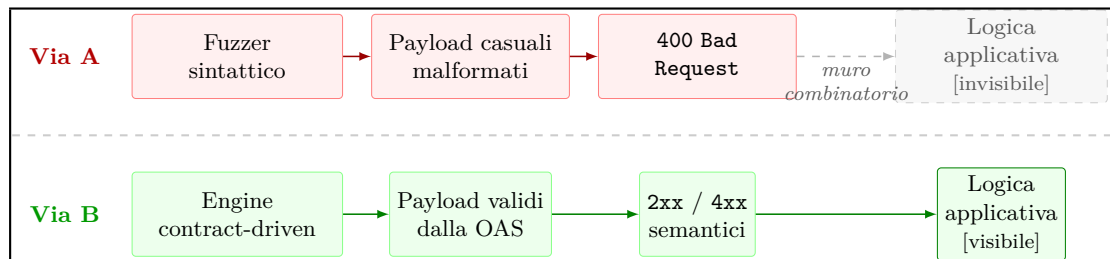


Figura 2.1: Il *combinatorial wall*: fuzzer sintattico (Via A) vs. approccio contract-driven OAS (Via B).

emergono dall'interazione tra componenti a runtime in presenza di identità, credenziali e stato; questa classe di comportamenti non è osservabile dall'analisi del codice sorgente isolato.

ZAP [11]³ è un DAST open source progettato per applicazioni web che supporta API REST tramite importazione della OAS. Anche con la specifica disponibile, lo strumento la utilizza principalmente per enumerare gli endpoint e costruire richieste strutturalmente valide, valutando le risposte rispetto a euristiche generiche di anomalia. Un campo sensibile restituito in una risposta 200 è rilevabile come anomalia solo se lo schema contrattuale non lo dichiarava atteso; in assenza di quella conoscenza il comportamento risulta indistinguibile da una risposta normale. Usare la specifica per costruire richieste valide e usarla come oracolo per valutare le risposte sono due capacità distinte: i paradigmi tradizionali si fermano alla prima.

2.4.2 Asimmetria Informativa e il Box Gradient

La capacità di rilevare una vulnerabilità dipende non solo dalla qualità dello strumento ma dall'informazione che lo strumento possiede sul sistema prima di iniziare la valutazione. Questa asimmetria informativa è il principio che sottende la classificazione Black/Grey/White Box nel testing del software: un valutatore senza accesso alla configurazione interna opera in regime Black Box, osservando solo il comportamento esterno; un valutatore con accesso completo alla configurazione opera in regime White Box, potendo verificare per ispezione diretta proprietà non osservabili dall'esterno. Il regime Grey Box rappresenta gli scenari intermedi in cui il valutatore dispone di alcune informazioni privilegiate, ad esempio credenziali applicative per uno o più account, senza avere accesso alla configurazione interna del sistema.

Applicata alle API REST esposte tramite gateway, questa distinzione acquista una concretezza tecnica precisa. Alcune garanzie di sicurezza sono fisicamente raggiungibili

³Originariamente noto come OWASP ZAP, il progetto ha lasciato la OWASP Foundation nel settembre 2023 per ragioni di finanziamento e opera ora come progetto indipendente.

solo con determinate precondizioni: il tipo di evidenza che ciascuna verifica richiede di raccogliere determina il livello di visibilità necessario, indipendentemente dallo strumento utilizzato. Un controllo verificabile osservando il comportamento esterno del sistema richiede solo accesso di rete; un controllo che emerge dal confronto tra risposte ottenute con identità diverse richiede credenziali per più account; un controllo sulla configurazione interna richiede accesso al piano amministrativo del gateway, perché nessuna risposta agli endpoint esposti restituisce quella informazione. La disponibilità di queste precondizioni determina quale sottoinsieme di garanzie sia raggiungibile in un dato contesto di assessment.

2.4.3 Contract-Driven Security e Approcci Ibridi

Il limite strutturale del fuzzing cieco ha motivato una linea di ricerca che utilizza le specifiche formali delle API come punto di partenza per la generazione dei test. L'approccio contract-driven sostituisce le euristiche sintattiche con la conoscenza del contratto: la specifica fornisce la struttura delle richieste valide, le dipendenze tra operazioni e gli schemi di risposta attesi, consentendo di costruire test che raggiungono sistematicamente la logica applicativa. Questa direzione ha prodotto strumenti con obiettivi diversi, dal fuzzing stateful alla verifica di proprietà specifiche, che condividono l'uso della specifica formale ma differiscono nel modo in cui ne sfruttano le informazioni. Una revisione sistematica di 92 contributi pubblicati tra il 2009 e il 2022 [2] documenta che il campo si è concentrato prevalentemente sul *system testing*, inteso come verifica che l'API risponda correttamente, non vada in errore e rispetti i formati dichiarati: 72 paper su 92 rientrano in questa categoria. Il *security testing*, ovvero la valutazione dei rischi e l'identificazione delle vulnerabilità che potrebbero essere sfruttate per compromettere il sistema, è affrontato esplicitamente da soli 8 paper su 92 e identificato come sfida aperta per il lavoro futuro in altri 10. La sproporzione riflette una sfida strutturale del dominio: le verifiche di sicurezza richiedono criteri di valutazione che trascendono la conformità strutturale della risposta, e la costruzione automatizzata di tali criteri in modo sistematico resta tra le sfide aperte del settore. Du et al. [12] documentano empiricamente questa disparità: nel confronto diretto tra fuzzer stateful per API REST, solo il 2-5% dei bug HTTP 500 rilevati dagli strumenti analizzati, tra cui RESTler, corrisponde a vulnerabilità di sicurezza effettive, mentre la grande maggioranza sono crash funzionali causati da parametri malformati.

RESTler [7], il primo fuzzer stateful per API REST presentato all'ICSE 2019, analizza la OAS e inferisce le dipendenze produttore-consumatore tra le richieste: se un endpoint `POST /resources` produce un `resource_id` che un endpoint `GET /resources/{id}` consuma come parametro di path, RESTler genera automaticamente la sequenza nell'ordine corretto. La specifica viene utilizzata per costruire richieste strutturalmente valide; l'oracolo adottato per la rilevazione dei difetti è il codice di stato HTTP 500: una sequenza valida che produce un *Internal Server Error* è classificata come bug. Gli autori riconoscono

esplicitamente che questo oracolo semplice non può rilevare vulnerabilità che non si manifestino attraverso i codici di stato HTTP, tra cui le violazioni di controllo d'accesso e l'esposizione di campi sensibili in risposte con codice 200.

Atlidakis et al. [8] estendono RESTler con quattro regole di sicurezza formali e altrettanti *active property checker*: **use-after-free** (una risorsa cancellata non deve restare accessibile), **resource-leak** (una creazione fallita non deve lasciare side-effect nel backend), **resource-hierarchy** (una risorsa figlia non è accessibile da un parent diverso da quello originario) e **user-namespace** (una risorsa non è accessibile da un namespace distinto da quello che l'ha creata). A differenza del monitoraggio passivo, ciascun checker genera attivamente nuovi test case mirati a innescare la specifica violazione; i checker classificano come difetto anche risposte 2xx inattese laddove la regola impone un rifiuto, una classe di violazioni invisibile all'oracolo di RESTler che riconosceva come bug i soli codici HTTP 500. Lo stesso paradigma contract-driven è stato applicato a classi di vulnerabilità specifiche: per il mass assignment, i campi presenti negli schemi di risposta delle operazioni di lettura ma assenti negli schemi di input delle operazioni di scrittura sullo stesso tipo di risorsa sono i candidati (read-only) alla vulnerabilità, identificabili tramite testing attivo black-box [10] o, a partire dalla sola specifica, già prima che l'API sia in esecuzione [13].

NAUTILUS [14] affronta la classe di vulnerabilità che richiedono sequenze di più operazioni in ordine preciso: SQL injection, XSS, command injection e privilege escalation via parameter injection non sono raggiungibili da scanner che testano ogni endpoint in isolamento. A partire dalla specifica OpenAPI, lo strumento genera automaticamente annotazioni che codificano dipendenze logiche tra operazioni e le aggiorna durante il testing; il rilevamento si articola in una fase di esplorazione, che costruisce sequenze valide, e in una fase di exploitation, in cui payload di attacco vengono iniettati e verificati con oracoli per tipo di vulnerabilità: variazione del codice di stato, struttura della risposta e relazione semantica tra payload e output.

Nella stessa direzione si colloca Schemathesis [15], uno strumento di property-based testing che deriva fuzzer⁴, strutturalmente e semanticamente consapevoli, dalla OAS. Schemathesis verifica un insieme di proprietà più ampio del solo codice HTTP 500: conformità delle risposte allo schema OAS, rispetto dei vincoli RFC 7231 (corpo non vuoto per 200 OK, corpo vuoto per 204 No Content, header Allow obbligatorio per 405), correttezza del content-type e assenza di status code non dichiarati nel contratto. Le proprietà sono derivate automaticamente dalla specifica e dagli standard; i casi di test sono ridotti al controesempio minimale. Lo strumento rimane tuttavia orientato alla scoperta statistica di difetti di conformità strutturale: non dispone di un oracolo per le garanzie di sicurezza del sistema, come policy di autorizzazione, controlli di autenticazione o configurazione del gateway.

⁴Casi di test generati automaticamente a partire dalla OAS tramite la libreria Hypothesis

Una direzione distinta affronta la costruzione stessa dell'oracolo: mentre tutti gli approcci descritti definiscono manualmente i criteri di valutazione delle risposte, AGORA [16] e AGORA+ [17] li generano automaticamente analizzando le coppie richiesta/risposta osservate durante l'esecuzione tramite un instrumenter dedicato, rilevando le invarianti probabili dell'API e convertendole in oracoli eseguibili. SATORI [18] adotta lo stesso obiettivo con approccio statico: analizza la OAS tramite Large Language Model (LLM) e inferisce i criteri di valutazione senza richiedere esecuzioni; i due approcci risultano complementari nella copertura del contratto. Entrambi operano nel dominio della conformità funzionale: producono oracoli che verificano se l'API restituisce ciò che il contratto dichiara, non se applica le garanzie di sicurezza del sistema.

Sahin et al. [9] propongono nove oracoli automatizzati per la rilevazione di fault di autorizzazione, autenticazione e injection nelle API REST, integrati come fase di post-processing in un fuzzer esistente. I requisiti sono la sola OAS e credenziali per almeno due utenti con ruoli distinti; nessuna annotazione aggiuntiva è necessaria. Al termine della sessione di fuzzing, gli oracoli analizzano i test generati e ne costruiscono di nuovi mirati a verificare se risorse protette risultino accessibili senza credenziali, se il sistema riveli l'esistenza di risorse riservate attraverso la distinzione tra codici 403 e 404, e se le risposte HTTP 500 esponano stack trace interni. Tra gli approcci descritti, questo rappresenta il tentativo più esplicito di affrontare il security testing come obiettivo primario: non un'estensione di un processo orientato alla copertura funzionale, ma una fase dedicata alla verifica delle politiche di accesso e autorizzazione.

La progressione degli approcci in questa sezione segue una logica interna: RESTler introduce le sequenze stateful dalla specifica, Atlidakis et al. aggiungono regole di sicurezza formali, NAUTILUS orienta il paradigma verso la vulnerability detection, Schemathesis estende l'oracolo a un insieme strutturato di proprietà standard, AGORA e SATORI automatizzano la generazione degli oracoli stessi, e Sahin et al. portano questa direzione al security testing esplicito. La Tabella 2.2 sintetizza le dimensioni di confronto tra gli approcci esaminati.

Tool	Uso della specifica OAS	Cosa verifica	Limite dell'oracolo
RESTler [7]	Inferisce dipendenze produttore-consumatore per generare sequenze valide	Crash del server	Solo HTTP 500; nessuna verifica semantica
Atlidakis et al. [8]	Come RESTler, esteso con checker attivi	Uso after free, resource leak, resource hierarchy, user namespace	Regole fisse; nessuna adattabilità alla configurazione del sistema
NAUTILUS [14]	Base per le annotazioni dinamiche delle dipendenze tra operazioni	SQL injection, XSS, command injection, privilege escalation	Focalizzato sulle vulnerabilità da injection; non copre controllo d'accesso o config infrastrutturale
Schemathesis [15]	Genera casi di test tipizzati; deriva proprietà da OAS e RFC 7231	Conformità schema, status code non dichiarati, header mancanti o errati	Conformità strutturale dell'interfaccia; nessuna verifica delle garanzie di sicurezza del sistema
AGORA [16] e AGORA+ [17]	Affiancata all'analisi dinamica del traffico di esecuzione per generare oracoli eseguibili	Conformità funzionale (verifica delle invarianti probabili dell'API)	Valuta l'aderenza al contratto ma non verifica l'applicazione delle garanzie di sicurezza del sistema
SATORI [18]	Analizzata staticamente tramite LLM per inferire criteri di valutazione senza esecuzione	Conformità funzionale (verifica statica di ciò che il contratto dichiara)	Condivide il limite di AGORA: nessuna verifica sulle garanzie di sicurezza del sistema
Sahin et al. [9]	Punto di partenza per il fuzzing; gli oracoli analizzano le risposte in post-proc.	Violazioni di autorizzazione e autenticazione, stack trace esposti, endpoint non dichiarati	Limitato al layer applicativo; non copre la configurazione del gateway

Tabella 2.2: Uso della specifica OpenAPI e dell'oracolo nei principali approcci contract-driven al testing delle API REST.

Capitolo 3

Metodologia, Architettura e Principi di Design

APIGuard Assurance è un framework di security assessment automatizzato per API REST esposte tramite gateway, progettato per essere applicazione-agnostico, contract-driven e deterministicamente riproducibile. Nasce dalla necessità di superare la frammentazione degli approcci manuali e target-specifici: strumenti che richiedono di essere riscritti per ogni nuovo target, che producono risultati non confrontabili tra esecuzioni, e che non possono essere integrati in processi di verifica continuativi. Il suo obiettivo è produrre un assessment sistematico, tracciabile e ripetibile, applicabile a qualsiasi API REST documentata senza modifiche al codice.

La presentazione di *APIGuard Assurance* segue un approccio *top-down*: dai principi generali di sistema che ne definiscono il perimetro fino alle scelte implementative che li concretizzano. Architettura e implementazione sono trattate in capitoli distinti perché operano su livelli di astrazione distinti. Il Capitolo 4 mostra come i principi descritti vengono implementati.

Il design di *APIGuard Assurance* è formalizzato in 39 proprietà architetture distribuite su sette categorie; la Tabella 3.1 le raccoglie come mappa di riferimento.¹

3.1 Principi Fondamentali: il Perché delle Scelte Architetture

In coerenza con la progressione *top-down* dichiarata, l'analisi ha inizio dal livello di massima astrazione: le decisioni che definiscono il carattere del tool prima ancora di

¹In grassetto le proprietà trattate in questo capitolo.

Dominio	Proprietà
D1 Architettura & Design	D1.P1, D1.P2, D1.P3, D1.P4, D1.P5, D1.P6
D2 Estensibilità & Manutenibilità	D2.P1, D2.P2 , D2.P3, D2.P4, D2.P5, D2.P6, D2.P7, D2.P8
D3 Config & Riproducibilità	D3.P1, D3.P2, D3.P3, D3.P4, D3.P5
D4 Robustezza & Sicurezza	D4.P1 , D4.P2, D4.P3, D4.P4, D4.P5, D4.P6, D4.P7, D4.P8, D4.P9
D5 Qualità & Osservabilità	D5.P1, D5.P2, D5.P3, D5.P4, D5.P5
D6 CI/CD & DevEx	D6.P1, D6.P2, D6.P3
D7 Packaging & Deployment	D7.P1, D7.P2, D7.P3

Tabella 3.1: Proprietà APIGuard Assurance per dominio.

descrivere i componenti. Le quattro proprietà discusse in questa sezione costituiscono la struttura portante del sistema: le scelte architetturali trattate nelle sezioni successive si stratificano su di esse, ciascuna risolvendo una frizione specifica all'interno dei vincoli che queste proprietà fondamentali stabiliscono.

3.1.1 Agnosticismo Applicativo (D1.P1)

Quali target applicativi è possibile valutare senza modificare il codice del tool?

Tutta la conoscenza del target viene derivata a runtime da due sole sorgenti: il file `config.yaml`, che raccoglie i parametri operativi del deployment (descritti in dettaglio nella Sezione 3.1.4), e la specifica OpenAPI del target, che fornisce la topologia completa degli endpoint, la definizione dei parametri di path, query e header per ciascuna operazione, gli schemi dei payload di richiesta e risposta, e i codici di stato attesi. Ne consegue un vincolo strutturale preciso: APIGuard Assurance non contiene nessun riferimento hardcoded a path, strutture dati o comportamenti di un'applicazione specifica, e qualsiasi API REST documentata è testabile senza modifiche al codice.

Un test che interroga la superficie di attacco derivata dalla specifica e non individua endpoint pertinenti alle proprie condizioni di esecuzione produce **SKIP**, lo stato previsto per uno scope legittimamente vuoto, distinto dall'**ERROR** che segnalerebbe un malfunzionamento interno al tool. Il principio si estende a qualsiasi requisito non soddisfatto: un tool esterno non installato, la Admin API del gateway non raggiungibile, un insieme di endpoint privi di seed configurato; ciascuna di queste condizioni produce **SKIP** sui test che ne dipendono e lascia proseguire gli altri, a meno che il risultato non costituisca una violazione confermata di un controllo perimetrale fondamentale (come l'assenza totale di

autenticazione su tutti gli endpoint esposti), caso in cui la pipeline può essere interrotta esplicitamente tramite configurazione.

La differenza tra uno script target-specifico e un tool di assessment generale non è di complessità, ma di generalità: uno script con path e strutture dati scritti nel codice funziona su quel target e solo su quello (sostituita l'applicazione, va riscritto), mentre un tool che deriva tutta la propria conoscenza dalla specifica formale del target è applicabile a qualsiasi API REST documentata senza modifiche. Questa generalità è precisamente il confine che separa un artefatto di progetto da un contributo riutilizzabile.

Il costo accettato è la dipendenza da una specifica valida. Una specifica obsoleta o deliberatamente incompleta produce un assessment parziale senza che il sistema possa rilevarne l'incoerenza: il tool non ha modo di distinguere un'assenza di vulnerabilità da un'assenza di copertura, perché il fondamento contrattuale di cui dispone è la specifica stessa. Questa tensione costituisce il limite strutturale discusso nella Sezione 6.1 e si qualifica come problema aperto nel testing contract-driven, non come difetto correggibile con intervento locale.

3.1.2 La Specifica OpenAPI come Fondamento Contrattuale (D3.P2)

Quale fondamento formale governa la costruzione delle richieste e delimita la superficie di assessment?

La specifica OpenAPI è il fondamento contrattuale dell'intera pipeline: governa la costruzione delle richieste nel rispetto dei tipi, dei formati e dei parametri dichiarati, delimita la superficie di attacco agli endpoint documentati e fornisce il criterio di valutazione primario per le risposte ricevute. Questa scelta coincide con il primo controllo pre-runtime obbligatorio identificato dal NIST SP 800-228 [5]: i controlli REC-API-1 e REC-API-2 della Sezione 3.1.1 prescrivono che tutte le API siano descritte da una specifica formale con schema di request e response dichiarato, come condizione necessaria per qualsiasi sistema di verifica automatizzato. Per alcune classi di garanzie, la specifica è integrata da sorgenti aggiuntive, come la configurazione del gateway per i test WHITE_BOX (Sezione 3.2.4) o il confronto tra comportamenti osservati con identità distinte per i test GREY_BOX (Sezione 3.3.2): sorgenti che la specifica dell'interfaccia applicativa da sola non può esprimere.

Un fuzzer che genera payload senza conoscere il contratto affronta un ostacolo strutturale prima ancora di raggiungere la logica applicativa: una quota sostanziale delle richieste viene respinta con HTTP 400 dal layer di validazione dell'input, perché i tipi, i formati e i vincoli enumerati nello schema non sono rispettati. Questo è il *combinatorial wall* discusso nella Sezione 2.3.1: lo spazio di tutti i payload sintatticamente possibili è vastissimo, ma lo spazio dei payload strutturalmente validi rispetto al contratto è definito e navigabile.

come dimostrato da RESTler [7] nel fuzzing stateful delle API REST. Usare la specifica come oracolo formale abbatte questa barriera: ogni richiesta è costruita nel rispetto dei vincoli dichiarati, spostando il punto di osservazione dalla sintassi alla semantica. La letteratura ha già dimostrato che usare la OAS come base per un insieme strutturato di proprietà verificabili va significativamente oltre i limiti del solo oracolo HTTP 500 [15], il problema dell'oracolo nel contesto del security testing, e il modo in cui APIGuard lo affronta con sorgenti multiple, è discusso nella Sezione 3.1.3.

Ne derivano due capacità operativamente distinte rispetto al fuzzing cieco. In primo luogo, la superficie di attacco viene delimitata con precisione: gli endpoint documentati dalla specifica costituiscono l'insieme di ciò che il contratto dichiara raggiungibile, mentre qualsiasi endpoint risponda senza essere incluso in quella lista è, per definizione, una Shadow API. In secondo luogo, le risposte ricevute vengono valutate rispetto a ciò che il contratto prevede per quella richiesta, non rispetto a euristiche generiche: un campo sensibile restituito in risposta a una richiesta autenticata è un finding solo se lo schema di risposta contrattuale non lo dichiarava atteso.

Anche questa proprietà porta con sé una dipendenza dalla qualità della specifica, già enunciata per la Sezione 3.1.1, ma con una conseguenza distinta. Lì il problema era di copertura: una specifica incompleta riduce gli endpoint osservabili senza che il tool possa rilevarlo. Qui il problema è di affidabilità dei verdeti: una specifica contrattualmente errata produce valutazioni fondate su un oracolo sbagliato. Le implicazioni di questa dipendenza sulla correttezza dell'oracolo, e il suo inquadramento come limite strutturale del paradigma contract-driven, sono analizzate nella Sezione 3.1.3.

3.1.3 Il Problema dell'Oracolo nel Contract-Driven Security Testing

Come si stabilisce automaticamente se il comportamento osservato del sistema rispetta le garanzie di sicurezza dichiarate?

La generazione automatizzata di input verso il sistema è un problema in larga parte risolto dalla letteratura; stabilire automaticamente se il comportamento osservato è corretto rimane il collo di bottiglia strutturale dell'automazione del testing. Barr et al. [19] formalizzano questa sfida nella definizione di oracolo di test come funzione parziale $D : T_A \rightarrow \mathbb{B}$ che mappa una sequenza di attività a un verdetto. Nella tassonomia proposta nel survey, le tecniche di automazione si distribuiscono in quattro categorie: *specified* (oracoli ricavati da specifiche formali), *derived* (oracoli inferiti da artefatti quali esecuzioni precedenti o versioni alternative del sistema), *implicit* (anomalie universali come crash e segfault, rilevabili senza conoscenza del dominio) e *no-oracle* (strategie per ridurre il costo dell'intervento umano in assenza di oracolo automatizzabile).

La maggior parte degli approcci sul testing delle API REST, descritti nella Sezione 2.4.3.

opera nel dominio della conformità funzionale; la verifica esplicita delle garanzie di sicurezza rimane l'ambito strutturalmente meno sviluppato [2].

APIGuard adotta *specified oracles* definiti a priori per ogni singola verifica, ricavati dalla conoscenza del dominio di sicurezza del controllo da eseguire. L'approccio metodologico è uniforme: si studia il tipo di verifica da condurre, si identificano le condizioni di fallimento rilevanti, e si codifica l'oracolo nella logica della classe di test. L'oracolo può essere ricavato dallo schema contrattuale dichiarato nella OAS, confrontando ogni risposta con tipo, formato e campi attesi; può derivare dagli standard tecnici pubblicati, verificando il rispetto dei requisiti di conformità protocollare; oppure può essere costruito sulla base del controllo specifico da eseguire, come il confronto tra risposte ottenute con credenziali di ruolo distinto per la verifica dell'autorizzazione, o l'ispezione della configurazione del gateway per le garanzie infrastrutturali. Ogni test incapsula il proprio oracolo come logica di classe: una volta definito quel criterio, il tool lo applica deterministicamente a qualsiasi sistema che esponga la OAS. L'oracolo fisso garantisce che lo stesso sistema, interrogato nella stessa configurazione, produca sempre il medesimo verdetto: è questa proprietà a rendere l'assessment confrontabile tra esecuzioni successive e a fondare empiricamente la riproducibilità deterministica discussa nella Sezione 3.1.5. Conseguenza da questa scelta un limite strutturale: la correttezza dei verdetti è condizionale alla correttezza del contratto di riferimento, come discusso nella Sezione 6.1.1.

3.1.4 Config-Driven Development (D3.P1)

Esiste un'unica fonte di verità per tutti i parametri che governano il comportamento del tool?

Stabilito che la conoscenza del target risiede nella specifica OpenAPI, rimane da definire il meccanismo che governa l'esecuzione. Tutti i parametri operativi risiedono nel file `config.yaml`, strutturato in aree funzionali con responsabilità distinte: la localizzazione del gateway e della specifica OpenAPI; i parametri di esecuzione globali, tra cui soglia di priorità minima, strategia di filtraggio dei test, timeout per singola esecuzione e formato del logging; il dizionario `path_seed`, che associa a ogni parametro di path parametrico un valore concreto presente sul target; le configurazioni specifiche per singolo test, organizzate per dominio; e i parametri degli strumenti di analisi esterni, con abilitazione per tool, versione attesa e opzioni di invocazione. Le credenziali di autenticazione, per loro natura non versionabili, risiedono in un file `.env` separato, il cui contenuto viene interpolato dal loader prima del parsing. Il codice sorgente non contiene nessun literal decisionale hardcoded: qualsiasi parametro che possa variare tra un'esecuzione e l'altra è dichiarato esplicitamente e validato prima dell'avvio della pipeline.

La separazione tra logica e configurazione assegna a ogni variazione di comportamento un'unica origine dichiarata: il file `config.yaml` attivo durante quella run. Se il compor-

tamento del tool cambia tra un'esecuzione e l'altra, è perché è cambiata la configurazione: se la configurazione è identica, l'output è identico. Questa centralizzazione è anche la condizione che rende possibile la proprietà discussa nella sottosezione seguente: un sistema che legga valori da variabili d'ambiente non dichiarate, da file di stato impliciti o da sorgenti runtime non controllate non può garantire che due esecuzioni successive producano lo stesso risultato, indipendentemente dalla correttezza della logica di test. Fissare la configurazione è il primo passo necessario per fissare l'output.

3.1.5 Riproducibilità Deterministica (D3.P4)

Come viene garantito che l'assessment produca risultati confrontabili tra esecuzioni successive?

Lo stesso target, nella stessa configurazione, interrogato con lo stesso `config.yaml`, deve produrre risultati identici a ogni esecuzione. La riproducibilità è il presupposto che conferisce valore dimostrativo ai risultati del Capitolo 5: senza di essa, ogni run è un'osservazione singola non confrontabile, e l'assessment non è verificabile da terzi.

Il determinismo è ottenuto attraverso meccanismi distinti che si completano a vicenda:

- Sezione 3.1.4 (Config-Driven Development): se tutti i parametri decisionali sono fissi, la pipeline non ha sorgenti di variazione endogene.
- Struttura del DAG, trattata nella Sezione 3.2.2: un grafo aciclico orientato con risoluzione topologica deterministica produce sempre la stessa sequenza di batch, indipendentemente dall'ordine in cui i test vengono scoperti a runtime.
- Ordinamento lessicografico dei test all'interno di ogni batch del DAG: l'ordinamento topologico puro non garantisce stabilità tra test pronti allo stesso livello, e senza un criterio di tie-breaking esplicito la sequenza potrebbe variare tra esecuzioni; l'ordine lessicografico risolve questa ambiguità, garantendo che la sequenza sia sempre identica a parità di input.
- Oracolo definito a priori per ogni verifica, come discusso nella Sezione 3.1.3: a parità di risposta ricevuta dal sistema, il criterio di valutazione è fisso e il verdetto non dipende da generazione dinamica né da stato esterno all'esecuzione.

La garanzia di riproducibilità non riguarda ogni componente dell'output: timestamp e identificatori univoci dei singoli record di evidenza variano per natura tra esecuzioni. Ciò che deve essere invariante sono i contatori aggregati (PASS, FAIL, SKIP), i verdetti per singolo test e la distribuzione dei finding per dominio. La validazione empirica, condotta su due run indipendenti che hanno prodotto contatori aggregati byte-equivalenti

(9 PASS / 7 FAIL / 2 SKIP / 0 ERROR / 98 finding, con diff status per-test = 0 e delta wall-clock inferiore allo 0,3%), è discussa nella Sezione 5.3.

3.2 Architettura dell'Assessment Engine

Le proprietà discusse nella sezione precedente costituiscono la base del sistema. La Figura 3.1 ne visualizza la struttura d'insieme: gli ingressi della pipeline sul lato sinistro (`config.yaml` e la OAS) sono già stati introdotti nella Sezione 3.1; i moduli interni dell'Engine al centro, ovvero il DAG Scheduler, la separazione di contesto e l'Evidence Store, sono trattati nelle sottosezioni che seguono; l'infrastruttura target a destra, Kong Gateway con Forgejo come applicativo sottostante, è descritta nel suo deployment concreto nella Sezione 5.1. Le scelte di questa sezione si aggiungono alle proprietà fondamentali, ciascuna rispondendo a un'esigenza architetturale specifica che queste ultime lasciano aperta.

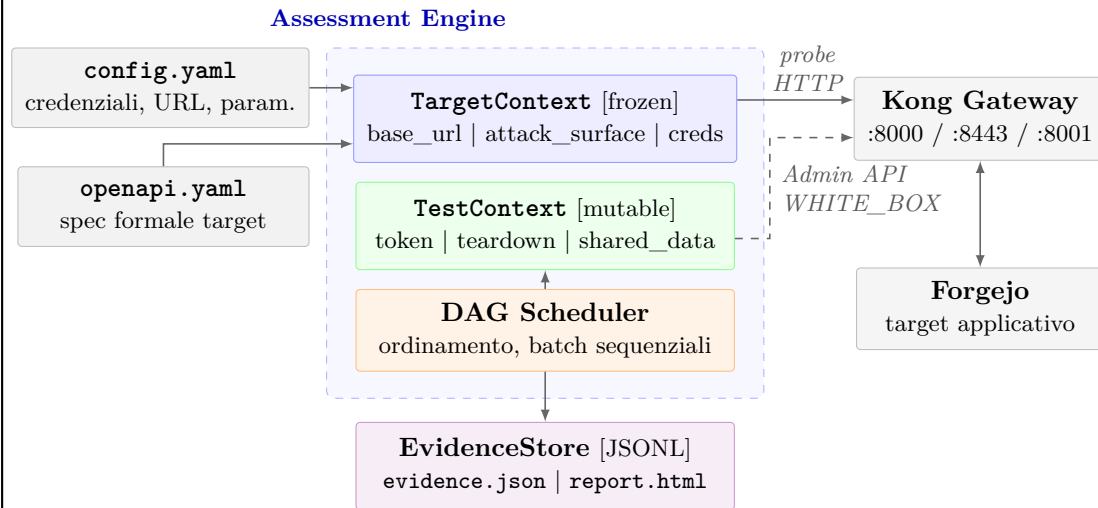


Figura 3.1: Architettura di APIGuard Assurance: Engine, DAG Scheduler ed Evidence Store.

3.2.1 Flusso di Dipendenze Monodirezionale (D1.P2)

Il grafo delle dipendenze tra strati è strettamente aciclico e orientato in una sola direzione: il nucleo del sistema è importato dagli strati di integrazione, che sono importati dagli strati di test, che sono importati dall'orchestratore. Nessun modulo può importare da un livello superiore nella gerarchia. In termini operativi: modificare un test non può rompere il nucleo; aggiungere un connector non può influenzare l'orchestratore; l'intera

base del nucleo può essere testata unitariamente senza istanziare alcun client HTTP né alcun test di sicurezza.

In assenza di un vincolo esplicito sulla direzione delle dipendenze, la pressione evolutiva naturale del codice tende verso cicli: un test che trova conveniente importare un'utility da un altro test, un connector che ha bisogno di un modello definito nell'orchestratore, e così via. La direzione è unidirezionale: un test può dipendere dal nucleo per accedere ai componenti condivisi, ma nessun modulo del nucleo può dipendere da un test. Ogni ciclo rende un modulo impossibile da isolare, e un modulo non isolabile non è testabile né sostituibile. Il vincolo è verificabile per ispezione del grafo di importazione: nessun modulo dello strato di test importa componenti degli strati superiori, e ogni strato dichiara esplicitamente nel proprio contratto le sole dipendenze che gli sono ammesse.

La struttura a strati rende praticabile uno degli sviluppi futuri descritti nella Sezione 6.3: un modulo di test contribuito da terze parti viene scoperto automaticamente² senza che nessuna modifica al nucleo del sistema sia necessaria, perché il nucleo non contiene nessun riferimento ai test che lo utilizzano.

3.2.2 Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)

L'ordine di esecuzione dei test è calcolato a partire dalle dipendenze che ogni test dichiara esplicitamente come parte del proprio contratto. Lo scheduler raccoglie queste dichiarazioni, costruisce un DAG orientato e risolve l'ordine topologico, producendo una sequenza di batch: gruppi di test il cui insieme di dipendenze è già stato soddisfatto. I batch sono sequenziali tra loro; all'interno di ogni batch i test sono privi di dipendenze reciproche. La traduzione di questo schema in codice, incluse le strutture dati e le librerie impiegate, è descritta nella Sezione 4.2.2.

Il problema che questa struttura risolve non è solo di ordinamento, ma di correttezza semantica. Un test che verifica la validità crittografica di un token JWT presuppone che il meccanismo di autenticazione funzioni: se eseguito prima del test di autenticazione, il token non è disponibile nel contesto condiviso e il test fallirebbe per la ragione sbagliata. Senza una dichiarazione esplicita della dipendenza, la disponibilità del token è una preconditione implicita che dipende dall'ordine di esecuzione scelto: un cambio di scheduling la viola silenziosamente. Con la dipendenza dichiarata, il DAG verifica la preconditione a monte dell'esecuzione, e un token non disponibile diventa un errore rilevabile prima che un singolo test venga avviato.

²La scoperta automatica non è priva di vincoli: un modulo viene incluso nella pipeline solo se rispetta i contratti del tool, ovvero se dichiara tutti gli attributi obbligatori e si colloca nella directory attesa. Un modulo che non soddisfi questi requisiti viene ignorato silenziosamente oppure produce un errore di validazione.

Lo scheduler include due proprietà di correttezza strutturale:

- Rilevazione statica dei cicli: se il grafo delle dipendenze contiene un ciclo, l'errore viene segnalato prima che un singolo test venga eseguito, impedendo che stati inconsistenti si propaghino silenziosamente nella pipeline.
- Salvaguardia contro lo stallo: se il grafo risulta ancora attivo ma non riesce a estrarre nessun test pronto, il sistema emette un errore strutturato che elenca i test mai schedati e interrompe l'esecuzione in modo controllato, fornendo all'operatore la diagnostica necessaria senza dover ricostruire lo stato interno del grafo.

L'ordinamento topologico puro non garantisce un ordine unico tra test pronti allo stesso livello: senza un criterio di tie-breaking³ esplicito la sequenza potrebbe variare tra esecuzioni. Lo scheduler risolve questa ambiguità ordinando lessicograficamente i test all'interno di ogni batch, convertendo la struttura parzialmente ordinata del grafo in una sequenza riproducibile, condizione necessaria per la proprietà di riproducibilità descritta nella Sezione 3.1.5.

La struttura a batch porta con sé una proprietà che la versione attuale non sfrutta: i test all'interno di uno stesso batch non hanno dipendenze reciproche e sono concettualmente parallelizzabili. Introdurre esecuzione parallela all'interno di un batch è una modifica localizzata allo scheduler, senza impatto sul design dei singoli test, a condizione che le implicazioni di concorrenza sullo stato condiviso vengano analizzate formalmente. Questa traiettoria è documentata come sviluppo futuro nella Sezione 6.3; la sequenzialità attuale è una scelta deliberata di scope, non un vincolo architetturale.

3.2.3 Split State e Immutabilità (D1.P3)

Durante l'esecuzione sequenziale dei test, due categorie di informazione coesistono nel processo: ciò che si sapeva del target prima che la pipeline iniziasse, e ciò che si scopre o si produce durante l'esecuzione. Trattarle con lo stesso oggetto mutabile introduce una categoria di bug difficile da diagnosticare: un test che scrive accidentalmente sulla rappresentazione del target altera la vista condivisa da cui ogni verdetto dipende, senza che il sistema possa rilevarlo.

La soluzione è la separazione netta in due oggetti con caratteristiche opposte:

- **TargetContext** - immutabile per costruzione: viene popolato una volta sola prima che la pipeline inizi e qualsiasi tentativo di modifica produce un errore immediato

³*Tie-breaking*: regola di ordinamento applicata tra elementi a parità di livello topologico, per garantire una sequenza stabile e riproducibile tra esecuzioni.

nel punto in cui viene effettuato. Raccoglie tutto ciò che si conosce del target prima dell'esecuzione:

- la specifica OpenAPI dereferenziata come mappa strutturata di endpoint, parametri e schemi;
 - i parametri di tuning specifici per ogni test ricavati dalla configurazione;
 - l'adapter del gateway iniettato prima dell'avvio della pipeline;
 - le credenziali, risolte dal loader a partire dal file `.env` e congelate nel `TargetContext` tramite la configurazione `frozen`.⁴
- **TestContext** - mutabile, raccoglie ciò che la pipeline produce. L'accesso è strutturato in tre canali tipizzati con responsabilità distinte, anziché in un dizionario ad accesso libero:
 - canale dei token: organizza le credenziali per ruolo, con accesso tramite un identificatore definito dal contratto del sistema, non tramite chiavi stringa arbitrarie;
 - canale di teardown: mantiene un registro LIFO delle risorse create durante la run, garantendo che il cleanup segua l'ordine inverso della creazione, come discusso nella Sezione 4.3.8;
 - canale dei dati condivisi: consente a test in batch successivi di consumare risultati prodotti da batch precedenti, senza che tra i moduli vengano introdotte dipendenze di importazione.

L'implementazione concreta di questa separazione, inclusi i tipi e i meccanismi di accesso, è descritta nella Sezione 4.1.3.

Qualsiasi informazione scoperta durante l'esecuzione (endpoint presenti sul target ma non dichiarati nella specifica, token acquisiti, risorse create) appartiene per definizione allo stato dell'assessment e non alla conoscenza preliminare del target: viene quindi registrata nel contesto di esecuzione, non in quello del target immutabile. La separazione fa sì che la base informativa da cui ogni verdetto dipende non possa essere alterata durante la run, condizione necessaria perché la riproducibilità descritta nella Sezione 3.1.5 sia dimostrabile empiricamente e non solo enunciabile come proprietà attesa.

⁴L'accesso avviene esclusivamente tramite i campi tipizzati del modello Pydantic costruito a runtime: non esiste un dizionario libero né accesso diretto ai valori grezzi.

3.2.4 Gateway-Agnostic Adapter (D1.P6)

La Sezione 3.1.1 stabilisce che la conoscenza del target applicativo deriva esclusivamente dalla specifica OpenAPI e dal `config.yaml`. La proprietà presentata in questa sezione estende lo stesso principio di agnosticismo a un livello distinto, lo strato di controllo del gateway: alcune garanzie di sicurezza, come la verifica che i timeout sui servizi upstream siano configurati, che il plugin di circuit breaking sia attivo o che le credenziali di servizio non siano hardcoded nelle variabili d'ambiente, non sono osservabili interrogando gli endpoint esposti ai client. Queste informazioni risiedono nel piano di configurazione amministrativo del gateway e richiedono un canale di accesso separato dai normali probe HTTP.

Il pattern scelto è un'interfaccia astratta di sola lettura verso il gateway: i test `WHITE_BOX` vi accedono esclusivamente tramite i metodi che questa interfaccia espone, senza contenere nessun riferimento al gateway specifico installato nel deployment. L'implementazione concreta viene selezionata a runtime in base alla configurazione e iniettata nel contesto del target prima che la pipeline abbia inizio. Se la Admin API del gateway non è configurata o non è raggiungibile, tutti i test `WHITE_BOX` transitano a `SKIP` con motivazione esplicita, senza errori a cascata. I dettagli dell'interfaccia e dell'implementazione Kong sono descritti nella Sezione 4.5.1.

Il vantaggio è che aggiungere supporto per un gateway diverso richiede di implementare una nuova classe concreta che soddisfi l'interfaccia, senza toccare nessuno dei test che la utilizzano: la logica di verifica non cambia, cambia solo il meccanismo con cui i dati di configurazione vengono recuperati. Il costo è che l'interfaccia può esprimere solo le verifiche comuni a tutti i gateway supportati: controlli che richiedano funzionalità vendor-specific non generalizzabili rimangono fuori dal perimetro dell'astrazione, tensione analizzata nel dettaglio nella Sezione 6.1.2.

3.2.5 Streaming Evidence Store (D4.P1)

Un finding prodotto da un test acquista valore dimostrabile solo se accompagnato dall'evidenza HTTP che lo supporta: la request inviata, la response ricevuta, il timestamp e gli identificatori di traccia. Senza di essa, l'esito del test è un'asserzione non verificabile da terzi, priva del requisito di non-repudiabilità che un audit trail tecnico richiede.

Il design originale dell'Evidence Store utilizzava un buffer a capacità fissa in memoria: i record più vecchi venivano rimossi silenziosamente quando il buffer si riempiva, generando riferimenti a record inesistenti nell'audit trail. Il risultato era un trail rotto senza che il sistema producesse nessun errore visibile. Il difetto era strutturale, non correggibile aumentando la capacità: qualsiasi limite fisso avrebbe trasformato la capacità in un para-

metro di tuning il cui valore corretto dipende da ciò che si vuole misurare, informazione non disponibile a priori.

L'Evidence Store scrive ogni record su file locali con flush immediato, senza buffering in memoria e senza dipendenze verso database o storage remoto: ogni record è permanente nel momento in cui viene prodotto e non esiste un limite superiore al numero di record per test. Il ciclo di vita dettagliato del componente è descritto nella Sezione 4.6.1. La scelta di operare esclusivamente su file locali è anche la condizione che abilita quanto descritto nella sottosezione seguente.

3.2.6 Assenza di Dipendenze di Stato Esterno (D3.P5)

APIGuard Assurance non ha dipendenze di stato esterno a runtime: tutto lo stato mutabile del processo risiede in memoria o in file locali, e le uniche connessioni di rete aperte durante un run sono quelle verso il target applicativo e, opzionalmente, verso la Admin API del gateway. Entrambe sono connessioni di test verso il sistema da valutare, non dipendenze infrastrutturali del tool. Un tool che richiede database, message broker o cache esterna trasferisce la propria affidabilità a servizi esterni: l'exit code al termine dell'esecuzione riflette lo stato dell'assessment solo se tutti quei servizi erano disponibili e corretti, una garanzia che il tool non può fornire autonomamente.

Nessun client di database, cache distribuita o SDK di storage remoto figura tra le dipendenze obbligatorie del progetto: il deployment si riduce all'installazione del package e alla fornitura del file di configurazione. Questa assenza è anche la condizione che rende il canale di comunicazione con la pipeline, descritto nella Sezione 6.2.1, strutturalmente affidabile: l'exit code al termine dell'esecuzione riflette esclusivamente lo stato dell'assessment, non quello di servizi accessori che potrebbero fallire in modo indipendente.

3.3 Domini di Sicurezza e Box Gradient Applicato

Le sezioni precedenti hanno definito le proprietà architetturali e strutturali su cui il sistema si fonda. Questa sezione definisce cosa il sistema misura e secondo quale logica organizza le proprie misure: è il punto di convergenza tra la tassonomia delle minacce discussa nel Capitolo 2, dove le categorie di vulnerabilità sono state analizzate in quanto problema, e la struttura operativa del tool, dove quelle stesse categorie diventano il perimetro di garanzie che un assessment sistematico è tenuto a coprire.

3.3.1 Tassonomia degli Otto Domini di Assurance

La superficie di sicurezza di un'API esposta tramite gateway non è omogenea: le garanzie che il sistema dichiara appartengono a famiglie semanticamente distinte, ciascuna con caratteristiche di verifica differenti. La tassonomia adottata organizza questa eterogeneità in otto domini di assurance, ciascuno corrispondente a una categoria di rischio specifica. Questa categorizzazione si allinea alle categorie di rischio per le API cloud-native e ai controlli runtime identificati, rispettivamente, nella Sezione 2 e nella Sezione 3.2.1 del NIST SP 800-228 [5]. La Tabella 3.2 li riporta con il loro titolo e l'ambito di verifica che ciascuno racchiude.

Dominio	Titolo	Ambito di verifica
D0	API Discovery	Inventario degli endpoint esposti; rilevazione di Shadow API non documentate nella specifica.
D1	Identity & Authentication	Meccanismi di riconoscimento del chiamante; robustezza del trasporto delle credenziali; ciclo di vita e revoca dei JWT.
D2	Authorization	Logiche di controllo degli accessi; difetti di segregazione orizzontale e verticale (RBAC, BOLA).
D3	Data Integrity	Coerenza strutturale dei payload; implementazione delle firme crittografiche (HMAC).
D4	Availability & Resilience	Difese contro l'esaurimento delle risorse; audit di circuit breaker e soglie di rate limiting.
D5	Observability	Qualità e tempestività della telemetria generata dal sistema.
D6	Configuration & Hardening	Rafforzamento della configurazione del gateway; policy di routing; header di sicurezza HTTP.
D7	Business Logic & Sensitive Flows	Falle semantiche complesse: elusione dei vincoli di processo, SSRF, race condition su operazioni critiche.

Tabella 3.2: Domini di Assurance di APIGuard Assurance.

La sequenza dei primi tre domini rispecchia una catena logica di precondizioni: non ha senso verificare l'autorizzazione (D2) senza avere prima verificato che l'autenticazione funzioni (D1), così come non ha senso verificare l'autenticazione senza conoscere la superficie di attacco (D0). Un assessment che verifichi D2 in assenza di certezze su D1 raccoglie evidenza su un sistema il cui comportamento in caso di autenticazione compromessa è ignoto. I domini D3-D7 non seguono la stessa dipendenza lineare: appartengono a famiglie semantiche distinte, ciascuna con le proprie precondizioni di

accesso, senza vincoli di ordinamento reciproco tra di loro. Questa catena logica si riflette nelle dipendenze dichiarate dai singoli test che operano su questi domini, il cui meccanismo di risoluzione è descritto nella Sezione 3.2.2.

Domain-5 occupa una posizione particolare nella tassonomia: il dominio è architetturealmente predisposto nella struttura del sistema, ma i test corrispondenti sono pianificati per la Milestone 2. La numerazione è riservata intenzionalmente per preservare la coerenza della sequenza nelle versioni successive.

3.3.2 Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)

La classificazione Black/Grey/White Box del testing, introdotta nella Sezione 2.4.2 come asimmetria informativa tra tester e sistema, viene derivata in APIGuard Assurance dalle precondizioni concrete che ogni garanzia impone al tester: non è una categoria assegnata retroattivamente ai test, ma il prodotto di ciò che ciascuna verifica richiede prima ancora di poter iniziare. Alcune garanzie sono fisicamente inaccessibili senza uno specifico livello di privilegio, e quella inaccessibilità è il criterio che determina il livello.

Il mapping adottato, sintetizzato nella Tabella 3.3, correla la strategia operativa alle precondizioni concrete che ogni classe di garanzie impone al tester. Il livello di visibilità di ogni test è determinato da ciò che la verifica richiede per essere condotta correttamente.

Strategia	Prerequisiti	Razionale
BLACK_BOX	Nessuno (solo accesso di rete)	Controlli perimetrali applicati dal gateway a qualsiasi interlocutore; simulazione di un attaccante esterno senza credenziali.
GREY_BOX	Token per ≥ 2 ruoli distinti	Garanzie che presuppongono stato autenticato con identità di ruolo distinte; inaccessibili senza credenziali valide.
WHITE_BOX	Accesso Admin API del gateway	Configurazione statica verificabile per ispezione diretta tramite piano di controllo del gateway.

Tabella 3.3: Box Gradient applicato in APIGuard Assurance.

I controlli perimetrali applicati dal gateway a qualsiasi interlocutore, autenticato o meno, non presuppongono nessuna conoscenza dell'infrastruttura interna: il risultato atteso è lo stesso indipendentemente da chi effettua la richiesta. Eseguirli senza credenziali è l'unico modo per ottenere evidenza non contaminata da privilegi impliciti, ed è per questo che i test BLACK_BOX sono la scelta naturale per questa classe di garanzie.

Un controllo di autorizzazione RBAC, al contrario, richiede che il sistema si trovi in uno

stato autenticato con almeno due identità di ruolo distinto. Senza quello stato il test non raggiunge nemmeno la logica che intende sondare: il gateway respinge la richiesta al livello di autenticazione, prima ancora che la logica di autorizzazione venga valutata, e il risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. La strategia **GREY_BOX** è la risposta a questa preconditione, riconosciuta anche in letteratura come requisito strutturale per la rilevazione automatizzata di fault di autorizzazione nelle API REST [9].

I test **WHITE_BOX** occupano una posizione metodologicamente distinta. Alcune garanzie sono più efficacemente verificabili tramite ispezione diretta della configurazione del gateway che tramite probe empirici verso gli endpoint: la versione TLS supportata, la presenza e la parametrizzazione dei plugin di sicurezza, l'assenza di credenziali hardcoded nelle variabili d'ambiente. Un test empirico che cerchi di inferire questi parametri dall'esterno richiederebbe attacchi elaborati con esito incerto; un audit della configurazione li verifica in modo deterministico e riproducibile. È per questo che i test **WHITE_BOX** delegano la lettura della configurazione all'adapter del gateway descritto nella Sezione 3.2.4, anziché effettuare probe sull'interfaccia pubblica.

In contesti industriali, la disponibilità delle credenziali e dell'accesso amministrativo varia da deployment a deployment. Un'organizzazione che non dispone di accesso amministrativo al gateway può eseguire l'intera suite **BLACK_BOX** e **GREY_BOX**, mentre i test **WHITE_BOX** transitano a **SKIP** con motivazione esplicita. La Sezione 6.2.3 descrive come questa variabilità si traduca in scenari concreti di deployment industriale.

Capitolo 4

Implementazione

Le proprietà architetture formalizzate nel Capitolo 3 si manifestano in questa sezione nella loro implementazione concreta. Il grado di trattamento rispecchia il peso strutturale di ciascuna: alcune definiscono l'organizzazione di interi componenti e sono trattate in sezioni dedicate; altre derivano dalle proprietà stabilite nel Capitolo 3 e vengono richiamate nel contesto del componente in cui si realizzano; altre ancora, di natura prevalentemente operativa, trovano collocazione nelle sezioni di questo capitolo pertinenti al loro locus, o nei capitoli successivi dedicati alla validazione e all'integrazione con i processi di sviluppo. La Tabella 3.1 costituisce la mappa di navigazione verso le sezioni dove ciascuna proprietà è introdotta.

Il Capitolo 3 ha percorso il primo livello di questo approccio top-down: ha stabilito i principi fondamentali del sistema, la metodologia di assessment, i domini di sicurezza e le proprietà architetture che ne definiscono la struttura. Questo capitolo ne rappresenta il livello successivo nella discesa: i principi prendono forma in moduli Python, le proprietà si concretizzano in classi, meccanismi e scelte implementative specifiche. Dove un componente è stato introdotto nella sua motivazione architetture, le sezioni seguenti aggiungono il dettaglio implementativo con un rimando esplicito alla sezione di origine.

4.1 La Pipeline di Esecuzione a Sette Fasi e il Core Engine

L'intero assessment si svolge in sette fasi sequenziali, orchestrate da `engine.py`. La sequenza rispecchia una dipendenza logica reale tra le operazioni, dove ogni fase produce le precondizioni che la successiva presuppone; tale dipendenza determina anche la semantica degli errori. Le fasi 1, 2 e 4 producono la configurazione validata, la mappa degli endpoint e il grafo delle dipendenze tra test: senza questi tre elementi nessun test può essere eseguito in modo significativo, e un errore in una qualsiasi di esse rende l'intero assessment privo di fondamento. Le fasi 5, 6 e 7 operano invece su un contesto già

validato, dove il fallimento è sempre recuperabile localmente: un test fallito produce un `TestResult(ERROR)` e la pipeline prosegue; un teardown parziale viene registrato come `WARNING` senza invalidare i risultati che lo precedono. Il flusso completo con le condizioni di errore per ciascuna fase è sintetizzato nella Figura 4.1.

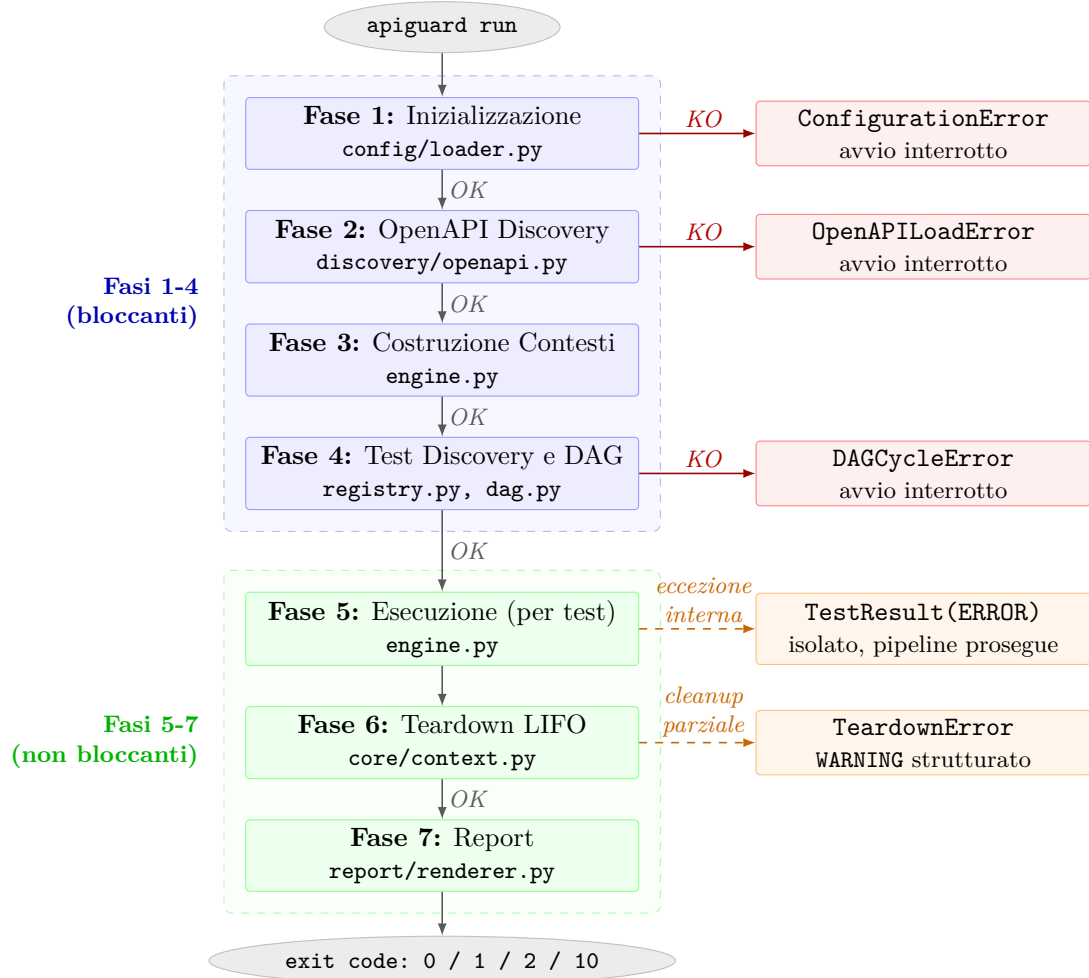


Figura 4.1: Pipeline a sette fasi: startup bloccante (Fasi 1–4) ed esecuzione non bloccante (Fasi 5–7).

Ogni fase corrisponde a un modulo separato nel filesystem (Figura 4.2), e `engine.py` è il solo file autorizzato a importare da tutti gli strati (Sezione 3.2.1). Il dettaglio di ogni fase viene presentato nell’elenco sottostante; si nota che le eccezioni citate per le fasi bloccanti appartengono alla gerarchia custom analizzata nella Sezione 4.1.2, dove il loro phase mapping e le relative politiche di recovery sono descritte.

1. **Fase 1: Inizializzazione** (Sezione 3.1.4). `config/loader.py` legge e valida `config.yaml` contro lo schema Pydantic, interpola i segreti da variabili d’ambiente

```

src/
├── cli.py (entry point CLI)
├── engine.py (orchestratore, Fasi 3 e 5)
├── config/ (Fase 1)
│   └── loader.py
├── discovery/ (Fase 2)
│   ├── openapi.py
│   └── surface.py
├── core/ (layer fondamentale, Fasi 3-5)
│   ├── context.py (Fase 3)
│   ├── evidence.py (Fase 3)
│   ├── dag.py (Fase 4)
│   ├── client.py (Fase 5)
│   ├── exceptions.py
│   └── gateway/
├── tests/ (Fasi 4-5)
│   ├── registry.py (Fase 4)
│   ├── base.py
│   ├── helpers/
│   └── domain_0/ ... domain_7/
├── external_tests/ (Fasi 4-5)
│   ├── registry.py (Fase 4)
│   └── base.py
├── connectors/ (Fase 5)
├── report/ (Fase 7)
│   ├── builder.py
│   ├── renderer.py
│   └── templates/

```

Figura 4.2: Layout di `src/` con mapping alle fasi della pipeline.

e costruisce il modello `RuntimeConfig`.

- *In caso di errore:* `ConfigurationError` interrompe lo startup.
 - *Perché bloccante:* senza configurazione valida non esistono credenziali, URL di target né parametri di esecuzione; qualsiasi fase successiva opererebbe su basi non definite.
2. **Fase 2: OpenAPI Discovery** (Sezione 3.1.2). `discovery/openapi.py` scarica la specifica OpenAPI del target, dereferenzia i `$ref`¹ tramite `prance`, la valida con `openapi-spec-validator` e costruisce l'`AttackSurface`: la mappa strutturata degli endpoint che ogni test interroga per determinare il proprio scope.
- *In caso di errore:* `OpenAPILoadError` interrompe lo startup.
 - *Perché bloccante:* la specifica OpenAPI è la sorgente esclusiva della conoscenza del target (Sezione 3.1.1); senza di essa nessun test può determinare su quali endpoint operare.
3. **Fase 3: Costruzione Contesti.** `engine.py` istanzia i tre oggetti di runtime condivisi: `TargetContext` (conoscenza del target frozen) e `TestContext` (stato dell'assessment mutabile), descritti nella Sezione 4.1.3, e `EvidenceStore` (streaming su disco), descritto nella Sezione 4.6.1.
- *In caso di errore:* nessuna eccezione prevista; la costruzione dei contesti non produce fallimenti recuperabili.
4. **Fase 4: Test Discovery e DAG.** `tests/registry.py` scopre dinamicamente i test disponibili via `pkgutil` e applica i filtri di proprietà e strategia configurati (Sezione 4.2.1); la lista risultante viene passata a `core/dag.py`, che costruisce il grafo delle dipendenze e calcola l'ordinamento topologico in batch (Sezione 4.2.2).
- *In caso di errore:* `DAGCycleError` interrompe lo startup.
 - *Perché bloccante:* un ciclo nelle dipendenze rende il grafo non risolvibile; tentare l'esecuzione produrrebbe stallo od ordinamento non deterministico.
5. **Fase 5: Esecuzione.** `engine.py` itera sui `ScheduledBatch` prodotti dalla fase precedente ed esegue i test in ordine topologico, invocando `BaseTest.execute()` per i test nativi (Sezione 4.3) e `ExternalToolTest.execute()` per i test con tool

¹Le specifiche OpenAPI usano `$ref` per referenziare schemi riutilizzabili definiti altrove nel documento o in file esterni. La dereferenziazione sostituisce ogni `$ref` con il contenuto del componente referenziato, producendo una specifica autonoma e navigabile senza dipendenze esterne.

esterni (Sezione 4.4); il meccanismo di isolamento degli errori è descritto nella Sezione 4.1.1.

- *In caso di errore:* le eccezioni interne a ogni test sono catturate dal contratto di `execute()` e convertite in `TestResult(ERROR)`; la pipeline prosegue con il test successivo.
- *Perché non-bloccante:* il fallimento di un singolo test è un dato dell'assessment, non un motivo per invalidare i risultati degli altri.

6. **Fase 6: Teardown.** `core/context.py` drena il registro delle risorse create durante l'esecuzione e tenta la loro rimozione in ordine LIFO tramite `SecurityClient`; il meccanismo è descritto nella Sezione 4.3.8.

- *In caso di errore:* `TeardownError` registrato come `WARNING` strutturato; non invalida i risultati dell'assessment.
- *Perché non-bloccante:* il mancato cleanup è un avvertimento operativo, non un'invalidazione scientifica dei finding già prodotti.

7. **Fase 7: Report.** `report/builder.py` aggrega i `TestResult` in un `ReportData` e produce tre output: `evidence.json` (archivio forense), `assessment_report.html` (report operativo con log completo delle transazioni) e `apiguard_report.json` (serializzazione strutturata per CI/CD), descritti nella Sezione 4.6.3.

- *In caso di errore:* ogni sotto-operazione di 7 dispone di un proprio blocco di gestione indipendente; un fallimento su un output non impedisce la produzione degli altri.

Il ruolo di `engine.py` è limitato all'orchestrazione pura: esegue le fasi nell'ordine corretto, trasferisce i contesti da una fase all'altra, e non contiene logica di dominio di nessun tipo (né regole di sicurezza, né criteri di valutazione, né conoscenza del formato della specifica OpenAPI). La scelta ha una ragione strutturale: un orchestratore che incorpora logica di dominio diventa il punto di concentrazione delle dipendenze, rendendo difficile testare i componenti in isolamento e trasformando ogni sostituzione di fase in un'operazione ad alto rischio di regressione. La coerenza con il principio del flusso monodirezionale (Sezione 3.2.1) è verificabile per ispezione diretta: `engine.py` importa da `core/`, `discovery/`, `tests/` e `report/`, ma nessuno di questi moduli importa da `engine.py`. Il flusso completo della pipeline con le condizioni di errore per ciascuna fase è sintetizzato nella Figura 4.1.

4.1.1 Fail-Safe Error Isolation (D4.P3)

Il contratto tra l'orchestratore (`engine.py`) e i test individuali si caratterizza per un'assenza strutturale: nessun blocco `try/except` racchiude la chiamata a `test.execute()` nell'engine. Si tratta di una scelta deliberata, che delega la responsabilità della gestione delle eccezioni a livello di test, impedendo che una singola probe possa interrompere l'esecuzione delle altre per errori non previsti. Il perimetro di responsabilità dell'engine termina alla firma del contratto: invoca `test.execute()` e riceve un `TestResult`, qualsiasi condizione interna al test, normale o anomala, non attraversa quel confine.

Perché questo contratto funzioni, `BaseTest.execute()` deve garantire due invarianti: restituire sempre un `TestResult` e non propagare mai eccezioni verso l'esterno. La realizzazione concreta è un blocco `try/except Exception` all'interno di `execute()`: in caso di eccezione imprevista, l'helper `_make_error()` costruisce un `TestResult(status=ERROR, message=str(e))` con il traceback troncato a 500 caratteri. Un test con un bug interno, una risposta HTTP inattesa o un timeout non gestito produce un `TestResult(ERROR)` registrato nell'evidenza con il proprio traceback; la pipeline prosegue con il test successivo nel batch senza interruzioni. La Figura 4.3 illustra il funzionamento: la solidità complessiva del sistema non è condizionata dalla correttezza di nessun singolo test.

4.1.2 Gerarchia delle Eccezioni con Phase Mapping (D4.P4)

Il meccanismo che rende possibile la distinzione tra errori bloccanti e non-bloccanti è la gerarchia delle eccezioni custom. Tutte le eccezioni del tool ereditano da `ToolBaseError`, radice comune della gerarchia. La struttura offre al codice chiamante due granularità di intercettazione: un `except ToolBaseError` copre qualsiasi condizione di errore del tool in un unico punto; scendere su un'eccezione specifica, come `except ConfigurationError` o `except DAGCycleError`, permette di distinguere la causa e rispondere in modo differenziato. Tale granularità rispecchia in modo diretto la distinzione tra errori bloccanti e non-bloccanti stabilita a livello architetturale, concretizzandosi in 11 classi distribuite su 3 file, la cui struttura gerarchica è illustrata nella Figura 4.4: le eccezioni delle fasi di avvio (1-4) interrompono la pipeline; quelle delle fasi di esecuzione (5-7) producono esiti recuperabili localmente; `AuthenticationSetupError` fa eccezione: sollevata dall'helper di autenticazione durante la Fase 5, è trattata come fatale se le credenziali configurate risultano invalide.

La prima scelta che riflette una decisione di design consapevole e non un vincolo tecnico, riguarda la collocazione di `GatewayAdapterError` in `gateway/base.py` invece che in `exceptions.py`. La scelta segue il principio di locality: l'eccezione è definita nello stesso file dell'interfaccia che la solleva, rendendo il contratto dell'ABC autosufficiente. Un consumer di `BaseGatewayAdapter` trova tutto ciò che gli serve in un unico modulo, senza dover importare da `core/exceptions.py`.

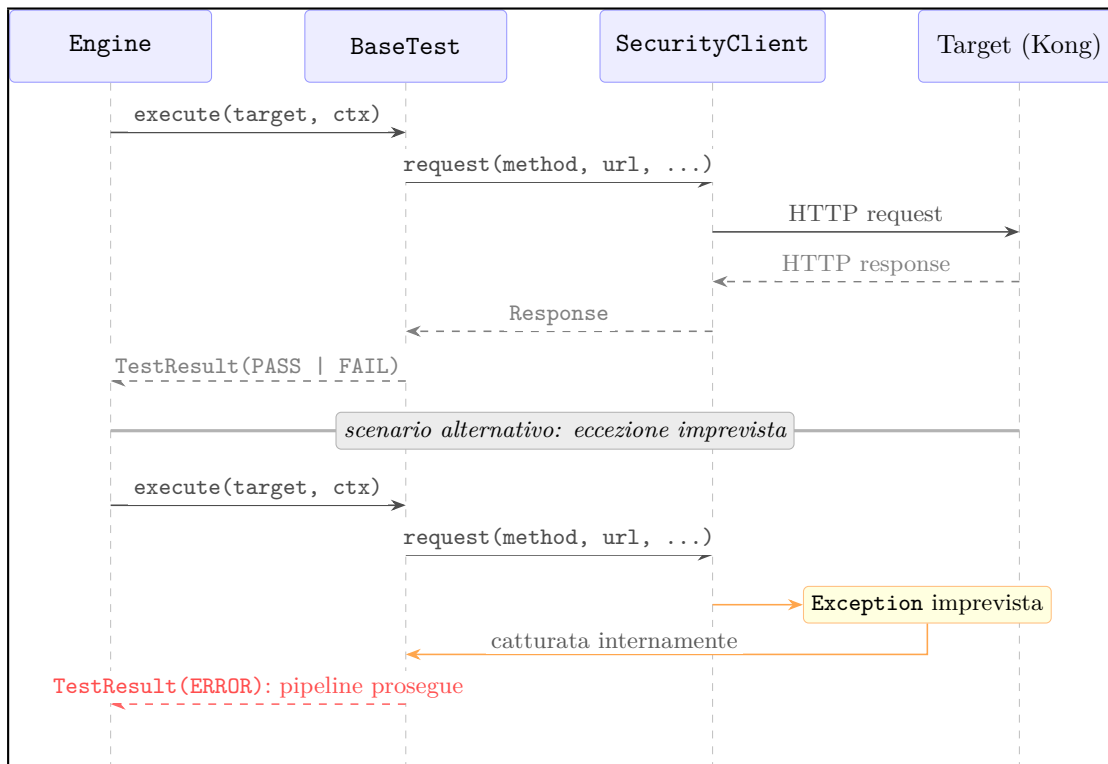


Figura 4.3: Ciclo di vita di un test in Fase 5: scenario normale e scenario di eccezione con error isolation (D4.P3).

A questa si affianca l'assenza di `ExternalToolNotFoundError`; un tool esterno mancante non è una condizione di errore imprevista: è una condizione operativa attesa, gestita a monte durante la Phase 4 tramite `is_available()` nel registry dei connector, come descritto nella Sezione 4.4.2. Il test che dipende da un tool assente riceve uno stato `SKIP` con motivo esplicito, non un'eccezione. Introdurre `ExternalToolNotFoundError` avrebbe aggiunto una classe senza aggiungere semantica: la condizione è già rappresentata da `TestResult(status=SKIP)`.

4.1.3 Costruzione dei Contesti di Esecuzione (D1.P3)

In Phase 3, l'engine costruisce i due oggetti che attraversano l'intera pipeline: `TargetContext` e `TestContext`. La motivazione architetturale della loro separazione è discussa nella Sezione 3.2.3.

`TargetContext` è dichiarato `frozen=True` a livello di modello Pydantic: qualsiasi tentativo di scrittura dopo la costruzione produce un errore immediato nel punto in cui viene effettuato. Raccoglie tutto ciò che il sistema conosce del target prima che la pipeline inizi.

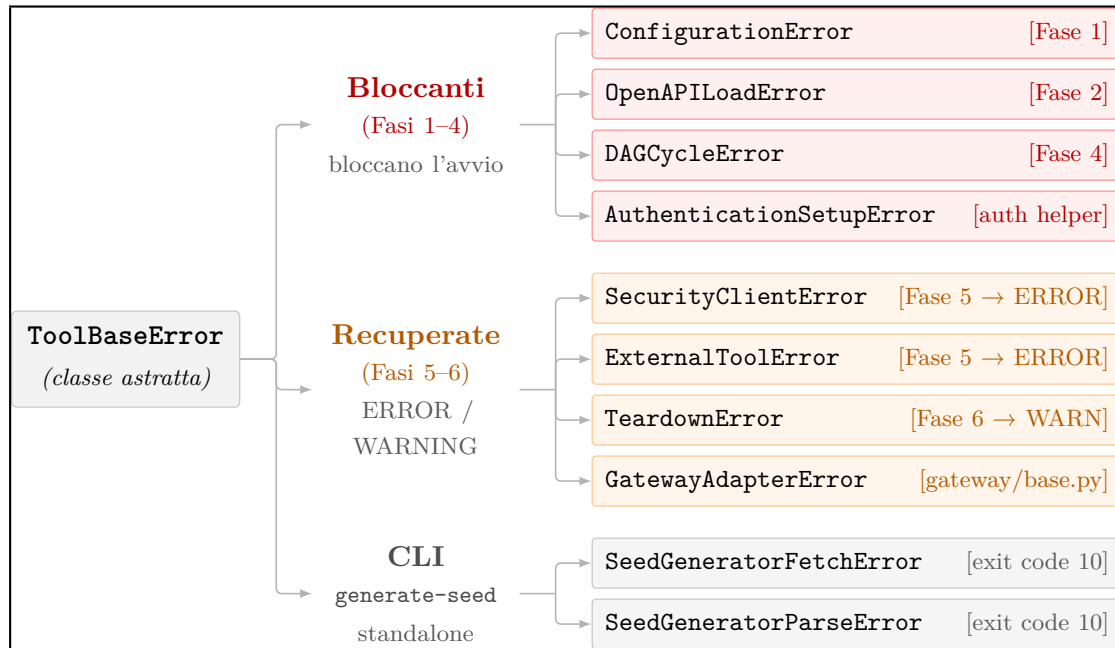


Figura 4.4: Gerarchia delle 11 eccezioni custom (D4.P4): bloccanti (Fasi 1-4), recuperate (Fasi 5-6) e CLI

l'AttackSurface costruita dalla specifica OpenAPI in Phase 2, le credenziali risolte dal loader, i parametri di tuning per singolo test, la configurazione dei tool esterni e l'adapter del gateway iniettato se configurato. Ogni test che legge TargetContext ha la garanzia assoluta che i dati che legge sono identici a quelli letti da ogni altro test nella stessa run.

TestContext è mutabile, ma l'accesso non avviene tramite un dizionario libero: lo stato è organizzato in tre canali tipizzati con responsabilità distinte. Il canale token gestisce le credenziali acquisite durante la run tramite set_token/get_token/has_token, con chiavi che sono costanti nominate (ROLE_ADMIN, ROLE_USER_A, ROLE_USER_B) e non stringhe arbitrarie. Il canale teardown mantiene il registro LIFO delle risorse create, descritto nella Sezione 4.3.8. Il canale dati condivisi permette a test in batch successivi di consumare risultati prodotti da batch precedenti tramite set_shared/get_shared, con la convenzione "{test_id}.{nome_dato}" che rende esplicito quale test ha prodotto ogni valore. La Figura 4.5 illustra la struttura interna del TestContext con i tre canali tipizzati, le rispettive API di accesso e la corrispondenza con le fasi del ciclo di vita dell'engine.

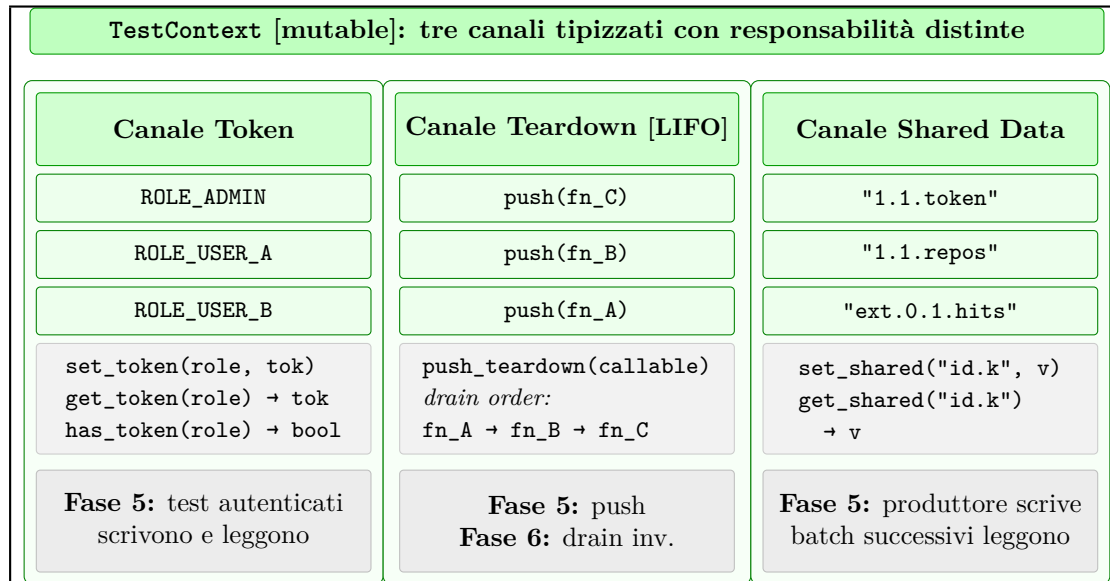


Figura 4.5: Struttura interna del TestContext (D1.P3): canali Token, Teardown LIFO e Shared Data.

4.2 Discovery Dinamico e Risoluzione del Grafo

Come stabilito nella Sezione 3.2.2 e anticipato nella Fase 4 dell'elenco nella Sezione 4.1, il DAG delle dipendenze tra test non è risolto dall'engine direttamente, ma delegato a due componenti con responsabilità distinte: il `TestRegistry`, che determina a runtime quali test esistono per ispezione del filesystem, e il `DAGScheduler`, che riceve la lista dei test scoperti e calcola l'ordine topologico a partire dalle dipendenze dichiarate. Il registry opera su moduli e classi Python; lo scheduler opera su grafi di stringhe. La separazione riflette la differenza di dominio tra le due operazioni: una riguarda la composizione del set di test attivi, l'altra la loro sequenza di esecuzione.

4.2.1 Discovery Dinamico via pkgutil (D2.P1)

Il `TestRegistry` in `src/tests/registry.py` scopre le classi concrete di test a runtime attraverso tre fasi sequenziali, denominate R1, R2 e R3, senza fare ricorso ad alcun elenco di registrazione manuale:

1. **Phase R1: Scansione del package.** La scansione del package `src.tests` è affidata a `pkgutil.walk_packages()`, che ne attraversa l'intera struttura importando tutti i moduli il cui nome finale rispetta il prefisso `test_`. Il meccanismo è indipendente dall'organizzazione delle sottodirectory: qualsiasi directory di

`src/tests/` configurata come package Python con il proprio `__init__.py` viene inclusa automaticamente, a prescindere dal nome. Per convenzione, i test sono organizzati in directory `domain_N/`, una per ciascun dominio di assurance, ma il registry non vincola questa struttura. Ogni file di test adotta la convenzione `test_N_M_description.py`, dove `N` è il numero di dominio, `M` il numero progressivo del test all'interno del dominio e `description` una label in `snake_case`; il `test_id` dichiarato nella `ClassVar` segue il formato puntato corrispondente `N.M`. Errori di importazione nei singoli moduli vengono catturati e registrati come `WARNING` strutturato senza interrompere la scansione degli altri.

2. **Phase R2: Estrazione delle classi concrete.** Dai moduli importati vengono estratte le sottoclassi concrete di `BaseTest` tramite `inspect.getmembers()`, scartando la classe base stessa, le sottoclassi astratte con metodi non implementati, e le classi prive dei `ClassVar` obbligatori verificati da `has_required_metadata()`.
3. **Phase R3: Filtraggio per priorità e strategia.** Vengono trattenuti i test la cui `priority` non supera la soglia `min_priority` e la cui `strategy` è inclusa nell'insieme delle strategie abilitate in `config.yaml`. Il filtraggio avviene per classe, prima dell'istanziamento, e ogni test escluso viene registrato a livello `DEBUG` con il motivo dell'esclusione.

Il meccanismo ha una conseguenza diretta sull'estensibilità del sistema: aggiungere un test richiede la creazione di un file nella directory di dominio corretta rispettando la convenzione `test_N_M_description.py` e implementare il contratto di `BaseTest` con i `ClassVar` obbligatori; nessun altro file deve essere modificato, nessuna registrazione manuale è necessaria.

L'`ExternalTestRegistry` in `src/external_tests/registry.py` segue le stesse fasi R1-R3, adattate alla gerarchia `ExternalToolTest` e alla convenzione di nomenclatura `ext_test_*.py`, con una fase aggiuntiva denominata R4. Dopo il filtraggio, il registry raggruppa i test sopravvissuti per `tool_name` e chiama `is_available()` una sola volta per gruppo: se il tool è presente, istanzia un connector condiviso e lo inietta in tutti i test del gruppo tramite `_injected_connector`; se il tool è assente, imposta `_skip_reason_from_registry` su ogni test del gruppo, in modo che `execute()` restituisca `SKIP` senza tentare l'invocazione. Un solo `WARNING` viene emesso per tool mancante, indipendentemente da quanti test ne dipendono.

Lo stesso principio di estensibilità vale per i test esterni: aggiungere un nuovo `ExternalToolTest` richiede la creazione di un file rispettando la nomenclatura `ext_test_*.py`, implementare il contratto di `ExternalToolTest` con i `ClassVar` obbligatori e dichiarare il `tool_name` corrispondente al connector. Questa proprietà costituisce il prerequisito tecnico del plugin system delineato come traiettoria futura nella Sezione 6.3: un package esterno che aggiunge la propria directory al `sys.path` e rispetta le convenzioni

di nomenclatura viene rilevato automaticamente al successivo avvio della pipeline, senza modifiche al core.

4.2.2 Scheduling Topologico e DAGScheduler (D2.P2)

Il DAGScheduler in `src/core/dag.py` riceve la lista dei `test_id` prodotta dal registry (Sezione 4.2.1) e restituisce una sequenza ordinata di `ScheduledBatch`: ciascun batch raggruppa i test che possono essere eseguiti nello stesso livello topologico perché le loro dipendenze sono già state soddisfatte nei batch precedenti. Il meccanismo di risoluzione, introdotto a livello architetturale nella Sezione 3.2.2, si concretizza qui nel pattern `get_ready() / done()` di `graphlib.TopologicalSorter`, la libreria standard Python introdotta nella versione 3.9. Il DAGScheduler costruisce, a partire dal campo `depends_on` di ogni test, una mappa da ogni `test_id` all'insieme dei suoi predecessori diretti, e la passa a `TopologicalSorter` come input per la risoluzione del grafo.² Chiama poi `prepare()` per intercettare eventuali cicli prima dell'esecuzione, e drena l'ordinamento livello per livello: ogni chiamata a `get_ready()` restituisce i nodi pronti, `done(*ready_nodes)` sblocca quelli che da essi dipendono, e ogni livello produce uno `ScheduledBatch`.³

`ScheduledBatch` è un `dataclass(frozen=True)` con due campi: `batch_index` (intero che indica il livello topologico incrementando a partire da zero) e `test_ids` (lista ordinata di stringhe). L'immutabilità garantisce che l'engine non possa alterare un batch durante l'esecuzione. Il determinismo all'interno di ogni batch, requisito discusso nella Sezione 3.1.5, dipende da una singola riga: `ready_nodes = sorted(sorter.get_ready())`. `graphlib.TopologicalSorter.get_ready()` restituisce i nodi pronti senza garantire nessun ordine sull'insieme estratto: a parità di grafo e di dipendenze dichiarate, l'ordine di iterazione può variare tra run, versioni di Python o piattaforme. L'ordinamento lessicografico esplicito sul `test_id` è l'unica fonte di ordine deterministico all'interno di un batch: due run con le stesse dipendenze dichiarate producono sempre la stessa sequenza di esecuzione, condizione necessaria perché D3.P4 sia verificabile empiricamente e non solo enunciabile. La topologia concreta prodotta dal DAGScheduler nell'assessment di validazione, con la sequenza di Phase A e Phase B risultante, è illustrata nella Figura 4.6.

Un caso limite, documentato nella Sezione 3.2.2 a livello architetturale, è gestito dalla stall detection: `is_active()` restituisce `True` ma `get_ready()` non produce nodi, segnalando che il sorter è in uno stato irrecuperabile nonostante l'assenza di cicli dichiarati. Il

²Ogni test dichiara solo le dipendenze immediate: se il test A dipende da B e B dipende da C, è sufficiente che A dichiari `depends_on = ["B"]`. Il vincolo su C è già catturato dalla dipendenza di B su C, e `TopologicalSorter` lo propaga automaticamente nel calcolo dell'ordinamento.

³Passare un nodo a `done()` segnala al sorter che quel `test_id` è stato completato: tutti i test che lo dichiarano in `depends_on` vedono soddisfatto quel vincolo e diventano eleggibili per la successiva chiamata a `get_ready()`.

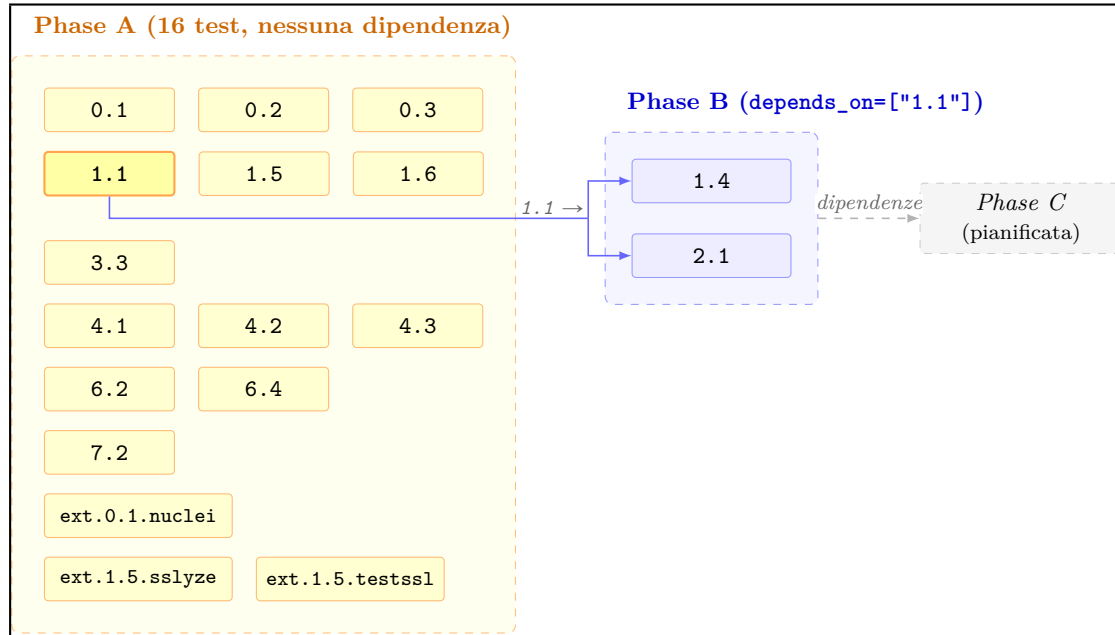


Figura 4.6: Topologia del DAG: Phase A (16 test), Phase B (`depends_on=["1.1"]`) e Phase C (pianificata).

`DAGScheduler` calcola in questo caso l'insieme dei `test_id` mai estratti, li registra in un `ERROR` strutturato con il campo `unscheduled_test_ids`, e interrompe il drain. Il log fornisce all'operatore l'elenco esatto dei test coinvolti senza richiedere la lettura di un traceback generico.

4.3 Architettura dei Test Nativi e Contratto `BaseTest`

I test nativi sono le unità di verifica che comunicano direttamente con il target tramite `SecurityClient` e producono evidenza HTTP osservabile. Il loro ciclo di vita nella pipeline, dalla scoperta in Fase 4 all'invocazione in Fase 5, è descritto nella Sezione 4.1; questa sezione ne descrive la struttura interna: il contratto che ogni test deve rispettare per essere scoperto e orchestrato correttamente, le invarianti sul risultato che il sistema applica per costruzione, il meccanismo di autenticazione condivisa, le regole di tracciabilità normativa, la policy con cui le probe HTTP vengono interpretate, il meccanismo di risoluzione dei path parametrici e il catalogo dei payload di attacco, e il meccanismo di cleanup delle risorse create durante la verifica. La Milestone 1 implementa test per i domini D0, D1, D2, D3, D4, D6 e D7; il Domain-5 (Observability) è architetturalmente predisposto e pianificato per la Milestone 2.

4.3.1 Gerarchia Duale BaseTest / ExternalToolTest (D1.P4)

Il sistema ospita due categorie di test con contratti distinti: i test nativi estendono `BaseTest` in `src/tests/base.py`; i test con tool esterni estendono `ExternalToolTest` in `src/external_tests/base.py`. Le due classi base sono gerarchie parallele e indipendenti per ragioni tecniche precise; la Figura 4.7 ne illustra la struttura d'insieme.

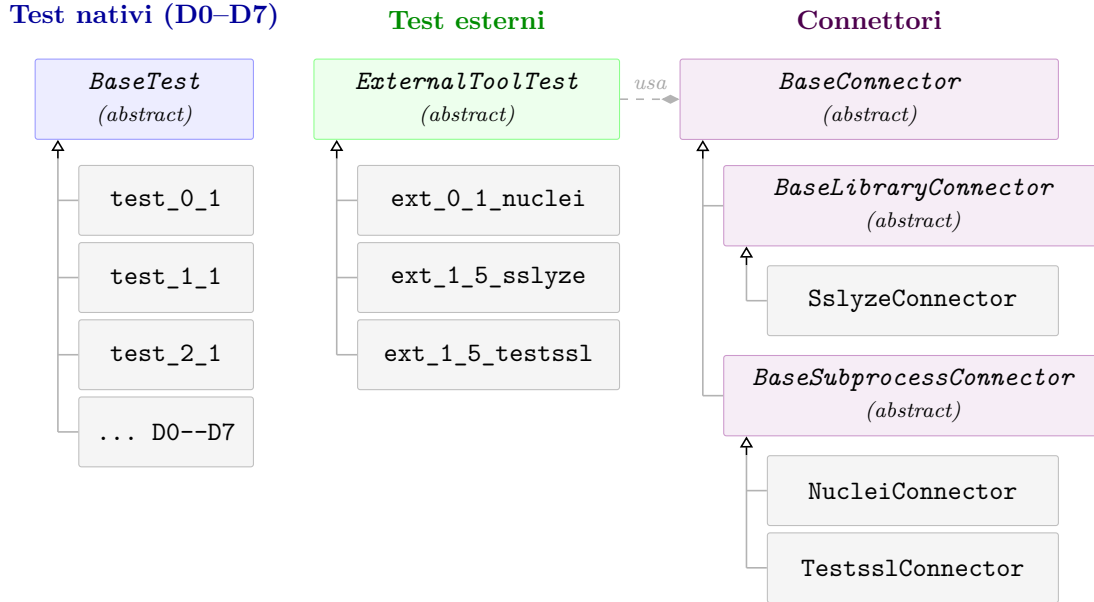


Figura 4.7: Gerarchia duale `BaseTest/ExternalToolTest` (D1.P4) e gerarchia dei connettori Library/Subprocess (D1.P5).

La scelta di mantenere le due gerarchie separate risponde a un problema concreto di ereditarietà indesiderata. `BaseTest` espone metodi come `_log_transaction()` e `_probe_endpoint()`, che operano tramite `SecurityClient` per eseguire richieste HTTP contro il target applicativo. Un `ExternalToolTest` non usa `SecurityClient`: delega la raccolta di dati al proprio connector, che invoca un processo di sistema o una libreria Python. Fare ereditare `ExternalToolTest` da `BaseTest` avrebbe reso disponibile a ogni test esterno un'interfaccia pensata per un pattern di integrazione diverso, ampliando la superficie di errore e oscurando il contratto che il test deve effettivamente rispettare.

Dal punto di vista dell'engine, la distinzione è quasi invisibile: entrambe le gerarchie producono un `TestResult`, vengono scoperte dal registry con gli stessi meccanismi di Phase R1-R3 descritti nella Sezione 4.2.1 (con l'aggiunta di Phase R4 per i test esterni), e vengono schedate dal `DAGScheduler`. L'unica differenza visibile in `engine.py` è un `isinstance` check al momento dell'invocazione, che seleziona la firma corretta di `execute()`. Il campo `TestResult.source` con tipo `Literal["native", "external"]`

propaga la distinzione fino al report, dove i risultati nativi e quelli di tool specializzati vengono riportati con sezioni visivamente distinte all'interno dello stesso dominio.

Il contratto che `BaseTest` impone a ogni implementazione concreta si articola in due livelli:

- **Dichiarazione statica via `ClassVar`:** ogni test deve dichiarare `test_id` (identificatore univoco corrispondente all'ID nella metodologia), `domain` (dominio di assurance 0-7), `priority` (P0-P3), `strategy` (BLACK_BOX, GREY_BOX, WHITE_BOX), `depends_on` (lista di `test_id` predecessori per il DAG), `cwe_id` e `tags` (riferimenti normativi). `has_required_metadata()` scarta le classi con `ClassVar` incompleti prima che vengano istanziate, così le implementazioni parziali non raggiungono mai la pipeline.
- **Implementazione di `execute(target, ctx, store)`:** il metodo che garantisce le due invarianti discusse nella Sezione 4.1.1: restituire sempre un `TestResult` e non propagare mai eccezioni verso l'engine.

4.3.2 Invarianti di `TestResult` a Livello di Tipo (D4.P9)

Ogni test produce un `TestResult`, il modello Pydantic che trasporta lo stato finale della verifica verso il report. La coerenza tra lo stato di un `TestResult` e il suo contenuto, se affidata alla disciplina distribuita di implementazioni indipendenti, è una proprietà che il sistema non può garantire strutturalmente: uno stato `FAIL` senza evidenza, uno `SKIP` senza motivazione, un `PASS` con findings allegati sono tutti costruibili sintatticamente senza che il sistema se ne accorga, producendo entry di report formalmente validi ma privi di significato verificabile. La soluzione adottata sposta l'enforcement a livello di tipo, tramite un `@model_validator(mode="after")` in `src/core/models/results.py` che applica tre invarianti nel momento in cui ogni test costruisce il proprio risultato, ovvero alla chiamata di `_make_pass()`, `_make_fail()` o `_make_skip()`, prima che l'oggetto raggiunga qualsiasi consumer:

1. `status=FAIL` richiede `findings` non vuoto. Un `FAIL` senza evidenza non è un `FAIL`: è un'asserzione.
2. `status=SKIP` richiede `skip_reason` non `None`. Ogni `SKIP` deve essere documentabile e auditabile.
3. `status=PASS` richiede `findings` vuoto. Un `PASS` non può coesistere con violazioni dichiarate.

La centralizzazione in un singolo `model_validator` ha un effetto architetturale preciso: il contratto tra stato e contenuto del `TestResult` è enforced nel punto in cui l'oggetto viene costruito, indipendentemente da quale test lo costruisce e da quale helper utilizza. Una violazione solleva `pydantic.ValidationError` nel punto in cui l'helper `_make_pass()`, `_make_fail()` o `_make_skip()` costruisce l'oggetto, con un traceback che punta direttamente alla riga incriminata nel test responsabile. Ogni nuova implementazione di test ottiene la verifica automaticamente: il validator intercetta risultati inconsistenti nel punto stesso della costruzione, senza che il test debba contenere logica di validazione propria.

4.3.3 Auth Abstraction Layer e Idempotenza dei Token (D2.P5)

I test `GREY_BOX` non possono iniziare la propria probe senza token validi: il gateway respinge qualsiasi richiesta priva di credenziali prima che la logica di autorizzazione venga anche solo valutata. Con una parte significativa dei 18 test attivi in Milestone 1 in modalità `GREY_BOX`, distribuire la logica di login in ogni test produrrebbe autenticazioni ripetute verso lo stesso target per ruoli già acquisiti. `acquire_tokens()` in `src/tests/helpers/auth.py` risolve questo problema centralizzando la gestione dei token nel `TestContext`: alla chiamata, la funzione verifica prima se il `TestContext` contiene già un token valido per il ruolo richiesto tramite `ctx.has_token(role)`; solo se il token è assente esegue il login HTTP e lo scrive nel canale token tramite `ctx.set_token(role, token)`. Il numero di login effettivi verso il target è così esattamente uguale al numero di ruoli distinti configurati, indipendentemente da quanti test li richiedano.

Il dispatcher in `auth.py` legge il campo `auth_type` dal `config.yaml` e delega l'acquisizione dei token all'implementazione corrispondente, astruendo i test dal dettaglio del protocollo. Attualmente sono supportati due metodi: `forgejo_token`, che acquisisce un token tramite l'endpoint Forgejo-specifico `POST /api/v1/users/{username}/tokens` con HTTP Basic Auth, e `jwt_login`, un flusso generico configurabile tramite `login_endpoint`, `username_body_field` e `token_response_path`, adatto a qualsiasi target che esponga un endpoint di login JSON, da crAPI a Django REST Framework. Aggiungere il supporto per un nuovo meccanismo richiede una nuova implementazione del flusso e un ramo aggiuntivo nel dispatcher; i test rimangono invariati perché il loro contratto con il layer di autenticazione è espresso esclusivamente in termini di ruolo richiesto.

4.3.4 Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)

Ogni test porta con sé, come `ClassVar`, i riferimenti normativi della garanzia che verifica: `cwe_id` per la classe di vulnerabilità nel CWE, e `tags` per le categorie OWASP, i riferimenti National Institute of Standards and Technology (NIST) e i documenti tecnici Request for Comments (RFC). Entrambi vengono propagati fino al modello `Finding`

tramite il campo `references: list[str]` e riportati dal report HTML a fianco di ogni risultato: un analista può risalire direttamente dal finding al codice CWE, alla categoria OWASP API Security e alle sezioni normative pertinenti senza consultare documentazione esterna.

La tracciabilità presuppone una distinzione semantica precisa tra due tipi di annotazione che un test può produrre. Un **Finding** è evidenza di una violazione: appartiene esclusivamente ai risultati **FAIL**, il suo contenuto deve essere sufficiente a giustificare la valutazione negativa, e viene conteggiato nei totali aggregati del report. Un **InfoNote** è un'annotazione informativa allegabile a un risultato **PASS**: documenta contesto rilevante per l'analista senza alterare il verdetto né influenzare i contatori. La distinzione è architetturale: i contatori del report aggregano esclusivamente oggetti **Finding**, rendendo il totale affidabile e non influenzabile da osservazioni di contorno. La coerenza tra tipo di risultato e tipo di annotazione è enforced come descritto nella Sezione 4.3.2: il `model_validator` di **TestResult** garantisce che un **PASS** non possa coesistere con **Finding** nel momento stesso della costruzione dell'oggetto.

Il valore operativo della distinzione emerge con chiarezza nel test 4.3, che verifica la presenza di un meccanismo di circuit breaking sull'infrastruttura del gateway. Kong OSS, nella configurazione di Milestone 1, non implementa circuit breaking attivo ma espone healthcheck passivi che svolgono una funzione parzialmente equivalente. Il test registra **PASS** perché il requisito minimo è soddisfatto, e allega una **InfoNote** che documenta la differenza tra il compensating control osservato e il circuit breaking completo. L'analista ottiene un quadro preciso senza che il sistema produca né un falso **FAIL** né un **PASS** silenzioso che nasconde la sfumatura.

4.3.5 Oracle a Tre Esiti e Safe HTTP Probing Policy (D4.P7)

Ogni test nativo che interroga un endpoint del target riceve una risposta HTTP che il tool deve interpretare e tradurre in un verdetto preciso. Il problema non è banale: uno status **404 Not Found** su un endpoint parametrico come `/users/{id}/repos` può significare che il gateway ha rifiutato la richiesta per mancanza di autenticazione, che il path non esiste per il valore di `{id}` usato nel probe, o che l'endpoint è genuinamente non protetto ma il path era errato. Interpretare un **404** come **FAIL** in tutti i casi genera falsi positivi inaccettabili in un assessment che si propone come affidabile.

La policy adottata definisce un framework a tre esiti applicato a ogni probe HTTP nei test nativi: ogni risposta è classificata come **ENFORCED** (il controllo funziona), **BYPASSED** (il controllo è assente o mal configurato) o **INCONCLUSIVE** (il risultato non è interpretabile con certezza sufficiente per produrre un **Finding**). Il mapping specifico degli status code a ciascun esito dipende dalla natura del test: per i test **BLACK_BOX** che verificano l'enforcement dell'autenticazione, la forma è la seguente:

1. **ENFORCED**: la risposta è 401 o 403. Il gateway ha applicato un controllo di accesso sul chiamante non autenticato.
2. **BYPASSED**: la risposta è 2xx. Il gateway non ha richiesto autenticazione. Il controllo è assente o mal configurato.
3. **INCONCLUSIVE**: qualsiasi altro status. Il risultato non è interpretabile con certezza sufficiente per produrre un **Finding** ⁴.

Solo un esito **BYPASSED** produce un **Finding** e contribuisce a un risultato **FAIL**. Un esito **INCONCLUSIVE** viene registrato nell'evidenza con il suo status effettivo, ma non altera il verdetto del test.

La codifica a tre esiti presuppone che la risposta ricevuta rifletta il comportamento del controllo di accesso, non un effetto collaterale della probe stessa. Questa condizione regge per i path fissi della specifica, dove il test raggiunge una risorsa che esiste e la risposta è interpretabile. Per i path parametrici la situazione è più sottile: un endpoint come `/users/{id}/repos` ha una struttura valida, ma il valore concreto del parametro determina se quella specifica risorsa esiste sul target. Con un valore che non corrisponde a nessun oggetto reale, il gateway o il backend restituisce **404** perché la risorsa è assente, non perché stia applicando un controllo di accesso; il risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. Classificare quel **404** come **INCONCLUSIVE** è l'unica risposta metodologicamente corretta. Da questa asimmetria nasce la segregazione in due tier: il Tier A raccoglie i path fissi e quelli parametrici per cui il seed è configurato (Sezione 4.3.6), dove la probe usa valori concreti verificati sul target e la risposta è interpretabile; il Tier B raccoglie i path parametrici senza seed, dove il risultato è **INCONCLUSIVE** e l'evidenza segnala all'operatore che configurare il seed risolverebbe l'ambiguità.

Un caso limite merita attenzione esplicita. Le probe con metodo **DELETE** su endpoint parametrici presentano un rischio operativo: se il gateway è mal configurato e non applica autenticazione, una richiesta **DELETE** con un path valido potrebbe eliminare risorse reali sul target. La policy prevede un fallback sicuro: quando il path seed non è configurato per un endpoint **DELETE** parametrico, il probe viene eseguito con il valore letterale `"apiguard-probe"` come sostituto del parametro. Un path con quel valore non corrisponde a nessuna risorsa reale su alcun sistema, garantendo che il probe non causi effetti collaterali indesiderati anche in presenza di un gateway non autenticato.

⁴Esempio osservato su Forgejo: senza path seed, il probe su `/api/v1/repos/{owner}/{repo}` usa il placeholder `"1"` per entrambi i parametri; Forgejo risponde **404** perché la risorsa non esiste, la richiesta non raggiunge il middleware di autenticazione e il risultato è **INCONCLUSIVE**. Con seed configurato, la stessa richiesta produce **ENFORCED** o **BYPASSED**.

4.3.6 Path Seed System (D2.P7)

Il path seed system affronta una tensione strutturale dell'agnosticismo applicativo. Derivare tutta la conoscenza del target dalla specifica OpenAPI significa conoscere la topologia degli endpoint ma non i valori concreti dei loro parametri: `/users/{owner}/repos/{repo}` è un endpoint documentato, ma senza sapere quale `owner` e quale `repo` esistano realmente sul target, la probe usa un placeholder il cui path quasi certamente non esiste, producendo 404 e un risultato INCONCLUSIVE. Il seed risolve questa tensione fornendo per ogni parametro la possibilità di specificare un valore concreto verificato sul target⁵: la generazione tramite il comando CLI `generate-seed`, che interroga la specifica OpenAPI, produce un template da completare e inserire in `config.yaml`. Con il seed configurato per Forgejo, i path con `{owner}` e `{repo}` producono risposte interpretabili: ENFORCED se il gateway applica il controllo atteso, BYPASSED se non lo applica. La Sezione 5.3 riporta i dati quantitativi di questo effetto sull'assessment descritto in questo capitolo.

4.3.7 Catalogo dei Payload di Attacco (D2.P8)

I test nativi applicano il principio di separazione tra dati e logica di valutazione che, come verrà illustrato nella Sezione 4.4.1, governa anche il contratto dei connector: i dati di input alla verifica risiedono separati dalla logica che li interpreta. Nel caso dei test nativi, questo si concretizza nel catalogo dei payload. Le sequenze di URL usate dal test SSRF, gli header malformati per i test di input validation, e altri vettori di attacco risiedono in moduli puri in `src/tests/data/`, distinti dalla logica che li applica. Aggiungere un nuovo vettore richiede la modifica del catalogo, senza toccare il test che lo utilizza. La separazione ha una conseguenza operativa concreta in contesti enterprise, dove i payload di attacco sono soggetti a un processo di approvazione separato dal codice applicativo.

4.3.8 Teardown Best-Effort in Ordine LIFO (D4.P6)

I test che raggiungono la logica applicativa del target spesso creano risorse necessarie alla verifica: token di autenticazione temporanei, utenti di test, risorse effimere. Lasciare queste risorse sul target al termine dell'assessment viola la proprietà di non-interferenza: un assessment successivo potrebbe incontrare uno stato alterato, e il target produttivo potrebbe accumulare risorse orfane nel tempo.

Il cleanup delle risorse si basa sulla registrazione esplicita al momento della creazione: ogni volta che un test crea una risorsa persistente, chiama immediatamente `ctx.register_resource_for_teardown(method, path, headers)` passando l'end-

⁵Un valore verificato è un identificatore di una risorsa realmente esistente nel deployment; nel caso di Forgejo: un utente, un repository, e così via.

point di cancellazione. La coppia creazione-registrazione avviene nella stessa transazione logica; se il test va in eccezione tra le due operazioni, la risorsa potrebbe non essere rimossa. Questa condizione è documentata come limite noto: il meccanismo di isolamento degli errori descritto nella Sezione 4.1.1 non può recuperare uno stato che non è mai stato registrato.

In Phase 6, l'engine chiama `ctx.drain_resources()`, che restituisce le risorse in ordine LIFO. Tale ordine inverso rispetto alla creazione è necessario per gestire correttamente le dipendenze tra risorse: se un test ha creato prima un repository e poi una issue al suo interno, la issue dipende dal repository; cancellarli nell'ordine di creazione eliminerebbe prima il repository, rendendo impossibile la cancellazione della issue. Invertendo l'ordine, la issue viene rimossa prima che il repository smetta di esistere. Per ogni risorsa restituita, l'engine tenta la chiamata HTTP di cancellazione tramite `SecurityClient`; un fallimento solleva `TeardownError`, catturato e registrato come `WARNING` strutturato. Il fallimento del teardown è un avvertimento operativo, non una condizione che invalida i risultati prodotti nelle fasi precedenti. La Sezione 5.3 riporta i dati della verifica empirica, che ha rilevato zero risorse residue su due run indipendenti.

4.4 Gerarchia dei Connettori e Integrazione Tool Esterni

La controparte della gerarchia duale introdotta nella Sezione 4.3.1 è `ExternalToolTest`: i test che delegano la raccolta di dati a tool specializzati invece di comunicare direttamente con il target tramite `SecurityClient`. Seguono lo stesso ciclo di vita nella pipeline (scoperti in Fase 4 dall'`ExternalTestRegistry` con l'aggiunta di Phase R4, invocati in Fase 5), ma con un pattern di integrazione strutturalmente diverso. Scanner di vulnerabilità, analizzatori TLS, fuzzer producono tipi di evidenza che un client HTTP generico non può raccogliere: la negoziazione TLS richiede dialogo a livello di protocollo, l'analisi con template nuclei richiede una base di signature aggiornata. Il layer dei connector è il punto in cui questa delega prende forma concreta: isola la logica di invocazione di ogni tool, normalizza il suo output in un modello comune e restituisce al test un oggetto strutturato invece del raw output del processo. In assenza di questo layer, ogni test esterno conterrebbe sia la logica di verifica della garanzia sia la meccanica di subprocess management o di importazione della libreria, accoppiando due responsabilità che evolvono per ragioni distinte.

4.4.1 Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)

La gerarchia dei connector si articola su tre livelli, ciascuno dei quali eredita solo i meccanismi pertinenti al proprio pattern di integrazione.

`BaseConnector` in `src/connectors/base.py` è il primo livello: un Abstract Base Class (ABC) puro con un solo `ClassVar` (`TOOL_NAME: str`) e tre metodi astratti che costituiscono il contratto universale verso il test. `is_available()` verifica se il tool è utilizzabile nell'ambiente corrente; `get_version()` restituisce la versione del tool per fini di audit; `run(target_url, timeout_seconds)` esegue il tool e restituisce un `ConnectorResult`. Il parametro `timeout_seconds` è privo di valore di default per design: ogni chiamante deve fornirlo esplicitamente dal `config.yaml`, rendendo impossibile dimenticarlo.

Il secondo livello si biforca a seconda della natura del tool:

- `BaseSubprocessConnector` viene utilizzato dai tool invocati come processo di sistema (`NucleiConnector`, `TestsslConnector`): fornisce implementazioni concrete di `is_available()` e `get_version()` basate su subprocess, e il metodo `_run_subprocess()` che gestisce esecuzione, timeout e parsing dell'output. La ricerca del binario avviene su tre canali, fermandosi alla primo soddisfatto, nel seguente ordine: binario locale nella directory `tools/` del progetto, poi `shutil.which()` sul `PATH` di sistema, infine una variabile d'ambiente per i deployment in cui il tool è esposto come microservizio HTTP. Il medesimo connector funziona così senza modifiche in sviluppo locale, in CI/CD e in ambienti containerizzati.
- `BaseLibraryConnector` serve i tool Python-native (`SslyzeConnector`): usa `importlib.util.find_spec()` per verificare la disponibilità della libreria senza invocare alcun subprocess, e legge la versione tramite `importlib.metadata`. La disponibilità è determinata dalla presenza del package nell'ambiente Python, non da un binario sul filesystem.

Le implementazioni concrete si limitano a dichiarare le `ClassVar` specifiche del tool⁶ e implementare `run()`, il cui contratto formalizza una separazione precisa tra raccolta di dati e valutazione della garanzia. Il connector restituisce un `ConnectorResult`, modello Pydantic frozen che trasporta `tool_name`, `tool_version`, `raw_output: dict[str, Any]`, `exit_code`, `execution_time_ms` e `timed_out`: dati grezzi, privi di qualsiasi giudizio sulla sicurezza del target.

La valutazione è responsabilità esclusiva dell'`ExternalToolTest`, che implementa `_evaluate()` e applica il proprio oracle al `ConnectorResult`. La ragione di questa separazione è la riusabilità: il `NucleiConnector` non sa nulla della garanzia che il test vuole verificare, e per questo può essere condiviso da test con logiche di valutazione completamente diverse. Un secondo test che vuole usare nuclei per injection scanning scriverebbe un `_evaluate()` diverso ma userebbe lo stesso connector, senza alcuna modifica alla logica di invocazione.

⁶Per `BaseSubprocessConnector`: `TOOL_NAME`, `BINARY_NAME`, `LOCAL_TOOLS_SUBDIR` e `SERVICE_ENV_VAR`
Per `BaseLibraryConnector`: `TOOL_NAME` e `LIBRARY_NAME`.

4.4.2 Lifecycle dei Connector e Dependency Injection (D2.P4)

Come descritto nella Sezione 4.2.1, la Phase R4 dell'`ExternalTestRegistry` raggruppa i test per `tool_name` e chiama `is_available()` una sola volta per gruppo. Il risultato di quella chiamata determina il lifecycle del connector per tutti i test del gruppo: disponibilità o assenza del tool si traduce in due comportamenti distinti, gestiti tramite due attributi di istanza di `ExternalToolTest`.

Se il tool è disponibile, il registry istanzia un connector condiviso e lo scrive in `_injected_connector` su ogni test del gruppo: tutti condividono la stessa istanza, costruita una volta sola. Se il tool è assente, il registry scrive invece in `_skip_reason_from_registry` la motivazione di skip già formulata. Al momento dell'esecuzione, `_run()` controlla questo campo come prima operazione: se valorizzato, restituisce `SKIP` immediatamente senza tentare la costruzione del connector né interrogare il filesystem. In entrambi i casi, la decisione è presa una volta sola in Phase R4 e propagata a tutti i test del gruppo tramite questi due attributi.

La conseguenza operativa misurabile è nel log di esecuzione. Con cinque test che dipendono da nuclei e il binario assente, il log produce un solo `WARNING` del tipo *"nuclei not found: 5 tests will SKIP"*, invece di cinque entry identiche. In un assessment con molti tool e molti test, questa differenza rende il log leggibile invece che ridondante. Il comportamento complessivo del sistema in presenza di infrastruttura parzialmente disponibile, compresi gli scenari con Admin API assente o master switch disattivato, è descritto nella Sezione 4.5.2.

4.4.3 Classificazione A/B dei Connector (D2.P6)

I connector sono classificati in due categorie che riflettono il loro rapporto con la copertura nativa. I connector di Categoria A coprono garanzie che i test nativi non possono raggiungere con soli probe HTTP: la loro assenza produce `SKIP` su verifiche altrimenti inaccessibili. I connector di Categoria B estendono la copertura su garanzie già verificate parzialmente dai test nativi, aggiungendo profondità senza essere necessari per l'assessment di base. Nella Milestone 1, `NucleiConnector` e `TestsslConnector/SslyzeConnector` appartengono entrambi alla Categoria A: il primo perché l'analisi di Shadow API tramite template specializzati non è replicabile con probe HTTP generici; i secondi perché la negoziazione TLS richiede dialogo a livello di protocollo che supera le capacità di un client HTTP standard.

4.4.4 Isolamento della Dipendenza AGPL (D7.P2)

`sslyze`, la libreria Python per l'analisi TLS, è distribuita sotto licenza GNU Affero General Public License (AGPL) v3. Le dipendenze dichiarate in `[project.dependencies]` di `pyproject.toml` vengono installate automaticamente con qualsiasi `pip install apiguard-assurance`; includervi `sslyze` propagherebbe i suoi obblighi copyleft a chiunque distribuisca o utilizzi il tool, indipendentemente dall'uso effettivo della libreria. La sezione `[project.optional-dependencies]` del `pyproject.toml` è pensata esattamente per questo tipo di situazione: raccoglie le dipendenze con vincoli di licenza o peso computazionale che non devono essere imposti per default, lasciando all'operatore la scelta consapevole di installarle tramite un extra esplicito.

`sslyze` è dichiarata in `[project.optional-dependencies.sslyze]`. L'installazione esplicita con `pip install "apiguard-assurance[sslyze]"` abilita `ext.1.5.sslyze` e comporta l'accettazione consapevole della licenza AGPL per quella componente. In assenza dell'installazione, `SslyzeConnector.is_available()` restituisce `False` tramite `importlib.util.find_spec()`, il test riceve `SKIP` con motivazione esplicita, e il core rimane privo di qualsiasi import condizionale della libreria, isolandola strutturalmente. Il pattern è replicabile per qualsiasi dipendenza con vincoli analoghi configurando il `pyproject.toml` appositamente.

4.5 Implementazione del Gateway Adapter

La Sezione 3.2.4 ha stabilito il principio di agnosticismo verso il gateway e introdotto `BaseGatewayAdapter` come interfaccia astratta di sola lettura verso il piano di configurazione amministrativo. Questa sezione ne descrive la concretizzazione: l'implementazione Kong che viene iniettata nel `TargetContext` in Fase 3 e i tre livelli di degradazione controllata che il sistema applica quando parti dell'infrastruttura non sono disponibili.

4.5.1 KongGatewayAdapter: Accesso Read-Only all'Admin API

`KongGatewayAdapter` in `src/core/gateway/kong.py` è l'unica implementazione concreta di `BaseGatewayAdapter` nella Milestone 1, e realizza il principio dichiarato nella Sezione 3.2.4: i test `WHITE_BOX` 3.3, 4.2, 4.3 e 6.4 chiamano `target.gateway.get_plugins()`, `get_services()` e `get_upstreams()` tramite l'interfaccia astratta, senza contenere nessun riferimento a Kong. L'implementazione interroga la Admin API di Kong DB-less tramite richieste HTTP GET. Una scelta implementativa merita particolare attenzione: il componente usa `httpx` direttamente invece di `SecurityClient`. Le chiamate alla Admin API sono audit di configurazione dell'infrastruttura, non traffico di test verso il

target applicativo; appartengono a un confine di fiducia distinto, non devono apparire nell'EvidenceStore e non sono soggette alla policy di retry del SecurityClient. Un client httpx dedicato mantiene i due confini separati nel codice in modo esplicito e verificabile.

I metodi pubblici, `get_routes()`, `get_plugins()`, `get_services()`, `get_upstreams()` e `get_plugin_by_name()`, delegano tutti la lettura a `_fetch_paginated()`. La Admin API di Kong restituisce le collezioni in formato paginato: ogni risposta contiene un campo "data" con gli elementi della pagina corrente e un campo "next" con il path della pagina successiva, oppure null se non ci sono ulteriori elementi. `_fetch_paginated()` segue questo cursore iterativamente finché "next" è null, concatenando i risultati in un'unica lista. In modalità DB-less tutti gli oggetti entrano in una singola pagina e "next" è sempre null, quindi il ciclo termina alla prima iterazione; lo stesso meccanismo funziona correttamente in ambienti con Kong con database, dove le risorse possono essere distribuite su più pagine.

Il ciclo di vita dell'adapter si articola su due momenti distinti. In Fase 3 si verifica tramite `check_connectivity()` che la Admin API sia raggiungibile prima di popolare `TargetContext.gateway`: se la verifica fallisce, il campo rimane None e tutti i test WHITE_BOX transitano a SKIP per l'intera run. Durante l'esecuzione dei test nella Fase 5, qualsiasi errore nelle chiamate HTTP verso l'Admin API viene incapsulato in `GatewayAdapterError`, che il test cattura e converte in `TestResult(ERROR)` senza propagare l'eccezione all'engine.

4.5.2 Degradazione Controllata a Tre Livelli (D4.P5)

Un assessment eseguito in ambienti eterogenei incontra sistematicamente situazioni in cui parti dell'infrastruttura non sono disponibili: tool esterni non installati, Admin API non esposta, integrazione con tool esterni disabilitata per policy. La risposta del sistema non è un errore a cascata, ma una degradazione controllata su tre livelli distinti che produce risultati parziali corretti con motivazioni esplicite in ogni caso.

- **Livello 1 - master switch globale.** Il flag `external_tools.enabled` in `config.yaml` disabilita l'intera gerarchia `ExternalToolTest` in un'unica operazione: l'`ExternalTestRegistry` restituisce una lista vuota senza scandagliare il filesystem né importare moduli, con zero overhead e zero SKIP nel log. Serve per ambienti in cui la presenza di tool esterni è impossibile per policy di sicurezza o di deployment.
- **Livello 2 - tool esterno assente.** Quando il master switch è attivo ma un singolo tool non è disponibile, la Phase R4 descritta nella Sezione 4.2.1 imposta

`_skip_reason_from_registry` su ogni test del gruppo: ogni test riceve **SKIP** con motivazione esplicita, e un solo **WARNING** viene emesso nel log per tool mancante.

- **Livello 3 - Admin API del gateway assente.** Se `gateway_adapter` non è configurato in `config.yaml` oppure se `check_connectivity()` fallisce in Phase 3, `TargetContext.gateway` rimane `None`. Ogni test `WHITE_BOX` chiama `_requires_admin_api(target)` come prima operazione e restituisce **SKIP** con testo esplicito senza eseguire alcuna logica di verifica. L'assessment copre in questo scenario i soli domini `BLACK_BOX` e `GREY_BOX`, con un numero di **SKIP** pari al numero di test `WHITE_BOX` nel set attivo.

In tutti e tre i casi, gli **SKIP** prodotti sono distinguibili nel report per motivazione (campo `skip_reason`), permettendo all'operatore di separare le garanzie non verificate per impossibilità tecnica da quelle non verificate per prerequisiti mancanti.

4.6 Evidence Store e Sanitizzazione delle Credenziali

Ogni verifica di sicurezza produce due tipi di output: un verdetto, espresso come `TestResult`, e l'evidenza che lo sostiene, composta dalle transazioni HTTP effettuate dai test nativi e dai dati grezzi prodotti dai tool esterni. L'evidenza è ciò che distingue un risultato verificabile da un'asserzione: senza di essa, un **FAIL** non può essere riprodotto né contestato. Raccoglierla introduce però una responsabilità: un audit trail non sanitizzato che contiene token o credenziali è esso stesso un rischio di sicurezza: il file viene condiviso con il team di analisi e archiviato nei sistemi di ticketing, dove la sua esposizione sfugge al controllo di chi ha eseguito l'assessment. Questa sezione descrive come l'`EvidenceStore` gestisce raccolta affidabile e sanitizzazione strutturale durante la Fase 5, e come vengono generati i tre deliverable finali dell'assessment in Fase 7 (introdotti nella Sezione 4.1): `evidence.json` per l'archiviazione forense, `assessment_report.html` per la consultazione operativa e `apiguard_report.json` per l'integrazione strutturata nei sistemi CI/CD.

4.6.1 Ciclo di Vita dello Streaming Evidence Store (D4.P1)

Il design dell'`EvidenceStore` è motivato nella Sezione 3.2.5, dove viene discusso il problema strutturale del buffer a capacità fissa che produceva audit trail rotti in silenzio. Questa sezione descrive l'implementazione concreta che ha risolto quel problema.

Il ciclo di vita dell'`EvidenceStore` si articola in quattro operazioni sincronizzate con le fasi dell'engine. In Phase 3, l'engine istanzia `EvidenceStore` e crea la directory temporanea `outputs/evidence tmp/`. Prima di eseguire ogni test

in Phase 5, l'engine chiama `store.begin_test(test_id)`: il metodo apre il file `outputs/evidence_tmp/{test_id}.jsonl` in modalità `append` e lo mantiene aperto per l'intera durata del test. Durante l'esecuzione, ogni chiamata a `store.log_transaction()` o `store.pin_artifact()` scrive immediatamente un singolo record in formato JSON Lines (JSONL), forzando la scrittura su disco tramite `flush()` esplicito: zero buffering in memoria, zero rischio di perdita in caso di crash. Al termine del test, `store.end_test()` chiude il descrittore del file. In Phase 7, `store.merge_and_finalize()` recupera tutti i file `.jsonl` dalla directory temporanea, applica un ordinamento lessicografico basato sul `test_id` per preservare il determinismo dell'output, concatena i record in una singola collezione strutturata in formato JavaScript Object Notation (JSON) e li scrive nel file definitivo `outputs/evidence.json`. La directory temporanea viene poi rimossa.

Riportando la soluzione iniziale con `deque(maxlen=100)`, il primo record di un test con più di 100 transazioni veniva silenziosamente scartato dalla coda, generando `Finding.evidence_ref` che puntavano a record inesistenti nell'audit trail senza produrre nessun errore visibile. Il design corrente elimina questa classe di bug per costruzione: ogni record scritto su disco è permanente, e il numero di record per test è illimitato. L'unico parametro che incide sulla dimensione dell'archivio è `RESPONSE_BODY_MAX_CHARS = 10 000`: la costante tronca il body di risposta di ogni singolo record per evitare che risposte molto grandi saturino il disco, lasciando il conteggio dei record invariato.

La proprietà di recuperabilità è un effetto collaterale dell'architettura. Se il processo va in crash tra Phase 5 e Phase 7, i file `.jsonl` rimangono su disco e sono leggibili manualmente con qualsiasi strumento JSONL-compatibile: l'evidenza parziale è recuperabile anche in assenza di un run completo.

4.6.2 Sanitizzazione Centralizzata delle Credenziali (D4.P2)

Un audit trail che contiene token di autenticazione, API key o password è esso stesso una superficie di rischio: `evidence.json` viene condiviso con il team di sicurezza, archiviato nei sistemi di ticketing, e potenzialmente incluso in repository. Delegare la redazione ai singoli connector presuppone che ogni implementazione, incluse quelle di terze parti, ricordi di oscurare i propri campi sensibili: l'adozione di logiche custom e frammentate elimina la possibilità di definire garanzie uniformi, poiché ogni estensione opererebbe secondo criteri arbitrari e privi di coordinamento.

La responsabilità della redazione viene pertanto centralizzata nel punto di scrittura su disco, operando attraverso due flussi distinti in base alla natura dell'evidenza: gli artifact generati dai tool esterni e le transazioni HTTP prodotte dai test nativi. Gli artifact dei tool esterni passano per `EvidenceStore._sanitize_artifact()`, invocato automaticamente da `pin_artifact()` prima della scrittura su disco. Questo meccanismo

solleva i connector dall'onere della pulizia dei dati e applica tre tecniche in sequenza sul dizionario `raw_output` grezzo, descritte di seguito e schematizzate nella Figura 4.8:

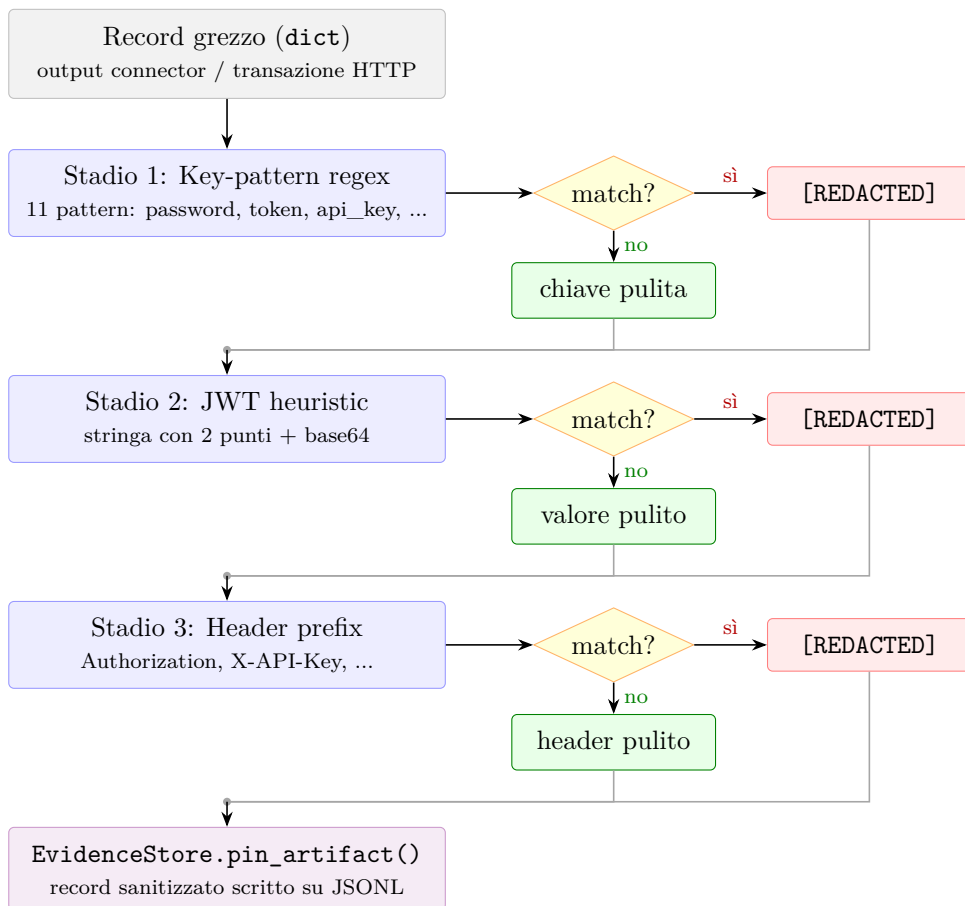


Figura 4.8: Sanitizzazione a imbuto dell'evidenza (D4.P2): key-pattern, JWT heuristic e header prefix in sequenza.

1. **Key-pattern matching:** un'espressione regolare esamina ricorsivamente ogni chiave del dizionario rispetto a 11 pattern sensibili (`token`, `password`, `api_key`, `apikey`, `authorization`, `bearer`, `secret`, `credential`, `auth`, `private_key`, `access_token`). Il confronto richiede l'esatta corrispondenza della stringa e ignora le sottostringhe: questa scelta previene falsi positivi ed evita la redazione involontaria di campi legittimi (ad esempio, una chiave `author` non viene censurata pur contenendo la sequenza `auth`). Qualsiasi chiave con corrispondenza vede il proprio valore sostituito con `[REDACTED]`, indipendentemente dal tipo.
2. **JWT value detection:** la regex `_SANITIZE_JWT_PATTERN` intercetta le stringhe che contengono la firma di un token JWT⁷, offuscando il valore indipendentemente

⁷La struttura standard di un token JWT si compone di tre parti distinte, denominate *header*, *payload* e

dal nome della chiave che lo ospita; cattura quindi i token che appaiono come valori in chiavi con nomi arbitrari.

3. **Header prefix matching:** qualsiasi stringa che inizia con "Bearer ", "Basic ", "Token " o "token " viene redactata, intercettando i token che compaiono nei valori di header HTTP inclusi nei record di evidenza.

La sovrapposizione dei tre meccanismi copre scenari che nessuno dei tre individualmente intercetterebbe: un token JWT in una chiave "result" arbitraria supera il primo meccanismo (nome non riconosciuto) ma viene catturato dal secondo (struttura base64url); un header `Authorization` in un dict annidato viene catturato dal primo meccanismo (key-pattern matching ricorsivo). Spostando il presidio di sicurezza al confine dello storage, un connector di terze parti privo di controlli interni non compromette l'integrità dell'audit trail. Il sistema diventa intrinsecamente resiliente a implementazioni difettose, poiché la garanzia risiede nel punto di consolidamento e non nel punto di generazione del dato.

I tre meccanismi appena descritti si applicano esclusivamente agli artifact dei connector; le transazioni HTTP generate dai test nativi seguono invece un percorso separato. La redazione non avviene a posteriori sul dizionario, ma interviene direttamente al livello del modello Pydantic: un `field_validator` applicato all'attributo `EvidenceRecord.request_headers` intercetta la creazione del record e sostituisce il valore dell'header `Authorization` con [REDACTED]. Questo approccio preventivo assicura che nessuna credenziale nativa raggiunga l'`EvidenceStore` in chiaro.

4.6.3 Dual Audit Trail e Report Domain-Centric (D5.P5)

I tre deliverable finali dell'assessment assolvono funzioni distinte e si rivolgono a interlocutori diversi. Il termine *Dual* si riferisce quindi al parallelismo tra la persistenza tecnica destinata alle macchine (`evidence.json`) e la presentazione operativa destinata all'analisi umana (`assessment_report.html`).

`evidence.json` è l'archivio forense. Contiene due categorie di record unite in un'unica struttura serializzata: le transazioni HTTP che i test nativi hanno marcato come evidenza di FAIL, con il body di risposta fino a 10.000 caratteri, e gli artifact dei tool esterni, ossia gli output JSON dei connector sanitizzati e pinned tramite `pin_artifact()`. Ogni record è autosufficiente per ricostruire la probe o l'analisi che ha rivelato la vulnerabilità. Il file affianca i metadati di intestazione (`generated_at_utc`, `record_count`) alla collezione lineare dei record, creando un formato JSON nativamente compatibile con l'ingestion da parte delle piattaforme di log aggregation.

signature, codificate in Base64URL e separate da un punto.

`assessment_report.html` costituisce il deliverable operativo, renderizzato tramite il motore Jinja2⁸ in `report/renderer.py`. Il documento include le statistiche aggregate, la conformità normativa e il `TransactionSummary`: la cronologia completa delle transazioni per ciascun test, che attesta la *proof of coverage*, permettendo di verificare il perimetro interrogato dal tool e di fare triage senza aprire `evidence.json`. Il report è generato come artefatto autonomo, con stili Cascading Style Sheets (CSS) e script integrati *inline* per garantirne la consultazione offline senza dipendenze esterne. Le transazioni adottano inoltre tecniche di lazy loading per mantenere la reattività dell'interfaccia su dataset massivi.

`apiguard_report.json` fornisce la serializzazione strutturata dell'oggetto `ReportData`, il quale contiene tutti i risultati, le statistiche aggregate e i metadati della run. L'architettura garantisce una rigorosa identità strutturale tra questo documento (prodotto nativamente dalla CLI) e i dati JSON scaricabili in locale dall'interfaccia HTML del report. Questa sincronizzazione assicura che il `TransactionSummary` visionato dall'utente e i dati consumati dalle pipeline CI/CD o dai sistemi Security Information and Event Management (SIEM) coincidano esattamente, azzerando il rischio di incongruenze.

L'organizzazione visiva del report riflette la classificazione operata da `report/builder.py`, all'interno di ogni dominio di assurance, i verdetti dei test nativi precedono i risultati derivati dai tool esterni, sfruttando il campo `TestResult.source`. Questa separazione guida la lettura dell'analista, isolando le verifiche protocollari dirette dall'orchestrazione di strumenti di terze parti.

⁸Una libreria di templating per Python che consente la generazione dinamica di HyperText Markup Language (HTML) a partire da strutture dati tipizzate.

Capitolo 5

Validazione sperimentale e risultati

I Capitoli 3 e 4 hanno descritto come APIGuard Assurance è stato progettato e realizzato. La suite attiva comprende test per i domini D0, D1, D2, D3, D4, D6 e D7; il Dominio 5 (Observability) è architetturalmente predisposto ma non include test attivi in questa fase, come dichiarato nella Sezione 3.3.1. Questo capitolo porta quelle scelte architetturali a confronto con un target reale, verificando empiricamente che producano i comportamenti dichiarati.

La validazione si articola in quattro livelli con una relazione di preconditione: il testbed stabilisce le condizioni di osservazione (Sezione 5.1); la qualità ingegneristica del codebase certifica che lo strumento di misura è affidabile (Sezione 5.2); la copertura funzionale e l'idempotenza dimostrano che i risultati sono stabili e privi di effetti collaterali (Sezione 5.3); il profilo computazionale ne qualifica l'integrabilità operativa (Sezione 5.4).

Tutti i dati quantitativi di questo capitolo provengono da verifiche condotte sulla versione 0.1.0 del tool: misurazioni di sistema, esecuzioni ripetute dell'assessment e ispezioni statiche del codebase, ciascuna con una procedura o un comando riproducibile riportato nel testo.

5.1 Topologia del Testbed Sperimentale

La scelta dell'ambiente target è derivata dalla matrice di copertura della suite: serviva un'applicazione con OAS nativa non generata a posteriori, con endpoint protetti da autenticazione reale, almeno due livelli di privilegi distinti, endpoint parametrici con valori di path risolvibili, e comportamenti osservabili e stabili attraverso run successivi senza modifiche al target. Forgejo 14.0.3, esposto attraverso Kong 3.9 in modalità DB-less,

soddisfa tutti questi requisiti. La scelta conferma inoltre in modo concreto l'agnosticismo applicativo discusso nella Sezione 3.1.1: il tool non contiene nessun riferimento specifico a Forgejo: cambiare target significa aggiornare i valori nel `config.yaml` (URL del gateway, percorso della specifica, path seed, credenziali), non modificare il codice.

5.1.1 Composizione e Connettività dell'Ambiente di Test

L'ambiente di test è composto da tre servizi persistenti all'interno di un network Docker dedicato (**lab-net**): Forgejo¹ 14 come applicazione target, Kong 3.9 in modalità DB-less come gateway che riceve e instrada tutto il traffico verso Forgejo, e PostgreSQL 15-alpine come backend di persistenza richiesto da Forgejo; un quarto container effimero (**forgejo-setup**) crea gli account al primo avvio e poi termina. La topologia è illustrata nella Figura 5.1. Il traffico del tool parte dall'host, raggiunge Kong attraverso le porte mappate sul sistema, e Kong lo inoltra a Forgejo attraverso un upstream dichiarato nel file di configurazione `kong.yml`. L'intero ambiente è dichiarato in un singolo file Docker Compose ed eseguito su host di sviluppo, senza hardware dedicato né virtualizzazione separata. La Tabella 5.1 riporta le versioni fissate.

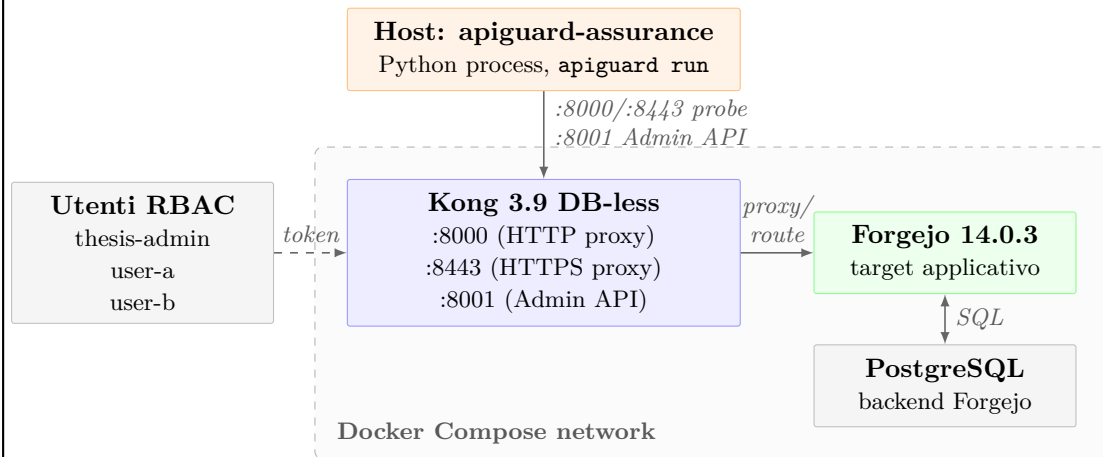


Figura 5.1: Topologia dell'ambiente di validazione sperimentale (Forgejo 14.0.3 + Kong 3.9 DB-less, Docker Compose).

Kong espone tre porte sull'host: la porta 8000 per il traffico HTTP semplice, usata dal test 1.5 per verificare il redirect verso HTTPS; la porta 8443 per il traffico HTTPS con certificato self-signed generato localmente, che costituisce l'endpoint primario per tutti gli altri test; la porta 8001 per la Admin API, che fornisce i dati di configurazione ai test `WHITE_BOX` dei Domini 3, 4 e 6. La configurazione di Kong è interamente contenuta

¹Forgejo è un servizio di hosting Git self-hosted, derivato da Gitea, con API REST documentata tramite specifica OpenAPI nativa. Espone repository, utenti, token e webhook tramite un'interfaccia HTTP standardizzata.

Componente	Versione	Ruolo nel testbed
apiguard-assurance	0.1.0	Tool sotto validazione
Python	3.12.3	Interprete del tool
Forgejo	14.0.3 (gitea-1.22.0)	Target applicativo: API REST con specifica OpenAPI nativa
PostgreSQL	15-alpine	Backend di persistenza di Forgejo (dipendenza funzionale, non componente autonomo)
Kong	3.9 DB-less	API Gateway: proxy HTTP/HTTPS, instradamento verso Forgejo, Admin API su porta 8001
nuclei	3.8.0	Tool esterno per Shadow API discovery (test <code>ext.0.1</code>)
nuclei-templates	10.4.3	Base di template di scansione versioned: garantisce che lo stesso set di firme venga usato in ogni run
testssl.sh	3.2.3	Analisi TLS black-box (test <code>ext.1.5.testssl</code>)
sslyze	(libreria Python)	Analisi TLS via libreria Python (test <code>ext.1.5.sslyze</code>)

Tabella 5.1: Componenti e versioni dell'ambiente di test.

nel file `kong.yml`, montato in sola lettura nel container: Kong legge il file a ogni avvio e opera senza database. Questo pattern, detto DB-less, è rilevante per la validazione: la configurazione è interamente nel file versionato `kong.yml` e non può essere modificata a runtime tramite l'Admin API, il che garantisce che lo stato del gateway sia identico a ogni avvio e che le verifiche `WHITE_BOX` interroghino sempre la stessa configurazione. Kong supporta anche la modalità con database, ma in quel caso la configurazione potrebbe essere alterata tra una run e l'altra.

5.1.2 Configurazione del Testbed per la Validazione Funzionale

Validare un tool di assessment su un ambiente completamente configurato in modo corretto produrrebbe una suite con tutti `PASS`: un risultato tautologico che non dimostra nulla sulla capacità del tool di rilevare violazioni reali. Il testbed è configurato perché alcuni test producano `FAIL` attesi e altri `PASS` attesi; la combinazione costituisce un banco di prova funzionalmente completo in cui il tool deve saper distinguere i due casi.

Le condizioni che producono `FAIL` osservabili e attesi nell'assessment sono le seguenti:

- Il plugin `rate-limiting` di Kong è commentato nel file `kong.yml`. Il test 4.1, che

verifica la presenza di un limite di velocità inviando richieste in loop fino a ricevere HTTP 429, registra FAIL con 2 finding. La disattivazione del rate limiting è una condizione ricorrente in ambienti non produttivi dove il throttling interferirebbe con i test applicativi, e costituisce perciò uno scenario realistico.

- La specifica OpenAPI di Forgejo dichiara a livello globale un requisito di autenticazione tramite il campo `security` del documento. Questo marcatore viene propagato a tutti gli endpoint dall'analizzatore della specifica, che li classifica come richiedenti autenticazione. Alcuni di quegli endpoint sono però genuinamente pubblici: quando il test 1.1 li interroga senza credenziali e riceve 2xx, l'oracle a tre esiti della Sezione 4.3.5 classifica il risultato come BYPASSED e produce un finding. Il verdetto è FAIL con 74 finding. Il comportamento del tool è corretto: la discrepanza tra ciò che la specifica dichiara e ciò che il target restituisce è esattamente la classe di anomalia che il contract-driven testing è progettato per rilevare. Questo caso illustra in modo concreto la dipendenza dall'oracolo descritta nella Sezione 3.1.2: quando la specifica dichiara un requisito che il target non applica completamente, i finding prodotti sono corretti dal punto di vista contrattuale ma richiedono che l'analista valuti se la discrepanza sia una configurazione intenzionale o una vulnerabilità effettiva.
- Il testbed usa un certificato TLS autofirmato generato localmente tramite lo script `certs/gen-certs.sh`. Questa è una scelta deliberata: un certificato autofirmato presenta per costruzione configurazioni al di sotto degli standard raccomandati per la produzione, suite di cifratura, durata e catena di certificazione incluse, e costituisce uno scenario realistico per ambienti di sviluppo e staging. I test `ext.1.5.sslyze` e `ext.1.5.testssl` rilevano rispettivamente 1 e 3 finding sulla configurazione del protocollo, verificando che l'analisi TLS del Dominio 1 sappia distinguere le singole debolezze e produrre finding specifici per ciascuna.

Le configurazioni che producono invece PASS attesi sono:

- Il plugin `response-transformer` inietta gli header di sicurezza HTTP su tutte le risposte: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Content-Type-Options` e `X-Frame-Options`. Il test 0.1 verifica la presenza di questi header e registra PASS.
- Il parametro `KONG_HEADERS=off` rimuove l'header `Server: kong/version` da tutte le risposte. Il test 6.2, che verifica l'assenza di fingerprinting del gateway, registra PASS.
- Il servizio `http-redirect-service` produce un redirect 301 con header `Location` corretto per qualsiasi richiesta HTTP sulla porta 8000. Il test 1.5, che verifica il redirect HTTP→HTTPS, registra PASS.

5.1.3 Struttura RBAC e Provisioning degli Utenti

Il container `forgejo-setup` crea tre account al primo avvio: `thesis-admin` (amministratore), `user-a` (utente con privilegi ordinari) e `user-b` (secondo utente non privilegiato). La struttura a tre livelli è il minimo necessario per la matrice di test: `thesis-admin` è il soggetto dei test di autenticazione che richiedono token con privilegi elevati; `user-a` è il soggetto del test 2.1 (**RBAC enforcement**), che tenta endpoint amministrativi con un token non-admin e attende 403; `user-b` è necessario per i test di isolamento tra oggetti di utenti distinti, che verificano che le risorse di `user-a` non siano accessibili con il token di `user-b`. I test 1.4 e 2.1 dipendono da token validi prodotti dal test 1.1, dipendenza dichiarata nel DAG come vincolo di Phase B e discussa nella Sezione 5.4.

Il provisioning è deterministico. Il container `forgejo-setup` usa l'interfaccia CLI di Forgejo con flag `--or-true`, così da non fallire se gli account esistono già da un avvio precedente. Questa scelta realizza concretamente la proprietà di riproducibilità discussa nella Sezione 3.1.5: il testbed raggiunge lo stesso stato iniziale indipendentemente da quante volte viene avviato, condizione necessaria perché i run successivi siano comparabili.

5.2 Qualità della Codebase come Prerequisito di Credibilità

Un tool che produce verdeti di sicurezza deve poter dimostrare che la propria implementazione risponde agli stessi criteri di qualità che applica nell'analisi del target. Una codebase con errori di tipo non rilevati può produrre verdeti formalmente corretti ma basati su dati mal classificati; dipendenze con Common Vulnerabilities and Exposures (CVE) note espongono il tool stesso ad attacchi che potrebbero alterarne i risultati; simboli pubblici privi di contratto documentato rendono l'estensione da parte di terzi una fonte di errori non rilevabili staticamente. La credibilità dell'output dipende dalla qualità del codice che lo genera, e questa sezione la stabilisce prima di discutere cosa il tool ha trovato sul target.

Le verifiche coprono tre aree: l'analisi statica della codebase tramite una suite di strumenti automatizzati (Sezione 5.2.1), la type safety a due strati che combina analisi statica con mypy e validazione a runtime con Pydantic (Sezione 5.2.2), e gli indicatori di qualità che riguardano documentazione interna, riproducibilità della build e portabilità dell'installazione (Sezione 5.2.3).

5.2.1 Analisi Statica della Codebase

La suite di analisi statica della codebase è composta da cinque strumenti con responsabilità distinte e complementari, i cui risultati sono riassunti nella Tabella 5.2:

- **ruff** svolge l'analisi di lint², l'ordinamento degli import, il rilevamento di pattern insicuri e la verifica della copertura delle annotazioni di tipo.
- **mypy** verifica la coerenza dei tipi lungo l'intera catena di dipendenze in modalità **strict**, la configurazione più restrittiva disponibile. Il suo funzionamento è discusso in dettaglio nella Sezione 5.2.2.
- **bandit** analizza la codebase alla ricerca di pattern associati a vulnerabilità note. I tre finding di bassa severità rilevati in `connectors/base.py` riguardano l'invocazione di `subprocess.run()` e sono soppressi con annotazione documentata: il comando è costruito esclusivamente da `ClassVar` dichiarati nel sorgente e da valori già validati da Pydantic, senza mai includere input HTTP grezzo dall'esterno.
- **vulture** identifica simboli pubblici definiti ma mai invocati direttamente nel codice, con soglia di confidenza all'80%. I metodi `execute()`, `model_*` e `validator_*` sono esclusi tramite whitelist: `execute()` è invocato dall'engine tramite `isinstance` check, mentre i validator Pydantic vengono attivati automaticamente dal framework al momento della costruzione di ogni oggetto, senza mai comparire come chiamate esplicite nel sorgente.
- **pip-audit** verifica le dipendenze dichiarate contro i database pubblici di vulnerabilità note.

L'analisi statica con mypy verifica che i tipi dichiarati tra moduli siano consistenti: se `execute()` dichiara di restituire `TestResult` e un punto di chiamata la tratta come `str`, mypy lo intercetta prima dell'esecuzione. Ciò che mypy non copre è il confine con l'esterno: un file `config.yaml` caricato da disco o l'output grezzo di un connector esterno entrano nel sistema come strutture non ancora tipizzate, e nessuna verifica statica può garantirne la forma. Quel secondo livello è trattato nella Sezione 5.2.2.

5.2.2 Dual-Layer Type Safety (D6.P3)

La type safety in un codebase Python ha due confini distinti con caratteristiche diverse. Il primo è interno: i contratti tra moduli, espressi tramite type hints, verificabili staticamente

²Con *lint* si intende l'analisi statica automatica del codice sorgente volta a identificare errori, violazioni di stile, pattern rischiosi e codice non conforme alle convenzioni del progetto, senza eseguire il programma.

Tool	Esito	Note
ruff	0 errori	Ruleset E/W/F/I/N/UP/B/S/ANN
mypy	0 errori	Modalità <code>strict</code> : <code>no-implicit-optional</code> , <code>disallow-untyped-defs</code> , <code>warn-return-any</code>
bandit	0 finding severity $\geq$ medium	3 Low (B404, S603x2) soppressi con <code># noqa</code> documentati: invocazione subprocess controllata per design
vulture	0 finding	Confidence $\geq 80\%$; metodi a dispatching dinamico esclusi via whitelist
pip-audit	0 CVE	Nessuna vulnerabilità nota nelle dipendenze dichiarate

Tabella 5.2: Risultati dell'analisi statica su APIGuard Assurance v0.1.0.

prima dell'esecuzione. Il secondo è esterno: il punto in cui dati non tipizzati provenienti dall'esterno, un file `config.yaml` caricato da disco o il raw output di un connector, entrano nel sistema. I due confini richiedono meccanismi diversi.

mypy in modalità `strict` verifica il confine interno sull'intera cartella `src/`: controlla che ogni funzione dichiari esplicitamente i tipi di ingresso e uscita (`disallow-untyped-defs`), che nessun parametro ammetta `None` implicitamente (`no-implicit-optional`), e che nessuna funzione restituisca `Any` dove è atteso un tipo concreto (`warn-return-any`). Questo include i modelli in `src/core/models/` (`TestResult`, `Finding`, `ConnectorResult`, `AttackSurface` e gli altri), le classi base dei test e dei connector, e tutta la logica dell'engine. Se durante lo sviluppo un campo di `TestResult` venisse rinominato o rimosso, ogni punto del codebase che vi accede genererebbe un errore mypy immediatamente, prima che il codice venga mai eseguito; allo stesso modo, se un tipo di ritorno dichiarato non corrisponde a ciò che la funzione restituisce effettivamente, mypy lo segnala al momento della modifica.

Pydantic v2 presidia il confine esterno. Ogni struttura dati che attraversa il confine del sistema, dai campi di `config.yaml` fino agli oggetti `ConnectorResult` prodotti dai tool esterni, è modellata come classe Pydantic con validatori espliciti. I `model_validator` e i `field_validator` applicano le invarianti nel momento della costruzione dell'oggetto: se nel `config.yaml` un tool esterno è marcato `enabled = True` ma il campo `timeout_seconds` è assente, la Phase 1 solleva un `ConfigurationError` con il percorso esatto del campo mancante e l'assessment non inizia. Se un connector produce un `ConnectorResult` con un campo obbligatorio mancante, Pydantic solleva un errore di validazione al momento della costruzione dell'oggetto, indicando il campo violato, prima che il dato raggiunga la logica di assessment.

I due strati hanno copertura complementare: mypy intercetta gli errori di interfaccia

tra moduli prima dell'esecuzione, Pydantic intercetta i dati malformati al confine con l'esterno al momento della costruzione. Un difetto che sfugge al primo viene catturato dal secondo; la copertura combinata riduce lo spazio degli errori che possono propagarsi in silenzio verso la logica di assessment. Per chi aggiunge un test o un connector, entrambi i meccanismi sono già attivi per costruzione: i modelli Pydantic di configurazione e di risultato sono già frozen e tipizzati, e la conformità ai type hints è verificabile con `mypy src/` senza configurazione aggiuntiva.

5.2.3 Indicatori di Qualità dell'Ingegneria di Produzione

Tre verifiche ulteriori riguardano la riproducibilità e la portabilità del tool come artefatto distribuibile, indipendentemente dall'output dell'assessment.

La prima è la copertura della documentazione interna: tutti i 292 simboli pubblici della codebase presentano una docstring, verificato tramite ispezione dell'Abstract Syntax Tree (AST)³. In un tool progettato per essere esteso, la documentazione dei contratti pubblici è il meccanismo che permette a chi aggiunge un test di determinare cosa può chiamare e con quali aspettative di comportamento.

La seconda è la riproducibilità della build: due esecuzioni consecutive del comando `hatch build --target wheel`⁴ producono un artefatto con hash SHA-256 identico (451 261 byte). Una build non deterministica introdurrebbe variabilità tra versioni nominalmente identiche, rendendo impossibile garantire che il tool in produzione corrisponda esattamente all'artefatto verificato.

La terza riguarda l'installabilità in ambienti puliti. Il pacchetto generato dalla build può essere installato in un ambiente virtuale Python privo di dipendenze preinstallate con `pip install`; al termine, `apiguard version` restituisce 0.1.0 e il tool è operativo. La verifica conferma che l'installazione non richiede dipendenze di sistema non dichiarate e che il comportamento del tool non dipende da pacchetti preinstallati nell'ambiente dell'esecutore, condizione descritta nella Sezione 3.2.6. I risultati delle verifiche sono riportati nella Tabella 5.3.

³L'Abstract Syntax Tree è la rappresentazione strutturata del codice sorgente usata dai compilatori e dagli analizzatori statici. Ispezionare l'AST permette di estrarre i simboli pubblici definiti e verificare la presenza della docstring associata senza eseguire il programma.

⁴`hatch` è il build backend del progetto. Il comando produce un file `.whl` (*wheel*), il formato standard di distribuzione per i package Python: installabile con `pip` senza compilazione, non dipende dalla piattaforma e non richiede estensioni native. Il tag `py3-none-any` nel nome lo conferma.

Aspetto	Stato	Dettaglio
Analisi statica	✓	0 errori su tutti e cinque gli strumenti (Sezione 5.2.1)
Type safety	✓	0 errori mypy strict; validazione Pydantic attiva su tutti i confini (Sezione 5.2.2)
Documentazione interna	✓	292/292 simboli pubblici con docstring (100%)
Riproducibilità della build	✓	SHA-256 identico su due build consecutive (451 261 byte)
Portabilità	✓	Cold install in ambiente pulito: <code>apiguard version</code> → 0.1.0

Tabella 5.3: Sintesi delle verifiche di qualità sulla codebase.

5.3 Copertura Funzionale e Idempotenza dell'Assessment

I test che creano risorse sul target, le interrogano e poi le eliminano introducono dipendenze di stato che, se mal gestite, producono risultati diversi tra un'esecuzione e la successiva: la prima modifica il target, la seconda trova uno stato alterato. Dimostrare empiricamente che due esecuzioni successive producono risultati identici è il modo concreto per verificare che la pipeline non lasci effetti collaterali osservabili e che le proprietà di teardown e riproducibilità, discusse nelle Sezioni 4.3.8 e 3.1.5, si comportino nel modo dichiarato su un target reale.

Le due esecuzioni sono state condotte in sequenza nella stessa giornata contro lo stesso testbed, senza alcun intervento manuale tra l'una e l'altra. Questa sezione analizza i risultati per dominio (Sezione 5.3.1) e verifica la stabilità dell'esecuzione e il corretto cleanup delle risorse (Sezione 5.3.2).

5.3.1 Analisi dei Risultati per Dominio

La mappa di copertura dell'assessment è illustrata nella Figura 5.2; il dettaglio per test, con status e finding count, è riportato nella Tabella 5.4. I 18 test attivi hanno prodotto 7 FAIL, 9 PASS e 2 SKIP per un totale di 98 finding; segue una lettura trasversale dei tre esiti.

I 7 FAIL corrispondono alle condizioni configurate nel testbed descritte nella Sezione 5.1.2. Il test 1.1 da solo produce 74 dei 98 finding totali (75.5%): quegli endpoint di Forgejo che il campo `security` della specifica marca come autenticati, ma che rispondono `2xx` a richieste prive di credenziali, sono finding corretti dal punto di vista del contratto tra specifica e comportamento osservato, come discusso nella Sezione 5.1.2. I restanti 24 finding sono distribuiti su test 0.1 (16), test 0.2 (1), test 4.1 (2), test 7.2 (1), `ext.1.5.sslyze` (1) e `ext.1.5.testssl` (3).

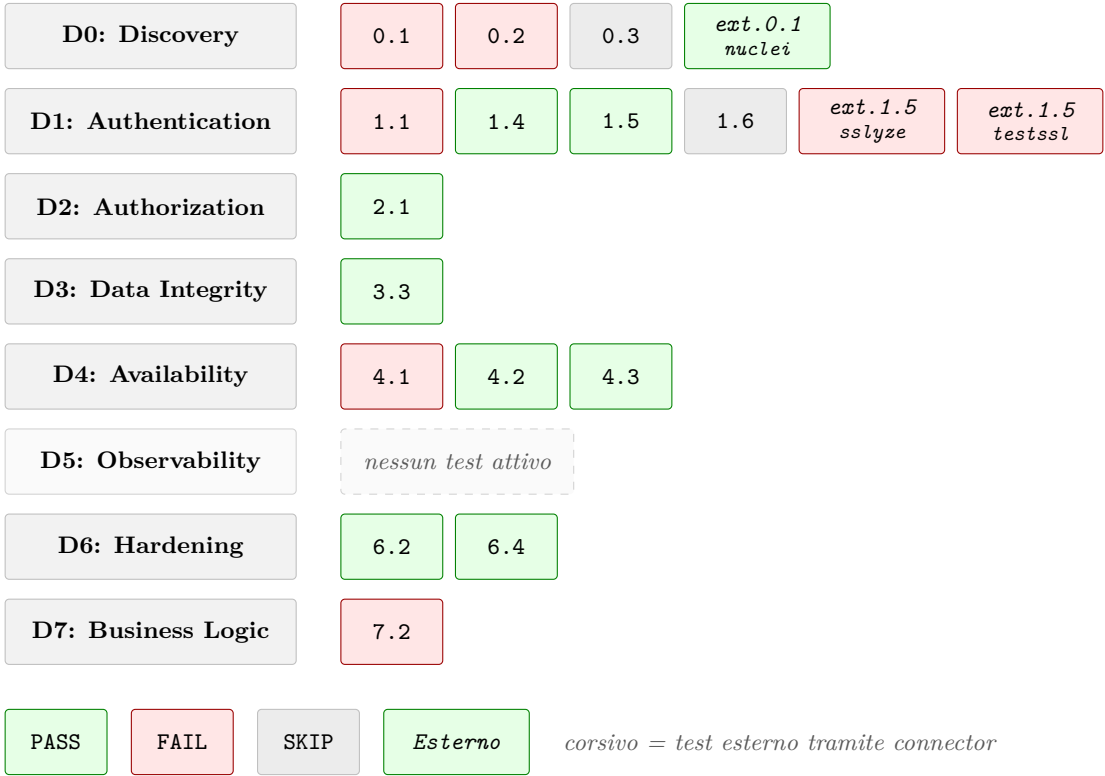


Figura 5.2: Mappa di copertura (v0.1.0): test attivi per dominio e relativi esiti.

Test ID	Dominio	Tipo	Status	Finding
0.1	D0: Discovery	Nativo	FAIL	16
0.2	D0: Discovery	Nativo	FAIL	1
0.3	D0: Discovery	Nativo	SKIP	0
1.1	D1: Auth	Nativo	FAIL	74
1.4	D1: Auth	Nativo	PASS	0
1.5	D1: Auth	Nativo	PASS	0
1.6	D1: Auth	Nativo	SKIP	0
2.1	D2: Authorization	Nativo	PASS	0
3.3	D3: Data Integrity	Nativo	PASS	0
4.1	D4: Availability	Nativo	FAIL	2
4.2	D4: Availability	Nativo	PASS	0
4.3	D4: Availability	Nativo	PASS	0
6.2	D6: Hardening	Nativo	PASS	0
6.4	D6: Hardening	Nativo	PASS	0
7.2	D7: Business Logic	Nativo	FAIL	1
ext.0.1.nuclei	D0: Discovery	Esterno	PASS	0
ext.1.5.sslyze	D1: Auth	Esterno	FAIL	1
ext.1.5.testssl	D1: Auth	Esterno	FAIL	3
Totale				98

Tabella 5.4: Status e finding count per test.

I 2 **SKIP** illustrano il comportamento del sistema in assenza di condizioni di applicabilità. Il contratto di `execute()`, descritto nella Sezione 4.3, stabilisce che ogni test produce sempre un `TestResult`: quando lo scope applicabile è vuoto o un prerequisito non è soddisfatto, il risultato è **SKIP** con motivazione esplicita, non un errore silenzioso. Il test 0.3 verifica che gli endpoint marcati `deprecated: true` nella specifica restituiscano **410 Gone** o portino un header `Sunset` (RFC 8594); la specifica OpenAPI di Forgejo 14.0.3 non dichiara nessun endpoint deprecato, e il test restituisce **SKIP** con motivo documentato. Il test 1.6 verifica la configurazione del session store tramite la Admin API di Kong; nella configurazione del testbed Kong non gestisce un session store esterno, e la condizione non è soddisfatta. In entrambi i casi uno **SKIP** con motivazione esplicita è la risposta semanticamente corretta, distinta da un **ERROR** che indicherebbe un malfunzionamento della pipeline.

I 9 **PASS** coprono le configurazioni correttamente applicate nel testbed: l'assenza di fingerprinting del gateway (test 6.2), il redirect HTTP→HTTPS (test 1.5), la revoca corretta del token (test 1.4), l'enforcement dell'autenticazione sui test `GREY_BOX` (test 2.1), la configurazione degli upstream Kong (test 6.4), i controlli di integrità dei dati e disponibilità (test 3.3, 4.2, 4.3), e il tool esterno `ext.0.1.nuclei` che non ha rilevato

Shadow API, confermando che la superficie documentata dalla specifica corrisponde a quella effettivamente esposta.

5.3.2 Idempotenza e Validazione del Teardown

La Tabella 5.5 riporta i Key Performance Indicator (KPI) aggregati delle due esecuzioni. I contatori di finding, gli status per-test, il codice di uscita e il `pass_rate_pct` sono byte-identici tra i due run: la parte deterministica del risultato non mostra alcuna variazione. Il delta sul wall-clock è di 0.67 secondi su una durata di circa 290 secondi, pari allo 0.23% della durata totale; questa variazione rientra nella fluttuazione attesa delle latenze di rete HTTP verso il target e non è attribuibile a comportamento non deterministico della pipeline.

KPI	Run 1	Run 2	Δ
PASS / FAIL / SKIP / ERROR	9 / 7 / 2 / 0	9 / 7 / 2 / 0	identico
Finding count totale	98	98	identico
<code>pass_rate_pct</code>	56.2%	56.2%	identico
<code>assessment_duration_seconds</code>	290.14 s	290.81 s	+0.23%
Exit code / label	1 / FAIL	1 / FAIL	identico

Tabella 5.5: KPI di idempotenza su due run indipendenti.

Il teardown, descritto nella Sezione 4.3.8, costituisce la preconditione dell'idempotenza: senza un cleanup completo delle risorse create durante la prima esecuzione, la seconda troverebbe uno stato alterato e produrrebbe risultati diversi. La verifica condotta dopo ciascuna delle due esecuzioni ha confermato che la Phase 6 ha drenato il registro LIFO senza lasciare risorse residue: nessun token con prefisso "`apiguard`" sull'account `thesis-admin` e nessuna repository con prefisso "`apiguard`" sull'account `user-a` erano presenti al termine di ciascuna esecuzione. Zero risorse residue su due esecuzioni indipendenti è la conferma empirica che la pipeline si comporta nel modo dichiarato in presenza di stato reale.

5.4 Footprint Computazionale e Benchmarking del DAG

L'integrabilità di un tool di assessment in una pipeline CI/CD dipende anche dal suo profilo di risorse: durata dell'esecuzione, consumo di memoria e dimensione degli output determinano su quale classe di infrastruttura il tool è operativamente sostenibile. Questa sezione riporta le misurazioni ottenute su due run di validazione.

Le metriche di sistema riportate nella Tabella 5.6 sono state ottenute tramite `/usr/bin/time -v` applicato al processo `apiguard run`. Il *wall-clock time* è il tempo reale trascorso dall'avvio alla terminazione, comprensivo di attese di rete, I/O e scheduling del sistema operativo. Lo *user time* e il *system time* misurano il tempo CPU consumato rispettivamente in user space (codice Python e librerie) e in kernel space (system call, gestione dei processi figli): entrambi sono prodotti da `/usr/bin/time -v` e la loro somma dà il tempo CPU totale effettivo.

Metrica	Run 1	Run 2
Wall-clock elapsed	290.14 s (4:50.14)	290.81 s (4:50.81)
User time	35.09 s	34.67 s
System time	41.51 s	41.62 s
Peak resident set size	287 MB	295 MB

Tabella 5.6: Metriche di sistema sui due run.

5.4.1 Regime di Esecuzione: Dominato dalla Latenza di Rete

La relazione tra user time, system time e wall-clock rivela il regime di esecuzione. Su 290 s di wall-clock, il processo ha consumato circa 35 s di tempo CPU in user space e 41 s in kernel space: la somma è 76 s, il 26% del totale. I restanti 214 s il processo li ha trascorsi in attesa delle risposte HTTP dal target, del completamento dei processi esterni (nuclei, `testssl.sh`) e dell'I/O su disco per la scrittura dell'evidence store. Il collo di bottiglia è la latenza di rete verso il target; ottimizzazioni al codice Python avrebbero un impatto marginale sul wall-clock totale. La riduzione più significativa del tempo di esecuzione verrebbe dalla parallelizzazione dei test privi di dipendenze reciproche, direzione discussa come sviluppo futuro nella Sezione 6.3.1. Con l'architettura sequenziale attuale, il profilo misurato rappresenta il tempo di esecuzione caratteristico per un assessment di questa dimensione.

Il picco di memoria di 287–295 MB e il footprint su disco di 4.5 MB ⁵ sono compatibili con le limitazioni tipiche dei sistemi di CI/CD, che operano generalmente su macchine con vincoli di memoria nell'ordine dei gigabyte. La relazione tra topologia del DAG e footprint è discussa nella Sezione 5.4.2.

⁵`apiguard_report.json` 2.26 MB, `assessment_report.html` 2.0 MB, `evidence.json` 254 KB

5.4.2 Topologia del DAG e Implicazioni sull'Ordinamento dell'Esecuzione

Eseguire tutti i test in un singolo batch senza DAG sarebbe la struttura più semplice: meno codice, nessun overhead di scheduling. Il problema è la correttezza dei risultati in presenza di dipendenze tra test.

Alcuni test richiedono che altri test abbiano già prodotto un certo stato nel `TestContext` prima di poter essere eseguiti. Nell'assessment di validazione, i test 1.4 (revoca del token) e 2.1 (enforcement RBAC) dipendono entrambi da un token valido prodotto dal test 1.1: senza quel prerequisito, i due test produrrebbero risultati che dipendono dall'ordine di esecuzione e non dal comportamento effettivo del target. Il DAG risolve questa dipendenza per costruzione: i test con `depends_on` dichiarato vengono schedulati in una fase successiva, e non possono mai essere avviati prima che il predecessore abbia prodotto un risultato.

La Tabella 5.7 riporta la topologia risultante: 16 test senza dipendenze formano la Phase A; 1.4 e 2.1 vengono schedulati in Phase B e non possono mai essere avviati prima che il test 1.1 abbia prodotto un risultato.

Fase DAG	Cardinalità	Test
Phase A (nessuna dipendenza)	16 test	0.1 · 0.2 · 0.3 · 1.1 · 1.5 · 1.6 · 3.3 · 4.1 · 4.2 · 4.3 · 6.2 · 6.4 · 7.2 · ext.0.1.nuclei · ext.1.5.testssl · ext.1.5.sslyze
Phase B (<code>depends_on</code> = ["1.1"])	2 test	1.4 · 2.1

Tabella 5.7: Topologia del DAG nell'assessment di validazione.

Il DAG di validazione è sparso: su 18 nodi, solo 2 hanno dipendenze dichiarate, entrambe convergenti sullo stesso predecessore. Con questo grado di connessione il calcolo dell'ordinamento topologico è istantaneo; la giustificazione del componente non è la complessità del caso attuale, ma la scalabilità: quando la suite crescerà a coprire più domini con dipendenze incrociate tra test di aree diverse, il grafo diventerà più denso e garantire la correttezza dell'ordinamento senza un meccanismo formale diventerà progressivamente più difficile. La validazione condotta non dimostra solo che le due dipendenze dichiarate sono gestite correttamente, ma che l'infrastruttura di scheduling è funzionante e pronta per accogliere grafi più complessi, come descritto tra gli sviluppi futuri nella Sezione 6.3.1, senza modifiche architetturali al core.

La topologia ha una relazione diretta con il footprint di memoria. Con 16 test in esecuzione sequenziale nella Phase A, il `EvidenceStore` mantiene aperto un file `.jsonl` per volta: il picco di memoria non è determinato dal numero di test paralleli ma dalla dimensione del

singolo audit trail. I 287–295 MB misurati sono il profilo effettivo di questa esecuzione. La transizione verso l'esecuzione parallela dei test all'interno della stessa fase, discussa come sviluppo futuro nella Sezione 6.3.1, richiederà una rivalutazione di questo profilo: più file `.jsonl` aperti concorrentemente comportano un footprint proporzionalmente maggiore, da misurare prima di fissare i requisiti di memoria per deployment CI/CD.

Sintesi della Validazione

Su un target reale con specifica OAS nativa, APIGuard Assurance v0.1.0 ha eseguito 18 test distribuiti su 7 domini di sicurezza, prodotto 98 finding, restituito risultati byte-identici su due run indipendenti, drenato tutte le risorse transitorie senza residui, e completato l'assessment in circa 4 minuti e 50 secondi. I quattro livelli di analisi di questo capitolo formano una gerarchia di precondizioni: ciascuno è condizione di interpretabilità del successivo, dal testbed che stabilisce le condizioni di osservazione fino al profilo computazionale che ne qualifica l'integrabilità operativa.

La validazione empirica misura comportamenti osservabili su un target reale; i limiti strutturali del paradigma adottato e le tensioni architetturali che ne derivano sono l'oggetto del Capitolo 6.

Capitolo 6

Discussione e Sviluppi Futuri

Il Capitolo 5 ha verificato che APIGuard Assurance produce i risultati dichiarati su un target reale; rimane aperta una questione di portata diversa, ovvero entro quali confini strutturali quella risposta vale e come il sistema si inserisce in contesti d'uso più ampi di quello sperimentale.

Questo capitolo affronta tale questione in tre direzioni. La prima è critica: i limiti strutturali del paradigma contract-driven, ciascuno radicato in una scelta architetturale specifica, vengono esaminati senza attenuazioni (Sezione 6.1). La seconda è applicativa: le condizioni concrete in cui il tool è integrabile in pipeline CI/CD industriali e come la variabilità di privilegio nei deployment reali influisce sulla copertura (Sezione 6.2). La terza riguarda le traiettorie future, distinte in estensioni operative che seguono direttamente dall'architettura corrente e in direzioni di ricerca che richiedono validazione sperimentale indipendente (Sezione 6.3).

6.1 Analisi Critica e Limiti del Paradigma Contract-Driven

Ogni scelta architetturale che garantisce un vantaggio specifico impone, per costruzione, una classe di situazioni in cui quel vantaggio diventa vincolo. Tre limiti strutturali emergono dall'analisi: la dipendenza dalla qualità della specifica OpenAPI come oracolo (Sezione 6.1.1), il confine intrinseco dell'astrazione del gateway (Sezione 6.1.2), e la dipendenza dall'accesso all'Admin API nei deployment cloud managed (Sezione 6.1.3).

6.1.1 Il Paradosso del Ground Truth (D3.P2)

Come discusso nella Sezione 3.1.3, la specifica OpenAPI è il fondamento contrattuale primario della pipeline: delimita la superficie di attacco, guida la costruzione delle probe

e fornisce il contratto rispetto al quale le risposte vengono interpretate. Il vantaggio è la precisione contrattuale: conoscendo il contratto, il tool costruisce richieste strutturalmente conformi che raggiungono la logica di sicurezza del target invece di fermarsi alla validazione sintattica dell'input. Questo è il *combinatorial wall* descritto nella Sezione 2.4.2, che il paradigma contract-driven supera usando la specifica formale come mappa della superficie di attacco.

Il prezzo è la dipendenza dalla qualità di ciò che si assume come oracolo. Una specifica che non riflette endpoint aggiunti dopo l'ultima pubblicazione, o che esclude deliberatamente endpoint sensibili dalla documentazione, restringe la superficie di attacco osservabile senza che il sistema possa rilevarlo: il tool non dispone di un meccanismo per distinguere un endpoint assente dalla specifica da uno assente dal sistema. La correttezza dell'assessment è in entrambi i casi condizionale alla correttezza del contratto; lo specification drift tra documentazione e implementazione reale è riconosciuto nella letteratura sul contract-driven testing come una delle cause principali di copertura incompleta, indipendentemente dallo strumento adottato. La traiettoria di ricerca che affronta strutturalmente questo paradosso è discussa nella Sezione 6.3.2.

6.1.2 La Tensione dell'Astrazione (D1.P6)

Come descritto nella Sezione 3.2.4, `BaseGatewayAdapter` è l'interfaccia che disaccoppia i test `WHITE_BOX` dal gateway concreto installato nel deployment: aggiungere il supporto per un nuovo prodotto richiede una nuova implementazione concreta senza modificare i test che la usano. Il prezzo di questa portabilità è che l'interfaccia può esporre solo ciò che è concettualmente generalizzabile a tutti i gateway che la implementano; controlli che dipendono da funzionalità vendor-specific, prive di equivalente strutturale in altri prodotti, rimangono fuori dal perimetro dell'astrazione.

Kong, ad esempio, supporta l'ordinamento esplicito dei plugin tramite un campo della sua Admin API: un plugin di autenticazione eseguito dopo un plugin di trasformazione del payload riceve dati già alterati, con implicazioni concrete sulla sicurezza. Verificare questa condizione richiede metadati specifici di Kong che non hanno corrispondente in altri gateway; includerli nell'interfaccia comune significherebbe vincolarla a una semantica vendor-specific. Il design attuale mantiene l'interfaccia generalizzabile e accetta questo come limite strutturale, non come dimenticanza. Una possibile direzione di sviluppo per superare questa tensione è descritta nella Sezione 6.3.1.

6.1.3 La Dipendenza dall'Admin API (D4.P5)

La degradazione controllata descritta nella Sezione 4.5.2 garantisce che l'assenza della Admin API non blocchi la pipeline, ma non elimina le conseguenze sulla copertura. In

ambienti cloud managed, come Amazon API Gateway, Azure API Management (APIM) o Google Cloud Endpoints, il piano di configurazione non è esposto tramite un'Admin API interrogabile direttamente: è accessibile solo attraverso le API del cloud provider, con modelli di autenticazione specifici per piattaforma. Configurare `gateway_adapter` per questi ambienti richiede un adapter dedicato per ciascun provider, attualmente non implementato.

La copertura si riduce ai domini `BLACK_BOX` e `GREY_BOX`: D0, D1, D2, D3, D4 parzialmente, D7. Il Dominio 6 (Configuration & Hardening) e le verifiche `WHITE_BOX` di D4 restano escluse. La riduzione è documentata nel report: ogni `SKIP` riporta la motivazione esplicita, rendendo la copertura effettiva immediatamente leggibile.

Questa dipendenza si manifesta in forme diverse a seconda del contesto di deployment. Nei deployment cloud managed, il piano di configurazione del gateway è accessibile solo attraverso le API del cloud provider, con modelli di autenticazione specifici per piattaforma e senza un'Admin API interrogabile direttamente: superare questo limite richiede l'implementazione di adapter dedicati, identificata come estensione operativa nella Sezione 6.3.1. In contesti di assessment formale con accesso privilegiato concordato, dove il team di sicurezza ottiene credenziali read-only con scope limitato alle risorse del progetto, la copertura `WHITE_BOX` è già raggiungibile senza modifiche al codice: è sufficiente configurare `gateway_adapter` con le credenziali e l'endpoint appropriati, a condizione che l'adapter per quel provider esista.

6.2 Applicabilità Industriale e Integrazione DevSecOps

Il tool è stato progettato per operare in contesti di integrazione continua, non solo come strumento di assessment isolato. Tre proprietà rendono concreta questa integrazione: la riproducibilità deterministica, che garantisce risultati comparabili tra esecuzioni successive; la degradazione controllata, che assicura che l'assenza di precondizioni produca output documentati senza bloccare la pipeline; e la semantica degli exit code, che fornisce un canale di comunicazione macchina-leggibile con il sistema che ospita il tool. La sezione successiva descrive come le proprietà del tool si traducono in pattern di deployment concreti: il canale di comunicazione con la pipeline (Sezione 6.2.1), i due profili di esecuzione disponibili (Sezione 6.2.2), e la variabilità di copertura in funzione del contesto organizzativo (Sezione 6.2.3).

6.2.1 Semantic Exit Codes come Canale di Comunicazione con la Pipeline (D6.P1)

Al termine di ogni esecuzione, il tool comunica il proprio esito attraverso quattro codici di uscita con significato preciso: 0 indica che tutti i test attivi hanno prodotto PASS o SKIP; 1 indica che almeno un test ha rilevato una violazione (FAIL); 2 indica che nessun test ha rilevato violazioni ma almeno uno ha prodotto ERROR, segnalando un malfunzionamento parziale della pipeline; 10 indica un errore infrastrutturale nelle fasi di avvio che ha impedito l'esecuzione dei test. Quando un assessment presenta sia FAIL che ERROR, il codice restituito è 1: le violazioni rilevate prevalgono sui malfunzionamenti parziali. Il routing basato su questi quattro codici è schematizzato nella Figura 6.1.

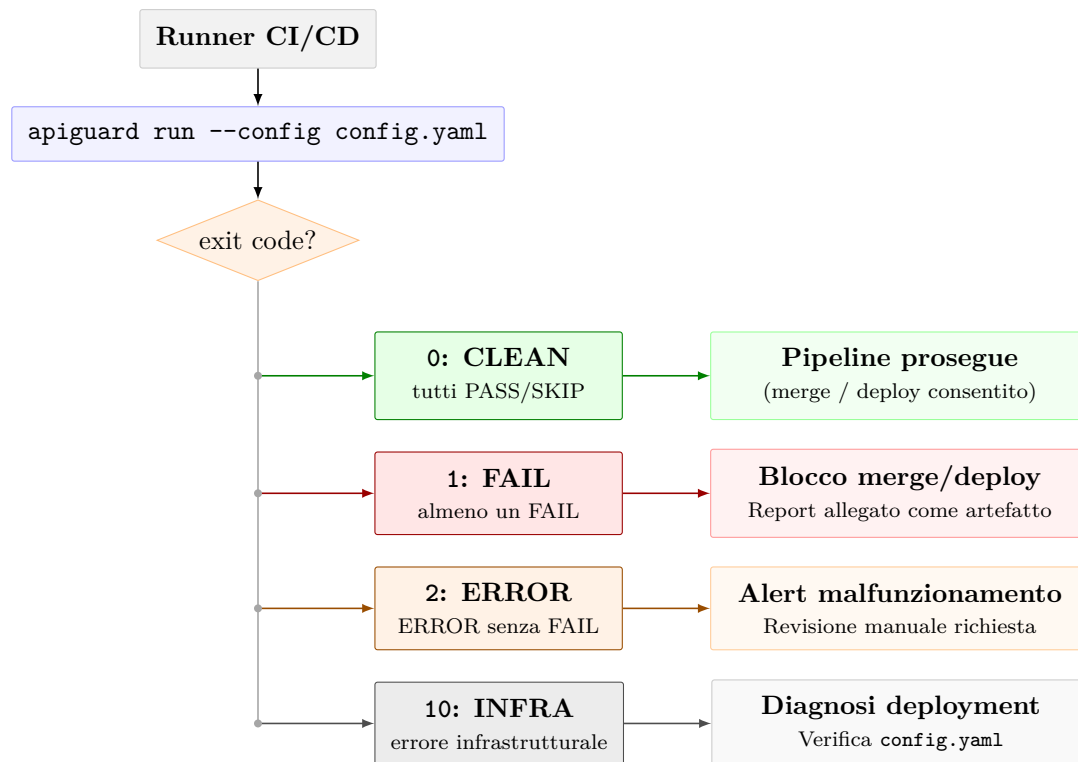


Figura 6.1: Routing degli exit code semantici (D6.P1) nella pipeline CI/CD.

La separazione tra 1 e 10 ha un valore operativo preciso: rilevare una violazione e non avviare l'assessment sono eventi con cause e azioni correttive radicalmente diverse. Il primo richiede di analizzare e correggere il target; il secondo richiede di diagnosticare il deployment della pipeline. Con codici semanticamente distinti questo routing è automatizzabile senza leggere i log.

Sistemi di CI/CD come GitHub Actions o GitLab Continuous Integration (CI)¹ trattano qualsiasi exit code diverso da zero come fallimento del passo: un'esecuzione di `apiguardrun` che restituisce exit 1 blocca la pipeline senza configurazione aggiuntiva, e il report rimane disponibile come artefatto per l'analisi post-hoc. Il gate decisionale è il codice di uscita; il report fornisce il dettaglio dei finding per la remediation.

La granularità attuale tratta ogni FAIL con lo stesso peso: una violazione P0 su un endpoint di autenticazione e una P3 su una configurazione subottimale producono entrambe exit 1. Rendere il predicato di blocco configurabile per livello di priorità è un'estensione operativa discussa nella Sezione 6.3.1.

6.2.2 Quality Gate a Due Velocità: Fail-Fast e Assessment Completo (D4.P8)

Il tool supporta due profili di esecuzione distinti configurabili sullo stesso file. Il comportamento di default esegue tutti i test attivi indipendentemente dagli esiti intermedi e produce un report completo su tutti i domini: è appropriato per gli assessment periodici su versioni consolidate del software, dove la completezza dell'evidenza ha valore autonomo e ogni dominio deve produrre risultati per supportare le decisioni di correzione.

La seconda modalità, attivata con `execution.fail_fast: true`, risponde a un'esigenza diversa. In un workflow di integrazione continua in cui ogni proposta di modifica viene verificata prima di essere integrata nel ramo principale, attendere il completamento di un assessment da cinque minuti per bloccare su una violazione rilevabile nei primi test P0 è un costo non proporzionale alla decisione. Con `fail_fast` attivo, la prima violazione confermata su un test di priorità P0 interrompe l'esecuzione e il processo termina con exit 1 senza avviare i test rimanenti. La condizione di interruzione si applica anche agli **ERROR** sui test P0, perché una verifica perimetrale incompleta per malfunzionamento è operativamente equivalente a una violazione: in entrambi i casi il controllo non produce la garanzia attesa. Il wall-clock si riduce da circa cinque minuti a pochi secondi nel caso peggiore.

6.2.3 Variabilità di Privilegio nei Deployment Industriali

Il meccanismo del box gradient descritto nella Sezione 3.3.2 ha una conseguenza operativa diretta nei deployment industriali: la profondità dell'assessment si adatta automaticamente alle precondizioni di accesso disponibili nel contesto organizzativo specifico, senza richiedere versioni diverse del tool o profili di configurazione strutturalmente distinti. La

¹GitHub Actions e GitLab CI sono piattaforme di integrazione continua che eseguono pipeline di build, test e deployment automatizzato al variare del codice sorgente. Entrambe trattano qualsiasi exit code diverso da zero come fallimento del passo, bloccando l'avanzamento della pipeline.

variazione è nella configurazione fornita nel `config.yaml`, che riflette le precondizioni effettivamente disponibili, non una scelta esplicita di modalità. Lo stesso tool che in un ambiente con accesso completo esegue l'intera matrice di domini riduce automaticamente la propria copertura in contesti più vincolati, documentando ogni riduzione tramite `SKIP` con motivazione esplicita: l'analista vede esattamente quali garanzie sono state verificate e quali non erano applicabili nel contesto dato.

6.3 Sviluppi Futuri

Le traiettorie di sviluppo si articolano su due scale temporali con natura distinta. Le estensioni operative seguono direttamente dall'architettura corrente: i punti di estensione sono già predisposti, i contratti tra i componenti sono già definiti, e la mancata implementazione riflette scelte di priorità, non vincoli architetturali. Le traiettorie di ricerca pongono invece problemi aperti che richiedono analisi formale o validazione sperimentale indipendente prima di poter essere affrontati concretamente, e in alcuni casi costituiscono oggetti di studio autonomi.

6.3.1 Estensioni Operative

L'architettura corrente è stata progettata per essere estesa: la tassonomia a otto domini definisce uno spazio di coverage più ampio di quello attualmente coperto, i contratti di `BaseTest` e `BaseConnector` permettono l'aggiunta di test e tool esterni senza modificare il core, e il DAG scheduler è già predisposto per accogliere nuove dipendenze. Le estensioni descritte di seguito completano la copertura della suite e rafforzano l'integrazione con le pipeline industriali; in tutti i casi, i punti di ancoraggio nell'architettura sono predisposti.

Copertura dei test e connectors. Sul fronte della copertura dei domini, il Dominio 5 (Observability) è l'unico senza test attivi nella fase corrente: i test 5.1 (audit logging) e 5.2 (alert real-time) sono già specificati nella metodologia e richiedono l'aggiunta di un log aggregator e un sistema di alerting al testbed, componenti che rimangono architetturalmente isolati dal core. Per i Dominii già parzialmente coperti, i test pianificati ma non ancora implementati si inseriscono nel DAG esistente senza interventi sull'ordinamento topologico: i test 1.2 e 1.3 sul ciclo di vita del JWT dichiarano `depends_on = ["1.1"]` e attendono il token prodotto da quel test, esattamente come già accade per i test 1.4 e 2.1 in Phase B. Il Dominio 2 (Authorization) copre attualmente solo il test 2.1; i test 2.2–2.5 con il connector OFFAT per la generazione dinamica di probe BOLA/Insecure Direct Object Reference (IDOR) completerebbero la copertura del controllo accessi a livello di oggetto. i

Domini 3, 6 e 7 hanno test pianificati che richiedono rispettivamente **Schemathesis** [15]² per l'injection testing, un connector socket per HTTP request smuggling, e connector **Vegeta** e **Interactsh** per race condition e SSRF blind.

Due di questi connector hanno caratteristiche tecniche che vale la pena evidenziare. Il **VegetaConnector** per il test 7.3 (Time-of-Check to Time-of-Use (TOCTOU)) risolve un limite strutturale del runtime Python: le richieste simultanee con sincronizzazione sub-millisecondo richieste da questo test sono inaffidabili con il Global Interpreter Lock (GIL) attivo; delegare la generazione del carico a **Vegeta**, scritto in Go con goroutine native, ottiene la finestra di concorrenza necessaria. Il connector è riutilizzabile dal test 4.1 per il load testing del rate limiting, in coerenza con il principio di separazione dati/valutazione descritto nella Sezione 4.4.1. L'**InteractshConnector** per il test `ext.7.4` affronta invece un problema di osservabilità: le vulnerabilità SSRF blind non producono output osservabile nel canale normale, e richiedono un oracle fuori banda. Il connector registra un indirizzo controllato su un server **interactsh** remoto, lo inietta nel probe e verifica se il target ha effettuato una richiesta verso quell'indirizzo; l'evidenza è nel callback ricevuto, non nella risposta al probe.

Integrazione della pipeline. Due estensioni riguardano il canale di comunicazione con la pipeline CI/CD, già descritto nella Sezione 6.2.1. La prima è la granularità degli exit code per livello di priorità, che permetterebbe di distinguere violazioni P0 (bloccanti in qualsiasi scenario) da violazioni P2/P3 (segnalabili come avvertimenti senza bloccare il rilascio). La seconda è l'implementazione degli adapter per i cloud provider principali (Amazon API Gateway, Azure APIM), che consentirebbe la copertura **WHITE_BOX** nei deployment cloud managed discussi nella Sezione 6.1.3.

Gerarchia estesa dell'adapter. La tensione descritta nella Sezione 6.1.2 tra portabilità e profondità delle verifiche vendor-specific ammette una direzione di sviluppo che non richiede modifiche al contratto esistente. **BaseGatewayAdapter** rimarrebbe l'interfaccia comune con i metodi generalizzabili; classi intermedie come **BaseKongAdapter** o **BaseAWSAdapter** esporrebbero metodi vendor-specifici per i test che li richiedono. Un deployment che analizza un gateway Kong istanzierebbe **KongGatewayAdapter**; uno che analizza un gateway AWS istanzierebbe il relativo adapter; entrambi sarebbero selezionabili via configurazione, seguendo lo stesso meccanismo di iniezione dell'adapter attuale. Questa struttura riflette la stessa logica della gerarchia dei connector descritta nella Sezione 4.4.1. Il lavoro necessario riguarda due aspetti: la definizione formale del confine tra metodi generalizzabili, che appartengono a **BaseGatewayAdapter** e sono disponibili a tutti i test, e metodi vendor-specifici, che appartengono al livello intermedio e sono accessibili solo ai test che ne hanno bisogno; e la gestione della compatibilità

²L'uso di **Schemathesis** come connector esterno è distinto dall'uso come strumento autonomo discusso nella Sezione 2.4.3: qui opera come fornitore di dati grezzi, la cui valutazione rimane responsabilità del test.

quando l'architettura evolve, affinché le implementazioni esistenti rimangano valide senza richiedere aggiornamenti in tutto il codebase.

Ottimizzazioni del motore. La struttura a batch del DAG è già parallelizzabile per costruzione, come discusso nella Sezione 4.2.2: i test all'interno di uno stesso batch non hanno dipendenze reciproche. Introdurre un `ThreadPoolExecutor` nella Phase 5 ridurrebbe il wall-clock proporzionalmente al numero di test eseguibili in parallelo; dalla distribuzione della topologia osservata nella Sezione 5.4.2, 16 dei 18 test attivi appartengono alla Phase A e sarebbero candidati alla parallelizzazione. I problemi di thread-safety che questa modifica introduce sulle strutture condivise sono l'oggetto del paragrafo successivo.

6.3.2 Traiettorie di Ricerca Aperta

Le direzioni descritte in questa sezione condividono una caratteristica: l'architettura corrente le abilita o le porta alla luce, ma la loro implementazione richiede analisi formale, scelte di design nuove o validazione sperimentale indipendente. Alcune sono evoluzioni dirette di meccanismi esistenti; altre aprono domande di ricerca autonome.

Parallelizzazione intra-batch e thread-safety. La parallelizzazione intra-batch, identificata come estensione operativa nella Sezione 6.3.1, porta alla luce due problemi di thread-safety che richiedono analisi formale prima dell'implementazione. Il primo problema riguarda l'integrazione dei risultati: ogni test scrive le proprie transazioni su un file `.jsonl` separato nella directory temporanea, ma il merge di questi file nell'evidence store finale e il calcolo del `ResultSet` aggregato avvengono su strutture condivise; con più test concorrenti che completano in ordine non deterministico, la correttezza di queste operazioni di merge richiede coordinazione esplicita. Il secondo problema, più ampio, riguarda la gestione dello stato condiviso nel `TestContext`: token acquisiti, risorse registrate per il teardown e path seed risolti vengono scritti e letti da test distinti; con esecuzione concorrente, accessi non coordinati a queste strutture possono produrre race condition il cui effetto dipende dall'ordine di schedulazione e non dal comportamento del target. Prima di affrontare la scelta della strategia di coordinazione, andrebbe misurato il guadagno effettivo della parallelizzazione nel contesto specifico: con un'esecuzione dominata dalla latenza di rete verso il target, come osservato nella Sezione 5.4.2, il beneficio dipende dalla distribuzione dei tempi di risposta e dalla possibilità di sovrapporre le attese. Senza questa misurazione, la complessità introdotta dalla gestione della concorrenza potrebbe non essere giustificata.

APIGuard come piattaforma modulare. L'architettura corrente è costruita attorno al paradigma REST/OAS, che costituisce l'unico protocollo per cui il tool produce

assessment completi. Classi di API con semantiche diverse, endpoint GraphQL, servizi gRPC, sistemi event-driven basati su AsyncAPI, richiedono meccanismi di scoperta della superficie di attacco, costruzione delle probe e interpretazione delle risposte radicalmente diversi. La traiettoria di ricerca è trasformare APIGuard Assurance da tool monoprodotto a piattaforma di assurance modulare, come illustrato nella Figura 6.2: il core, composto dall'infrastruttura di orchestrazione, dai modelli dati condivisi e dai contratti astratti di plugin, rimarrebbe invariato, mentre i moduli di protocollo, incluso quello REST/OAS attuale, verrebbero trattati come plugin intercambiabili. In questo modello, un modulo per GraphQL contribuirebbe i propri test e connector rispettando i contratti di `BaseTest` e `BaseConnector`, un operatore potrebbe comporre la suite combinando moduli per i protocolli rilevanti nel proprio contesto, e l'architettura del core rimarrebbe invariata. Le sfide di ricerca principali riguardano due fronti: la specifica formale del contratto tra modulo e core, che deve garantire compatibilità al variare delle versioni e permettere l'aggiunta di moduli senza richiedere modifiche al core; e la definizione di un modello di evidenza sufficientemente astratto da rappresentare semantiche di protocollo diverse, garantendo che i finding prodotti da moduli distinti siano confrontabili nel report finale.

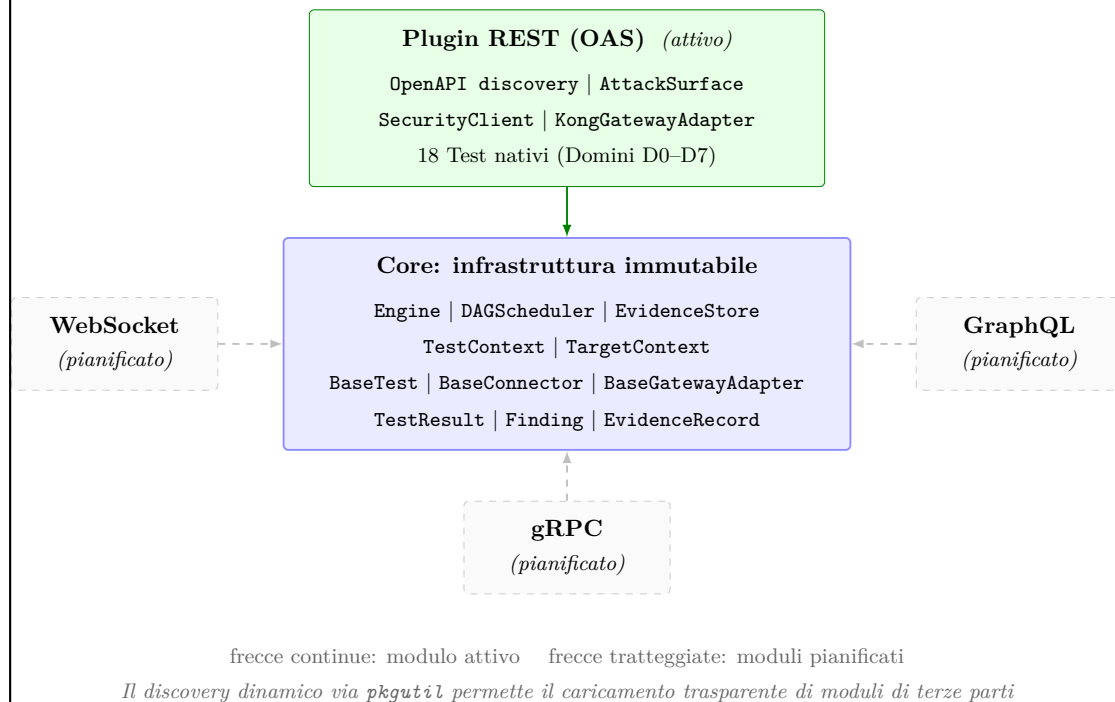


Figura 6.2: Evoluzione di APIGuard Assurance verso una piattaforma modulare: separazione netta tra il core invariante e i plugin di protocollo.

Security testing e oracoli di sicurezza come sfide complementari. La letteratura documenta che il security testing delle API REST è l'ambito strutturalmente meno

sviluppato nel campo [2]; le due sfide sono intrinsecamente collegate: l'automazione del security testing richiede oracoli in grado di valutare garanzie di sicurezza, la cui costruzione richiede a sua volta la conoscenza del dominio da verificare. Definire criteri formali e automatizzabili per garanzie di sicurezza, al di là della conformità funzionale, è un problema di ricerca aperto che la letteratura ha appena cominciato ad affrontare.

Coverage augmentation progressiva. La specifica OpenAPI definisce la superficie di attacco con cui il tool inizia ogni run, ma durante l'esecuzione il sistema raccoglie osservazioni sul comportamento reale del target che potrebbero estendere quella superficie. I test del Dominio 0 scoprono endpoint attivi non dichiarati nella specifica, i test di autenticazione producono token e schemi di risposta osservabili, i test a visibilità piena leggono la configurazione effettiva del gateway. La traiettoria di ricerca è usare queste osservazioni come input per i test successivi nella stessa run, trasformando la superficie di attacco da un'istantanea statica iniziale a una struttura che si arricchisce progressivamente al procedere dell'esecuzione: un endpoint scoperto nei test iniziali potrebbe alimentare i probe dei test successivi, che attualmente lo ignorano perché non dichiarato nella specifica di partenza. La sfida centrale è metodologica: definire quali osservazioni siano abbastanza affidabili da essere usate come input per test successivi, come propagarle nel DAG garantendo la riproducibilità, e come distinguere endpoint genuinamente non documentati da endpoint inaccessibili per mancanza di permessi.

Capitolo 7

Conclusioni

Il Capitolo 1 ha individuato tre limitazioni centrali negli approcci esistenti al security assessment delle API REST e ha formulato tre obiettivi di ricerca corrispondenti. Questo capitolo verifica se quegli obiettivi sono stati raggiunti e ne valuta la portata effettiva, collocando il contributo nel contesto di un approccio alla sicurezza che sia sistematico, ripetibile e integrabile nel ciclo di sviluppo.

7.1 Sintesi del Lavoro

Il punto di partenza è la constatazione che le vulnerabilità semantiche delle API REST si manifestano come richieste formalmente corrette che violano le regole di accesso del sistema, e per questa ragione sfuggono agli strumenti che cercano anomalie sintattiche nella forma delle richieste. Rilevarle in modo sistematico richiede la conoscenza del contratto che regola ogni interazione, da usare come criterio di valutazione delle risposte.

Da questo requisito derivano le scelte architetturali centrali del lavoro: la specifica OAS e il file di configurazione sono le due sole sorgenti da cui il tool ricava a runtime tutta la conoscenza del target: la prima fornisce la topologia degli endpoint, gli schemi dei parametri e i codici di stato attesi; il secondo porta la conoscenza concreta del deployment, come i valori degli endpoint parametrici, le credenziali dei ruoli e l'indirizzo del gateway. L'assenza di riferimenti hardcoded a qualsiasi sistema specifico è la condizione che rende il tool applicabile a qualsiasi API REST documentata senza modifiche al codice. Lo scheduling basato su DAG garantisce la correttezza dell'ordinamento anche quando i test hanno dipendenze reciproche, e ogni test dichiara nel proprio contratto i riferimenti normativi della garanzia che verifica, propagati al report come evidenza tracciabile verso categorie OWASP, codici CWE e standard applicabili.

La validazione empirica, condotta su Forgejo 14.0.3 esposto tramite Kong 3.9 DB-less, ha verificato che queste scelte producono i comportamenti dichiarati: 18 test su sette domini di sicurezza, 98 finding, risultati byte-identici su due run indipendenti e teardown completo senza risorse residue. Il Capitolo 5 riporta i dati con tracciabilità completa; il Capitolo 6 analizza i limiti strutturali del paradigma adottato e le traiettorie di sviluppo futuro.

7.2 Risoluzione dei Gap e Verifica degli Obiettivi

L'obiettivo O₁ Tool Contract-Driven Agnostico, in risposta al Gap G₁ Cecità Semantica, si realizza attraverso due meccanismi complementari: l'agnosticismo applicativo descritto nella Sezione 3.1.1, per cui tutta la conoscenza del target viene derivata a runtime dalla specifica e dalla configurazione senza riferimenti hardcoded, e l'uso della OAS come oracolo formale descritto nella Sezione 3.1.2, che governa la costruzione delle probe e la valutazione delle risposte rispetto al contratto dichiarato. La combinazione dei due consente di costruire probe che raggiungono la logica semantica del target: una richiesta autenticata con un token già revocato, un accesso a una risorsa con credenziali insufficienti per quel ruolo specifico, un campo sensibile restituito a un client che non avrebbe dovuto riceverlo. La tracciabilità normativa dei risultati è una proprietà distinta, realizzata dal contratto di ogni test: ogni classe di test dichiara nel proprio codice sorgente, come metadato fisso, i riferimenti a categorie OWASP, codici CWE e standard applicabili, che vengono propagati a ogni **Finding** nel report senza intervento manuale.

L'obiettivo O₂ Superamento del Compromesso Portabilità-Profondità, in risposta al Gap G₂ Portabilità e Profondità Semantica, è una conseguenza diretta della stessa architettura: un tool che deriva a runtime tutta la conoscenza del target dalla sua specifica e dalla configurazione, senza dipendenze hardcoded, è applicabile a qualsiasi sistema REST documentato senza modifiche al codice. La profondità dell'analisi si preserva perché le verifiche sono costruite e valutate rispetto alla specifica del target, con la stessa precisione indipendentemente dall'applicazione che la produce. La scelta di Forgejo 14.0.3 come target di validazione, selezionato per le sue proprietà strutturali e non per affinità con il tool, ne è la dimostrazione concreta.

L'obiettivo O₃ Determinismo e Integrazione nel Ciclo Continuativo, in risposta al Gap G₃ Riproducibilità Deterministica, è validato dai dati della Sezione 5.3: due run indipendenti con la stessa configurazione sullo stesso target hanno prodotto risultati byte-identici per tutti i contatori aggregati. Il determinismo dell'output, realizzato attraverso i meccanismi descritti nella Sezione 3.1.5, è la condizione che rende la verifica affiancabile al ciclo di sviluppo continuativo tramite exit code semantici, senza richiedere interpretazione manuale tra un'esecuzione e la successiva.

7.3 Contributi Metodologici e Ingegneristici

I contributi di questo lavoro si articolano su due piani distinti: la formalizzazione metodologica, che produce strumenti analitici riutilizzabili indipendentemente dall'implementazione specifica, e la realizzazione ingegneristica, che ne dimostra la concreta praticabilità.

Sul piano metodologico, la tassonomia a otto domini di assurance, descritta nella Sezione 3.3.1, organizza la superficie di sicurezza di un'API esposta tramite gateway in categorie semanticamente distinte, ciascuna con i propri criteri di valutazione e le proprie priorità, applicabile a qualsiasi assessment strutturato indipendentemente dagli strumenti impiegati. Il box gradient, formalizzato nella Sezione 3.3.2, aggiunge la dimensione del privilegio: ogni test dichiara la propria strategia di esecuzione (**BLACK_BOX**, **GREY_BOX** o **WHITE_BOX**) come metadato fisso, indicando il tipo di accesso necessario per eseguirlo. I tre livelli si traducono in altrettante profondità di copertura: i test **BLACK_BOX** sono eseguibili senza credenziali e simulano la prospettiva di un attaccante esterno; i test **GREY_BOX** richiedono credenziali applicative per i ruoli configurati; i test **WHITE_BOX** richiedono accesso al piano di controllo del gateway e producono la copertura più ampia. Le scelte architetturali che realizzano queste proprietà, trattate nei Capitoli 3 e 4, documentano per le decisioni rilevanti le motivazioni e i trade-off accettati, con l'obiettivo di rendere il modello di design comprensibile e adattabile a sistemi analoghi.

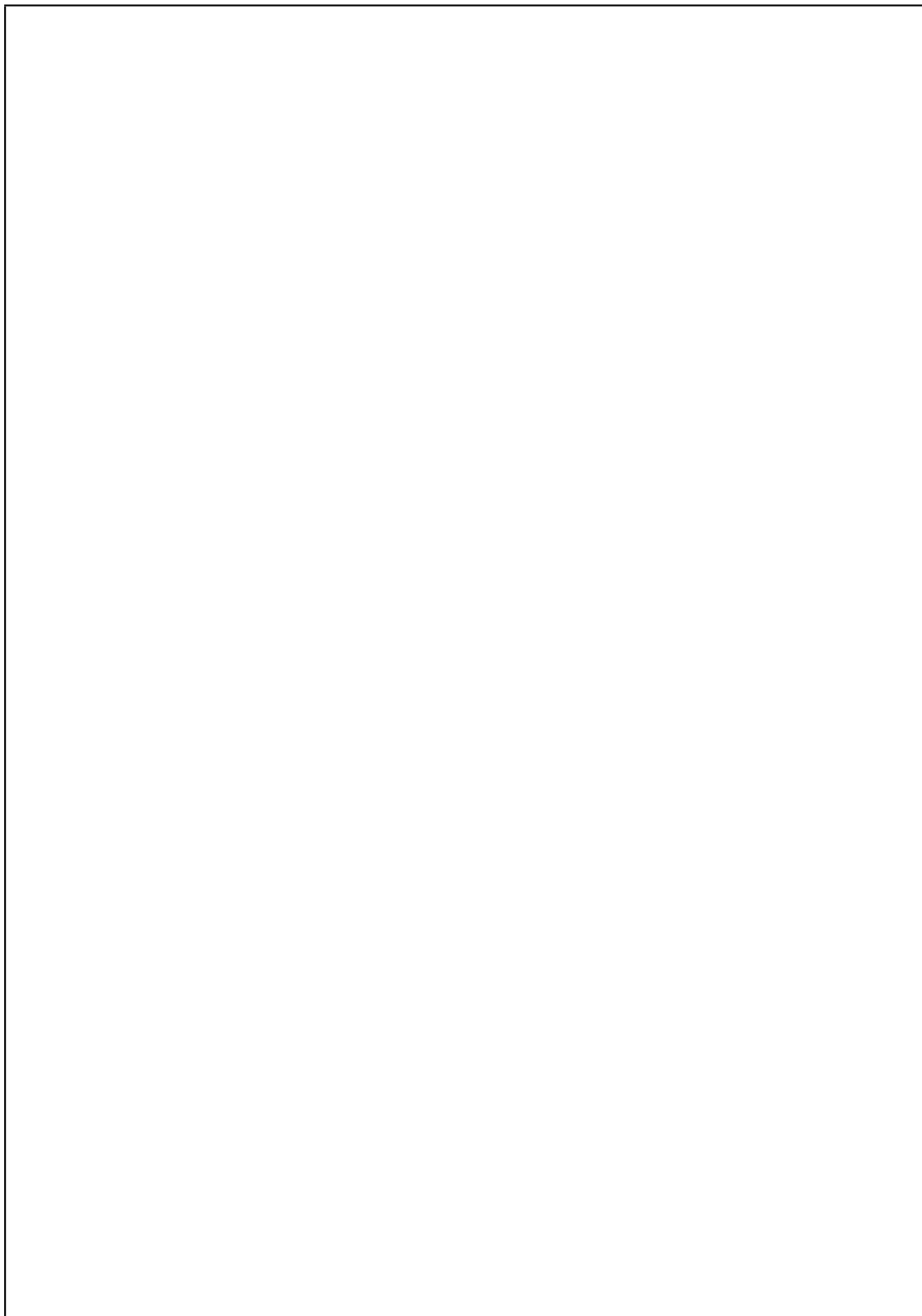
Sul piano ingegneristico, il design di un assessment engine deterministico basato su DAG, descritto nella Sezione 3.2.2, costituisce un contributo architetturale autonomo: la struttura a batch con risoluzione topologica garantisce la correttezza semantica dell'ordine di esecuzione in presenza di dipendenze tra test, una proprietà che uno scheduler a priorità statica non raggiunge per costruzione.

7.4 Considerazioni Finali sull'Assurance Continuo

Quando la valutazione della sicurezza diventa un processo continuo, integrato nel ciclo di sviluppo e ripetuto a ogni variazione del sistema, cessa di essere un attributo certificato in un momento specifico e diventa una proprietà del processo stesso, verificabile a ogni rilascio in modo automatico e confrontabile con le esecuzioni precedenti. Questa trasformazione richiede che le verifiche siano deterministiche, indipendenti dall'operatore e automaticamente interpretabili dalla pipeline che le ospita: condizioni che un assessment point-in-time condotto da un operatore non può soddisfare per costruzione, e che un tool config-driven con exit code semantici invece realizza direttamente.

Il penetration testing tradizionale produce un'osservazione puntuale condotta in un momento specifico, da un operatore con un insieme specifico di conoscenze e tecniche: rieseguire il test in condizioni nominalmente identiche produce output confrontabili solo per convenzione, perché quella variabilità è incorporata nel metodo. L'approccio realizzato in questo lavoro produce invece risultati deterministicamente identici a ogni riesecuzione con la stessa configurazione, verificabili da chiunque senza ricostruire il processo di chi li ha generati. La differenza non è di scala, ma di natura: da osservazione a garanzia misurabile.

Questo lavoro mostra che un assessment automatizzato, deterministico e integrabile nel ciclo di sviluppo è praticabile su un target con caratteristiche reali: specifica OpenAPI nativa, autenticazione effettiva e superfici di attacco concrete. Ogni verdetto è tracciabile agli standard normativi, ogni esecuzione produce un audit trail leggibile, e l'integrazione nella pipeline avviene tramite exit code semantici senza richiedere intervento manuale tra un rilascio e il successivo.



Bibliografia

- [1] OWASP Foundation, “OWASP Top 10 API Security Risks – 2023,” 2023. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [2] A. Golmohammadi, M. Zhang, and A. Arcuri, “Testing RESTful APIs: A Survey,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 1, pp. 27:1–27:41, Nov. 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3617175>
- [3] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: Yesterday, Today, and Tomorrow,” in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Cham: Springer International Publishing, 2017, pp. 195–216. [Online]. Available: http://link.springer.com/10.1007/978-3-319-67425-4_12
- [4] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [5] R. Chandramouli and Z. Butcher, “Guidelines for API Protection for Cloud-Native Systems,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication (SP) 800-228, Mar. 2026. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/228/upd1/final>
- [6] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” National Institute of Standards and Technology, Tech. Rep., Aug. 2020. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf>
- [7] V. Atlidakis, P. Godefroid, and M. Polishchuk, “RESTler: Stateful REST API Fuzzing,” in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. Montreal, QC, Canada: IEEE, May 2019, pp. 748–758. [Online]. Available: <https://ieeexplore.ieee.org/document/8811961/>
- [8] —, “Checking Security Properties of Cloud Service REST APIs,” in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification*

- (*ICST*). Porto, Portugal: IEEE, Oct. 2020, pp. 387–397. [Online]. Available: <https://ieeexplore.ieee.org/document/9159084/>
- [9] O. Sahin, M. Zhang, and A. Arcuri, “Enhancing REST API Fuzzing with Access Policy Violation Checks and Injection Attacks,” Apr. 2026. [Online]. Available: <http://arxiv.org/abs/2604.00702>
- [10] D. Corradini, M. Pasqua, and M. Ceccato, “Automated Black-Box Testing of Mass Assignment Vulnerabilities in RESTful APIs,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, May 2023, pp. 2553–2564. [Online]. Available: <https://ieeexplore.ieee.org/document/10172607>
- [11] ZAP Core Team, “ZAP: Zed Attack Proxy by Checkmarx.” [Online]. Available: <https://www.zaproxy.org/>
- [12] W. Du, J. Li, Y. Wang, L. Chen, R. Zhao, J. Zhu, Z. Han, Y. Wang, and Z. Xue, “Vulnerability-oriented Testing for RESTful APIs,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 739–755. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/du>
- [13] A. Mazidi, D. Corradini, and M. Ghafari, “Mining REST APIs for Potential Mass Assignment Vulnerabilities,” in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE ’24. New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 369–374. [Online]. Available: <https://dl.acm.org/doi/10.1145/3661167.3661204>
- [14] G. Deng, Z. Zhang, Y. Li, Y. Liu, T. Zhang, Y. Liu, G. Yu, and D. Wang, “NAUTILUS: Automated RESTful API Vulnerability Detection,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 5593–5609. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/deng-gelei>
- [15] Z. Hatfield-Dodds and D. Dygalo, “Deriving semantics-aware fuzzers from web API schemas,” in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, ser. ICSE ’22. New York, NY, USA: Association for Computing Machinery, Oct. 2022, pp. 345–346. [Online]. Available: <https://dl.acm.org/doi/10.1145/3510454.3528637>
- [16] J. C. Alonso, S. Segura, and A. Ruiz-Cortés, “AGORA: Automated Generation of Test Oracles for REST APIs,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. New York, NY, USA: Association for Computing Machinery, Jul. 2023, pp. 1018–1030. [Online]. Available: <https://dl.acm.org/doi/10.1145/3597926.3598114>

- [17] J. C. Alonso, M. D. Ernst, S. Segura, and A. Ruiz-Cortés, “Test Oracle Generation for REST APIs,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 1, pp. 19:1–19:37, Dec. 2025. [Online]. Available: <https://dl.acm.org/doi/10.1145/3726524>
- [18] J. C. Alonso, A. Martin-Lopez, S. Segura, G. Bavota, and A. Ruiz-Cortés, “SATORI: Static Test Oracle Generation for REST APIs,” in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Nov. 2025, pp. 1364–1376. [Online]. Available: <https://ieeexplore.ieee.org/document/11334625>
- [19] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The Oracle Problem in Software Testing: A Survey,” *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, May 2015. [Online]. Available: <https://ieeexplore.ieee.org/document/6963470>